



UNIVERZITET CRNE GORE
PRIRODNO-MATEMATIČKI FAKULTET
RAČUNARSTVO I INFORMACIONE TEHNOLOGIJE
PODGORICA

Seminarski rad

SORT ALGORITAM

Predmet: Paralelni algoritmi

Profesor:

dr Igor Jovančević

Asistent:

mr Nikola Pižurica

Studenti:

Milica Bjelica

Krsto Kustudić, 9/24

SADRŽAJ:

1.	UVOD	1
2.	Bubble SORT ALGORITAM	2
2.1	Implementacija algoritma.....	2
2.2	Složenost algoritma.....	4
2.2.1	Vremenska složenost(Time complexity)	4
2.2.2	Prostorna složenost (space complexity)	5
2.3	Paralelizacija algoritma	6
2.4	Implementacija u okviru projekta.....	9
2.4.1	Analiza rezultata	11
3.	SELECTION SORT ALGORITAM	14
3.1	Implementacija algoritma.....	14
3.2	Složenost algoritma.....	16
3.2.1	Vremenska složenost (Time complexity)	16
3.2.2	Prostorna složenost (Space complexity).....	17
3.3	Paralelizacija algoritma	18
3.4	Implementacija u okviru projekta.....	18
3.4.1	Analiza rezultata	20
4.	INSERTION SORT ALGORITAM	22
4.1	Implementacija algoritma.....	23
4.2	Složenost algoritma.....	25
4.2.1	Vremenska složenost (Time complexity)	25
4.2.2	Prostorna složenost (Space complexity).....	26
4.3	Paralelizacija algoritma	26
4.4	Implementacija u okviru projekta.....	27
4.4.1	Analiza rezultata	29
5.	Merge sort algoritam	31
5.1	Implementacija algoritma.....	32
5.2	Složenost algoritma.....	35
5.2.1	Vremenska složenost (Time Complexity).....	35
5.2.2	Prostorna složenost (Space Complexity)	36
5.3	Paralelizacija algoritma	36
5.4	Implementacija u okviru projekta.....	38

5.4.1	Analiza rezultata	40
6.	Quick sort algoritam	43
6.1	Implementacija algoritma.....	44
6.2	Složenost algoritma.....	48
6.2.1	Vremenska složenost (Time Complexity).....	48
6.2.2	Prostorna složenost.....	49
6.3	Paralelizacija algoritma	49
6.4	Implementacija u okviru projekta.....	51
6.4.1	Analiza rezultata	52
7.	HEAP SORT algoritam	55
7.1	Implementacija algoritma.....	56
7.2	Složenost algoritma.....	61
7.2.1	Vremenska složenost (Time Complexity).....	61
7.2.2	Prostorna složenost.....	61
7.3	Paralelizacija algoritma	61
7.4	Implementacija u okviru projekta.....	62
7.4.1	Analiza rezultata	63
8.	ZAKLJUČAK.....	65

1. UVOD

Sortiranje, poznato i kao ređanje, jedan je od najvažnijih zadataka u oblasti računarstva. U osnovi, sortiranje podrazumijeva linearno uređivanje niza elemenata po određenom kriterijumu, poput veličine brojeva, leksikografskog reda teksta, dužine nizova ili vrijednosti specifičnih polja u složenim strukturama podataka. Kao temeljna operacija, sortiranje je prisutno u brojnim primjenama koje svakodnevno koristimo, uključujući telefonske imenike, rječnike, kataloge i razne baze podataka. Uređeni podaci omogućavaju znatno bržu i jednostavniju pretragu, čime se povećava efikasnost izvršavanja različitih zadataka.

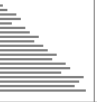
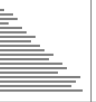
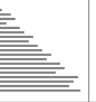
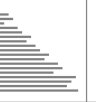
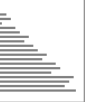
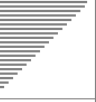
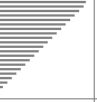
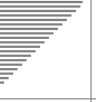
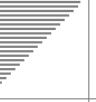
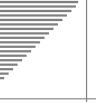
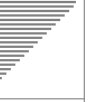
Sortirani nizovi olakšavaju izvođenje brojnih operacija, naročito pretrage, gdje binarna pretraga značajno smanjuje složenost u poređenju sa linearnom pretragom nesortiranih podataka. Ova činjenica jasno ističe važnost sortiranja za optimizaciju računarskih procesa i performansi sistema.

Postoji više različitih algoritama za sortiranje, a njihov izbor često zavisi od specifičnih potreba i zahtjeva primjene. Najjednostavniji algoritmi, poput **Bubble sort**, **Selection sort** i **Insertion sort** lako se implementiraju, ali imaju relativno slabije performanse, s vremenskom složenošću $O(n^2)$. S druge strane, složeniji algoritmi kao što su **Merge sort**, **Quick sort** i **Heap sort** omogućavaju bolje performanse, s prosječnom vremenskom složenošću $O(n \log n)$.

Prilikom analize efikasnosti algoritama za sortiranje obično se fokus stavlja na dvije osnovne operacije koje najviše utiču na performanse: poređenje elemenata i razmjenu elemenata niza. Kod nizova koji sadrže složenije tipove podataka, upravo ove operacije dominiraju ukupnom složenošću algoritma.

U ovom seminarskom radu detaljno će biti razmotren sort algoritam u kontekstu paralelnog računarstva, s posebnim osvrtom na prednosti, nedostatke i mogućnosti optimizacije kroz paralelizaciju procesa sortiranja. Kroz teorijski prikaz i praktične primjere implementacije, biće prikazano koliko paralelna implementacija može unaprijediti performanse sort algoritama, kao i pod kojim uslovima je njena primjena najefikasnija.¹

¹ „Algoritmi sortiranja“, Škola koda, pristupljeno 29. mart 2025, dostupno na: <https://skolakoda.github.io/algoritmi-sortiranja>.

									
Random									
Nearly Sorted									
Reversed									
Few Unique									

1. Ilustracija sort algoritama

2. BUBBLE SORT ALGORITAM

Bubble sort (sortiranje mjehurića) jedan je od osnovnih algoritama za sortiranje niza elemenata. Princip rada ovog algoritma zasniva se na uzastopnom poređenju susjednih elemenata u nizu i njihovom zamjenjivanju ukoliko nisu u odgovarajućem redosljedu.

Tokom svake iteracije, algoritam kreće od početka niza i poredi elemente jedan za drugim. Ako su dva susjedna elementa u pogrešnom redosljedu (na primjer, lijevi element je veći od desnog pri rastućem sortiranju), algoritam ih zamjenjuje. Na ovaj način, najveći ili najmanji element (u zavisnosti od načina sortiranja) „ispliva” na kraj niza, baš kao što se mjehurić vazduha podiže na površinu vode, po čemu je ovaj algoritam i dobio ime.

Nakon prve iteracije, sigurno je da je posljednji element na pravilnoj poziciji, pa algoritam ponavlja isti postupak, ali ovaj put ignorišući posljednji element. Svakom narednom iteracijom se područje pretrage smanjuje za jedan element, sve dok čitav niz ne bude sortiran.

Bubble sort algoritam spada u jednostavnije algoritme i lako se implementira, ali njegova najveća mana je slaba efikasnost, posebno na velikim skupovima podataka, zbog vremenske složenosti $O(n^2)$. Iz tog razloga se bubble sort uglavnom koristi za edukacijske svrhe ili male skupove podataka gdje efikasnost nije presudna.

2.1 Implementacija algoritma

Pretpostavimo da pokušavamo sortirati niz tj. poređati elemente u rastućem poretku.

Prva iteracija (uporedi i zamijeni)

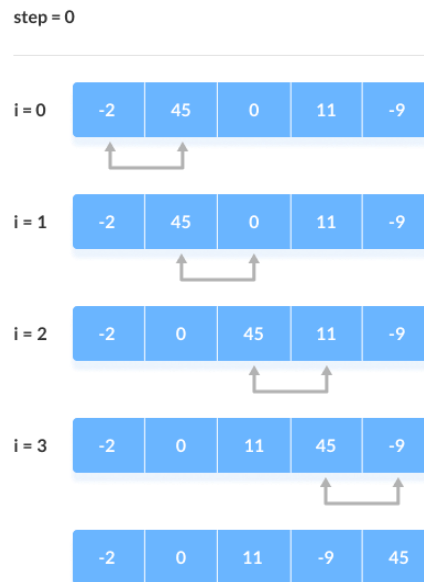
U prvoj iteraciji krećemo od samog početka niza (od prvog elementa) i vršimo sljedeće korake:

- Korak 1: Upoređujemo prvi element (na indeksu 0) sa susjednim drugim elementom (na indeksu 1). Ako je prvi element veći od drugog, to znači da

nisu u pravilnom (uzlaznom) redosljedu, pa vršimo njihovu zamjenu. Ukoliko je prvi element manji ili jednak drugom, ništa ne mijenjamo i prelazimo dalje.

- Korak 2: Zatim prelazimo na drugi element (indeks 1) i upoređujemo ga sa susjednim, trećim elementom (indeks 2). Ponovo, ukoliko drugi element nije u pravilnom redosljedu u odnosu na treći (veći je od njega), mijenjaju mjesta. Ako jesu pravilno raspoređeni, nastavljamo dalje.
- Korak 3: Navedeni postupak poređenja i zamjene susjednih elemenata nastavlja se sve do pretposljednjeg elementa, koji se poredi sa posljednjim elementom u nizu.

Na kraju prve iteracije najveći element sigurno „ispliva“ na kraj niza i zauzima svoju tačnu poziciju. Svakom narednom iteracijom postupak ponavljamo, ali svaki put sa jednim elementom manje, jer posljednji elementi niza postaju sortirani.



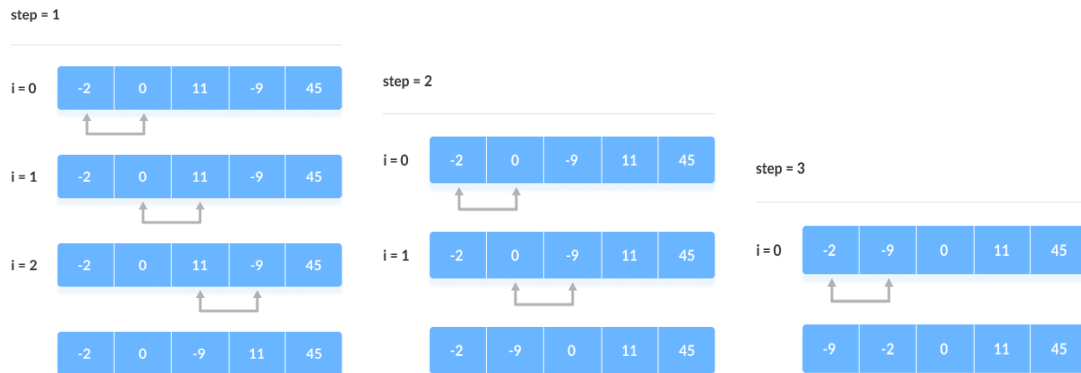
2. Ilustrovani prikaz prve iteracije

Druga iteracija

Nakon što završimo prvu iteraciju, postupak poređenja i zamjene elemenata se ponavlja za preostale, još uvijek nesortirane elemente niza.

U svakoj novoj iteraciji ponovo krećemo od prvog elementa, ali sada ignorišući elemente koji su već prethodno sortirani (koji se nalaze na kraju niza). Svaka nova iteracija će na sličan način kao i prethodna dovesti najveći element iz nesortiranog dijela niza na njegovo ispravno mjesto na kraju tog dijela.

Dakle, poslije svake završene iteracije, dio niza koji je sortiran raste, a dio koji ostaje nesortiran se smanjuje za jedan element. Postupak se nastavlja sve dok svi elementi niza ne budu pravilno sortirani po željenom kriterijumu.



3. Ilustracija preostalih iteracija

Pseudo-kod na slici 4 opisuje Bubble sort algoritam. Algoritam radi na sljedeći način: prolazi kroz niz u više iteracija pomoću dvije ugneždene petlje. Spoljašnja petlja određuje broj iteracija, a unutrašnja petlja u svakoj iteraciji poredi susjedne elemente, od prvog do posljednjeg nesortiranog elementa u nizu. Ako je lijevi element veći od desnog (pretpostavljajući uzlazno sortiranje), vrši se njihova zamjena. Nakon svake potpune iteracije spoljašnje petlje, najveći element koji nije bio sortiran dolazi na svoje pravo mjesto, odnosno „ispliva“ na kraj nesortiranog dijela niza. Ovaj proces se nastavlja sve dok se cijeli niz potpuno ne sortira.

```
bubbleSort(array)
  for i <- 1 to sizeOfArray - 1
    for j <- 1 to sizeOfArray - 1 - i
      if leftElement > rightElement
        swap leftElement and rightElement
    end bubbleSort
```

4. Pseudo-kod za Bubble sort algoritam

U slučaju opadajućeg sortiranja (od najvećeg ka najmanjem elementu), pseudo-kod Bubble sort algoritma funkcionira na isti način, ali sa obrnutim kriterijumom za poređenje elemenata. Ako je lijevi element manji od desnog, tada se vrši njihova zamjena mjesta. Na ovaj način, u svakoj iteraciji algoritma najmanji element „tone“ prema kraju niza. Na kraju svake iteracije najmanji element među nesortiranim elementima zauzima svoje pravo mjesto na kraju nesortiranog dijela niza. Postupak se ponavlja sve dok cijeli niz ne bude sortiran u opadajućem redoslijedu.

2.2 Složenost algoritma

2.2.1 Vremenska složenost(Time complexity)

Bubble sort algoritam ima različitu složenost u zavisnosti od stanja ulaznog niza:

- **Najbolji slučaj: $O(n)$**

Ovaj slučaj se javlja kada je niz već sortiran, pa algoritam prođe kroz niz samo jednom bez potrebe za zamjenama, te je složenost linearna.

- **Najgori slučaj: $O(n^2)$**

Najgori slučaj nastupa kada je niz u potpunosti obrnutog redoslijeda od željenog (na primjer, niz treba da bude uzlazno sortirani, a elementi su u silaznom redoslijedu). Tada svaki element mora „proći“ kroz cijeli niz, te su potrebna maksimalna poređenja i zamjene, što rezultira kvadratnom složenosti $O(n^2)$.

- **Prosječni slučaj: $O(n^2)$**

Prosječni slučaj nastaje kada su elementi niza slučajno raspoređeni, odnosno nisu ni idealno sortirani, niti idealno obrnuto sortirani. U prosjeku, broj poređenja i zamjena i dalje vodi do kvadratne složenosti $O(n^2)$.

U detaljnoj analizi, broj poređenja smanjuje se za jedan element svakom iteracijom algoritma, na sljedeći način: Prvi ciklus ima $(n - 1)$ poređenja, drugi ciklus $(n - 2)$ poređenja, treći ciklus $(n - 3)$ poređenja i tako dalje, sve do posljednjeg ciklusa koji ima 1 poređenje.

Ukupni broj poređenja se dobija zbirno kao:

$$(n-1) + (n-2) + (n-3) + \dots + 1 = n(n-1)/2$$

Ovaj izraz ima kvadratnu zavisnost od veličine ulaznog niza, što potvrđuje vremensku složenost $O(n^2)$.

2.2.2 Prostorna složenost (space complexity)

Bubble sort algoritam ima prostornu složenost $O(1)$, što znači da zahtijeva konstantnu količinu dodatnog prostora u memoriji, bez obzira na veličinu niza. Ovo je zbog toga što se za razmjenu elemenata koristi samo jedna dodatna privremena promjenljiva. Kod optimizovane verzije Bubble sorta (kada uvedemo dodatnu promjenljivu za provjeru je li niz već sortirani) koriste se ukupno dvije dodatne promjenljive, što opet predstavlja konstantnu složenost ($O(2)$, što se označava kao $O(1)$).

Bubble sort algoritam je stabilan, što znači da elementi s istim vrijednostima (npr. dva ista broja ili elementi koji se smatraju jednakim po kriterijumu sortiranja) nakon sortiranja zadržavaju originalni redoslijed koji su imali u ulaznom nizu.

Bubble sort se koristi uglavnom u sljedećim situacijama:

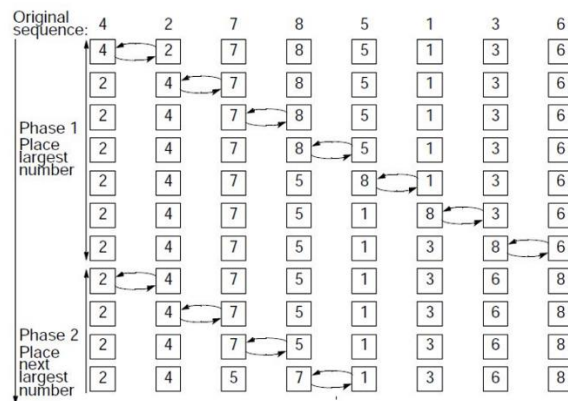
- Kad složenost nije bitna (npr. mali nizovi gdje nije problem što je sporiji).
- Kad je važnija jednostavnost koda i lakoća implementacije, npr. u edukativnim situacijama gdje se uči osnovni koncept algoritama i programiranja.

U praksi se rijetko koristi na velikim skupovima podataka zbog svoje neefikasnosti (spore performanse).²

² „Bubble Sort Algorithm“, Programiz, pristupljeno 29. mart 2025, dostupno na: <https://www.programiz.com/dsa/bubble-sort>.

2.3 Paralelizacija algoritma

Bubble sort, u obliku kakav je napisan na slici 4, predstavlja čisto sekvencijalni algoritam. Svaki korak u unutrašnjoj petlji se izvršava prije narednog, a cijela unutrašnja petlja mora biti završena prije nego što počne sljedeća iteracija spoljašnje petlje.

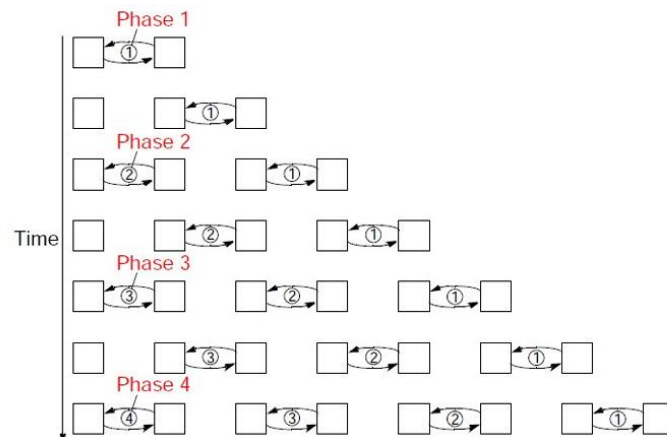


5. Koraci u Bubble sort algoritmu

Mnogi paralelne implementacije sekvencijalnih algoritama sortiranja, su sinhroni algoritmi, što znači da grupa akcija koje se izvode nad brojevima mora biti završena prije nego što započne sljedeća grupa akcija. Takav je i Bubble sort algoritam jer svaka operacija zavisi od prethodne ali to ne znači da se algoritam ne može prilagoditi paralelnom izvođenju. Ključna ideja je da se određene operacije ne moraju čekati da se prethodne potpuno završe, već se mogu izvršavati paralelno, pod uslovom da ne dolazi do konflikta između istovremenih zamjena.

Naime, zamjene u Bubble sort algoritmu (operacije poređenja i eventualne razmjene elemenata) mogu se organizovati tako da formiraju cjevasti tok izvršavanja (pipeline). U ovoj strukturi, svaki broj "putuje" kroz niz prema svojoj poziciji, ali sa vremenskim pomakom u odnosu na prethodne brojeve, analogno slično kao što vagoni voza ulaze jedan za drugim u tunel, ne sudarajući se međusobno.

Slika 6 ilustruje upravo ovu ideju. Prva zamjena (npr. pomjeranje najmanjeg broja ka početku) započinje odmah. Već u narednom vremenskom koraku, druga zamjena može da krene, ali ne odmah iza prve, već sa malim pomakom, kako ne bi došlo do „preklapanja“ sa prethodnom operacijom. Taj pomak se obezbjeđuje tako što je nova operacija pomjerena za jedno mjesto udesno, odnosno razdvojena jednim procesom. Na isti način, treća zamjena može da počne nakon druge, i tako redom. Na taj način se formira lanac istovremenih operacija koje teku nezavisno, ali sinhronizovano omogućavajući višestruke pomake brojeva kroz niz bez međusobnog ometanja.



6. Preklapanje koraka sortiranja u cjevovodu

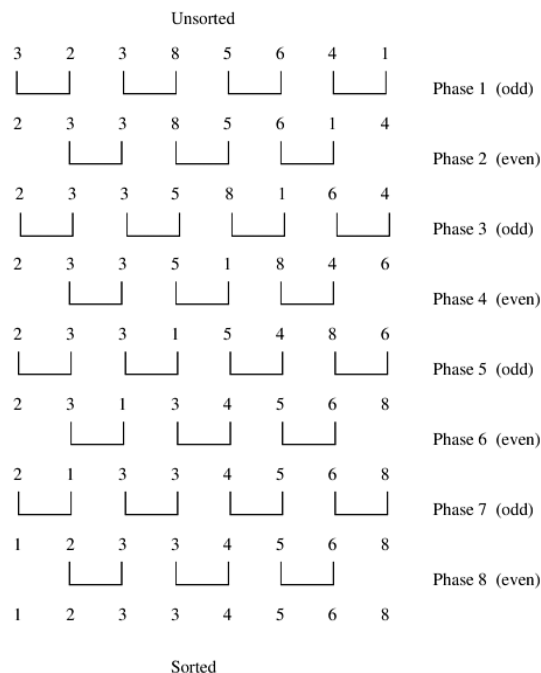
Ovaj pristup značajno poboljšava efikasnost izvođenja bubble sort algoritma u paralelnim sistemima, jer omogućava preklapanje koraka sortiranja i bolju iskorišćenost dostupnih procesora.

Na ovoj ideji zasniva se posebna verzija Bubble sort algoritma koja se naziva **NEPARNO-PARNO SORTIRANJE (ODD-EVEN TRANSPOSITION SORT)**.

Algoritam neparno-parnog sortiranja (odd-even transposition sort) sortira niz od n elemenata kroz ukupno n faza. Algoritam se zasniva na naizmjeničnom izvršavanju dvije vrste faza: neparne i parne.

Tokom neparne faze, porede se elementi sa neparnim indeksima sa svojim desnim susjedima, i ako su u pogrešnom redoslijedu, vrši se njihova zamjena.

Tokom parne faze, porede se elementi sa parnim indeksima sa desnim susjedima, uz identičan princip zamjene ako nisu pravilno raspoređeni.



7. Ilustracija algoritma neparno-parnog sortiranja (odd-even transposition sort)

Nakon što se izvrši svih n faza, niz će biti potpuno sortirani. Budući da svaka faza zahtijeva $O(n)$ operacija poređenja i zamjene, a ukupan broj faza je n , ukupna sekvencijalna vremenska složenost algoritma iznosi $O(n^2)$. Odd-even transposition sort nema posebnu prednost nad običnim bubble sort-om kada se koristi u sekvencijalnom programiranju, i zato se u tom kontekstu rijetko spominje.³

Međutim paralelna implementacija algoritma neparno-parnog sortiranja (odd-even transposition sort) ima manju vremensku složenost. Tokom svake faze algoritma, operacije poređenja i zamjene na parovima elemenata mogu se izvoditi istovremeno, što omogućava efikasnu raspodjelu posla među procesima.

Razmotrimo slučaj gdje se svaki element nalazi u zasebnom procesu, odnosno gdje je broj procesa jednak broju elemenata n koje treba sortirati. Pretpostavimo da su procesi organizovani kao jednodimenzionalni niz, i da se element a_i nalazi na procesu P_i , za $i = 1, 2, \dots, n$.

U toku neparne faze, svaki proces sa neparnim indeksom ($P_1, P_3, P_5, \dots$) poredi i, ako je potrebno, zamjenjuje svoj element sa elementom desnog susjeda. Tokom parne faze, isti postupak sprovode procesi sa parnim indeksom ($P_2, P_4, P_6, \dots$), opet sa svojim desnim susjedima.

U svakoj fazi, ili parni ili neparni procesi istovremeno izvršavaju korak poređenja i zamjene sa svojim susjedima. Ovakva razmjena može se izvršiti u vremenu $O(1)$ (konstantno vrijeme). Pošto se ukupno izvodi n faza, ukupno paralelno vrijeme izvršavanja ovog algoritma je $O(n)$.

<pre> procedure ODD-EVEN(n) begin for $i := 1$ to n do begin if i is odd then for $j := 0$ to $n/2 - 1$ do <i>compare-exchange</i>(a_{2j+1}, a_{2j+2}); if i is even then for $j := 1$ to $n/2 - 1$ do <i>compare-exchange</i>(a_{2j}, a_{2j+1}); end for end for end ODD-EVEN </pre>	<pre> procedure ODD-EVEN_PAR(n) begin $id :=$ process's label for $i := 1$ to n do begin if i is odd then if id is odd then <i>compare-exchange_min</i>($id + 1$); else <i>compare-exchange_max</i>($id - 1$); if i is even then if id is even then <i>compare-exchange_min</i>($id + 1$); else <i>compare-exchange_max</i>($id - 1$); end for end for end ODD-EVEN_PAR </pre>
---	---

8. Pseudo kod za sekvencijalnu(lijevo) i paralelnu(desno) implementaciju algoritama neparno-parnog sortiranja(odd-even transposition sort)

U sekvencijalnoj verziji jedan proces izvršava čitav algoritam, prolazeći kroz sve faze n puta, i u svakoj fazi iterativno poredi i eventualno zamjenjuje parove elemenata. Operacije se obavljaju redom, unutar for petlji, bez paralelizma.

S druge strane, paralelna verzija pretpostavlja da svaki element obrađuje poseban proces. Svaki proces ima svoj identifikator (id) i u svakoj od n faza samostalno odlučuje s kojim susjedom (lijevim ili desnim) će obaviti razmjenu, u zavisnosti od toga da li je faza i njegov id parni ili neparni. Sve operacije poređenja i zamjene u

³ Barry Wilkinson i Michael Allen, Parallel Programming: Techniques and Applications Using Networked Workstations and Parallel Computers, Second Edition, Prentice Hall, 2005, str. 303–311.

jednoj fazi se izvršavaju istovremeno, što omogućava ubrzanje putem paralelizacije.⁴

Prostorna složenost sekvencijalnog i paralelnog odd-even transposition algoritma prikazanih u pseudokodovima je linearna, odnosno $O(n)$. Oba algoritma zahtijevaju ulazni niz veličine n elemenata na kojem vrše sortiranje, ali ne koriste dodatne pomoćne strukture koje značajno rastu s veličinom ulaza. Potrebne dodatne varijable, kao što su indeksi petlji ili oznake procesa, zahtijevaju konstantan dodatni prostor, što znači da ne povećavaju ukupnu složenost. Dakle, ukupno gledano, oba algoritma su memorijski efikasna jer koriste minimalnu dodatnu memoriju, te njihova ukupna prostorna složenost ostaje $O(n)$.

2.4 Implementacija u okviru projekta

U okviru projekta realizovane su dvije verzije Bubble Sort algoritma: sekvencijalna i paralelna, kako bi se uporedila njihova funkcionalnost i performanse. Na slici 8.1 prikazana je sekvencijalna verzija algoritma. Funkcija prima niz cijelih brojeva (*int arr[]*) i njegovu dužinu (*int n*). Petlja *while* se ponavlja sve dok se u jednoj kompletnoj iteraciji ne izvrši prolaz bez ijedne zamjene, tj. dok *sorted* ostane *true*. Unutar svake iteracije koristi se *for* petlja koja ide od početka niza do pretposljednjeg elementa ($n - 1$) i upoređuje susjedne elemente. Ako je trenutni element veći od sljedećeg, oni se zamjenjuju, a *sorted* se postavlja na *false* da označi da je barem jedna zamjena izvršena, te da niz još nije potpuno sortiran.

```
void bubbleSort(int arr[], int n) {
    bool sorted = false;
    int tmp;

    while (!sorted) {
        sorted = true;

        for (int i = 0; i < n-1; ++i) {
            if (arr[i] > arr[i+1]) {
                tmp = arr[i];
                arr[i] = arr[i+1];
                arr[i+1] = tmp;
                sorted = false;
            }
        }
    }
}
```

8.1 Implementacija sekvencijalnog Bubble sort algoritma u C++

Paralelna verzija algoritma neparno-parnog sortiranja (odd-even transposition sort), prikazana na slici 8.2, predstavlja optimizaciju sekvencijalnog Bubble sort algoritma korišćenjem OpenMP biblioteke za paralelno programiranje. Algoritam uvodi dvije faze u svakoj iteraciji sort algoritma: parnu fazu (even phase) i neparnu fazu (odd phase). Unutar svake faze koristi se *#pragma omp for* direktiva da bi se svaka iteracija petlje raspodijelila na više niti, čime se omogućava istovremena obrada više parova elemenata. U parnoj fazi se porede parovi elemenata na indeksima 0 i 1, 2 i 3, itd., dok se u neparnoj fazi porede elementi na indeksima 1 i 2, 3 i 4, itd. OpenMP direktiva *reduction(&&:sorted)* se koristi kako bi se omogućila bezbjedna i precizna upotreba logičke promjenljive *sorted* tokom paralelnog

⁴ "Chapter 9: Odd-Even Transposition Sort," Parallel and Distributed Algorithms, pristupljeno: 6. april 2025, dostupno na: <http://users.atw.hu/parallelcomp/ch09lev1sec3.html>

izvršavanja više niti. Redukcija je tehnika u paralelnom programiranju kojom se omogućava da svaka nit nezavisno obrađuje svoju lokalnu verziju određene promjenljive, a zatim se svi ti lokalni rezultati kombinuju u jednu zajedničku vrijednost. U ovom slučaju, koristi se logička redukcija sa operatorom `&&`, što predstavlja logičku konjunkciju, odnosno operaciju "logičko I". To znači da će konačna vrijednost *sorted* biti *true* samo ako su sve lokalne verzije iz svih niti takođe *true*. Ako bar jedna nit detektuje da niz još uvijek nije sortirani i postavi *sorted = false*, konačan rezultat redukcije će takođe biti *false*, čime se signalizira da je potrebno još jedno prolazno sortiranje. Ova logika omogućava algoritmu da se zaustavi tek kada nijedna nit u toku parne i neparne faze ne izvrši zamjenu elemenata. Takođe, promjenljiva *tmp*, koja se koristi za privremeno čuvanje vrijednosti prilikom zamjene elemenata, označena je kao *private*, što znači da svaka nit koristi sopstvenu nezavisnu kopiju te promjenljive. Time se izbjegava međusobni konflikt pri pristupu zajedničkim resursima, što je ključno za ispravnost paralelnih algoritama.

```
void bubbleSortParallel(int arr[], int n) {
    bool sorted = false;
    int tmp;

    while (!sorted) {
        sorted = true;

        #pragma omp parallel private(tmp)
        {
            // Even phase
            #pragma omp for reduction(&&:sorted)
            for (int i = 0; i < n-1; i += 2) {
                if (arr[i] > arr[i+1]) {
                    tmp = arr[i];
                    arr[i] = arr[i+1];
                    arr[i+1] = tmp;
                    sorted = false;
                }
            }

            // Odd phase
            #pragma omp for reduction(&&:sorted)
            for (int i = 1; i < n-1; i += 2) {
                if (arr[i] > arr[i+1]) {
                    tmp = arr[i];
                    arr[i] = arr[i+1];
                    arr[i+1] = tmp;
                    sorted = false;
                }
            }
        }
    }
}
```

8.2 Impelemntacija paralelnog Odd-even transposition sort algoritma u C++

Na slici 8.3 prikazan je glavni program koji za potrebe analize performansi uključuje automatsko generisanje nasumičnih nizova, višestruko testiranje i upis rezultata u CSV fajl. Funkcija *generateRandomArray* popunjava dati niz nasumičnim cijelim brojevima do zadate maksimalne vrijednosti (podrazumijevano 10.000), dok funkcija *copyArray* služi da se originalni niz kopira u pomoćni, čime se obezbjeđuje da oba algoritma testiraju isti ulaz. Mjerenje vremena izvršavanja implementirano je kroz funkcije *averageSequentialTime* i *averageParallelTime*, koje izvede zadatu verziju sortiranja više puta (po defaultu 5 puta) i vraćaju prosječno vrijeme izvođenja. Za sekvencijalnu verziju koristi se *std::chrono::high_resolution_clock*, dok se za paralelnu koristi *omp_get_wtime()* radi tačnijeg mjerenja u OpenMP okruženju. Broj niti za paralelno izvršavanje se podešava pomoću *omp_set_num_threads*. U funkciji *main*, prvo se definišu različite veličine nizova (*n_values*) i različiti brojevi niti (*thread_values*) za testiranje. Za svaki test slučaj generiše se novi nasumični niz, koji se zatim koristi za mjerenje vremena sekvencijalne verzije algoritma. Nakon toga, paralelna verzija se testira sa različitim brojem niti, a za svaku kombinaciju se izračunavaju performanse: **speedup (ubrzanje)** kao odnos između vremena sekvencijalnog i paralelnog izvođenja, **efficiency (efikasnost)** kao odnos ubrzanja i

broja niti, i **cost (trošak)** kao proizvod vremena paralelnog izvođenja i broja niti. Dodatno, efikasnost se ograničava na maksimalnu vrijednost 1.0 kako bi se izbjegle numeričke anomalije. Rezultati se upisuju u CSV fajl koji je pogodan za analizu dobijenih rezultata. Na kraju, svi dinamički alocirani nizovi se brišu kako bi se oslobodila memorija.

```
void generateRandomArray(int* arr, int n, int maxVal = 10000) {
    for (int i = 0; i < n; ++i)
        arr[i] = rand() % maxVal;
}

void copyArray(int* src, int* dest, int n) {
    for (int i = 0; i < n; ++i)
        dest[i] = src[i];
}

double averageSequentialTime(int* original, int* buffer, int n, int runs = 5) {
    double total = 0.0;
    for (int i = 0; i < runs; ++i) {
        copyArray(original, buffer, n);
        auto start = std::chrono::high_resolution_clock::now();
        bubbleSort(buffer, n);
        auto end = std::chrono::high_resolution_clock::now();
        total += std::chrono::duration<double>(end - start).count();
    }
    return total / runs;
}

double averageParallelTime(int* original, int* buffer, int n, int threads, int runs = 5) {
    double total = 0.0;
    omp_set_num_threads(threads);
    for (int i = 0; i < runs; ++i) {
        copyArray(original, buffer, n);
        double start = omp_get_wtime();
        bubbleSortParallel(buffer, n);
        double end = omp_get_wtime();
        total += (end - start);
    }
    return total / runs;
}

int main() {
    std::srand(time(nullptr));

    std::vector<int> n_values = {1000, 5000, 10000, 20000};
    std::vector<int> thread_values = {1, 2, 4, 8};

    std::ofstream out("BubbleSortResults.csv");
    out << std::fixed << std::setprecision(6); // Zaokruživanje na 6 decimala
    out << "n;threads;seq_time;par_time;speedup;efficiency;cost\n";

    for (int n : n_values) {
        int* original = new int[n];
        int* seq_arr = new int[n];
        int* par_arr = new int[n];

        generateRandomArray(original, n);

        // Sekvencijalno mjerenje (prosjeck)
        double seq_time = averageSequentialTime(original, seq_arr, n);

        for (int threads : thread_values) {
            double par_time = averageParallelTime(original, par_arr, n, threads);

            double speedup = seq_time / par_time;
            double efficiency = speedup / threads;
            double cost = par_time * threads;

            // Ako je efficiency veći od 1 zbog greške - ograničavamo ga
            if (efficiency > 1.0) efficiency = 1.0;

            out << n << ";"; << threads << ";"; << seq_time << ";"; << par_time << ";";
            << speedup << ";"; << efficiency << ";"; << cost << "\n";

            std::cout << "n=" << n << ", threads=" << threads
                << " -> speedup=" << speedup
                << ", efficiency=" << efficiency << ", cost=" << cost << "\n";

            delete[] original;
            delete[] seq_arr;
            delete[] par_arr;
        }

        out.close();
        std::cout << "\nRezultati su sačuvani u 'BubbleSortResults.csv'\n";
        return 0;
    }
}
```

8.3 Kod programa za analizu performansi paralelne i sekvencijalne verzije Bubble sort algoritma u C++

Projekat uključuje i python skriptu (*BubbleSortGraphs.py*) koja omogućava vizuelnu analizu performansi Bubble Sort algoritama koristeći biblioteke *pandas*, *matplotlib* i *seaborn*. Učitava rezultate iz CSV fajla i prikazuje tri grafikona: vrijeme izvršavanja (za sekvencijalnu i paralelnu verziju), speedup (ubrzanje paralelne verzije u odnosu na sekvencijalnu) i efikasnost (koliko efektivno paralelizacija koristi dostupne niti). Svaki grafikon prikazuje uticaj veličine niza i broja niti na performanse sortiranja, omogućavajući jednostavno poređenje različitih pristupa.

2.4.1 Analiza rezultata

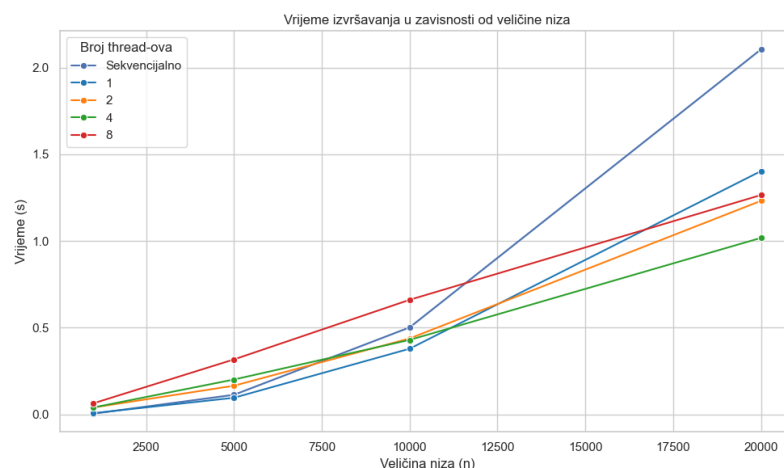
n	threads	seq_time	par_time	speedup	efficiency	cost
1000	1	0.004551	0.0072	0.632139	0.632139	0.0072
1000	2	0.004551	0.0398	0.114357	0.057178	0.0796
1000	4	0.004551	0.04	0.113785	0.028446	0.16
1000	8	0.004551	0.064	0.071116	0.008889	0.512
5000	1	0.113279	0.096	1.179994	1	0.096
5000	2	0.113279	0.1654	0.684882	0.342441	0.3308
5000	4	0.113279	0.2012	0.563019	0.140755	0.8048
5000	8	0.113279	0.317	0.357348	0.044669	2.536
10000	1	0.501972	0.3788	1.325164	1	0.3788
10000	2	0.501972	0.438	1.146055	0.573028	0.876
10000	4	0.501972	0.4296	1.168464	0.292116	1.7184
10000	8	0.501972	0.6608	0.759643	0.094955	5.2864
20000	1	2.105273	1.4032	1.500337	1	1.4032
20000	2	2.105273	1.2316	1.70938	0.85469	2.4632
20000	4	2.105273	1.018	2.068048	0.517012	4.072
20000	8	2.105273	1.265	1.664247	0.208031	10.12

8.4 Tabelarni prikaz dobijenih rezultata

Na osnovu analize eksperimentalnih rezultata prikazanih u tabeli 8.4, može se uočiti da paralelna verzija Bubble Sort algoritma ne donosi značajno ubrzanje za manje ulazne veličine, kao što su nizovi dužine 1000 i 5000 elemenata. U tim slučajevima, dodatno vrijeme koje je potrebno da se više niti međusobno usklade i rade zajedno veće je od koristi koje donosi paralelno izvršavanje, pa zbog toga program radi sporije i manje efikasno kada se koristi više niti. Najniži zabilježeni speedup (0.0711) i efikasnost (0.0089) zabilježeni su za $n=1000$ i 8 niti, što pokazuje da prekomjerno paralelizovanje za male ulaze može biti kontraproduktivno.

S druge strane, za veće nizove, kao što je $n=20000$, paralelizacija donosi jasne prednosti. Najveći speedup od 2.068 postignut je uz 4 niti, što predstavlja najbolje iskorišćenje paralelne arhitekture u ovom eksperimentu. Međutim, daljim povećanjem broja niti, efikasnost opada, a trošak (cost) postaje znatno veći, što se posebno vidi kod 8 niti (cost = 10.12). To ukazuje da iako se ubrzanje povećava, višestruki niti uvode dodatnu režiju i ne garantuju linearnu skalabilnost.

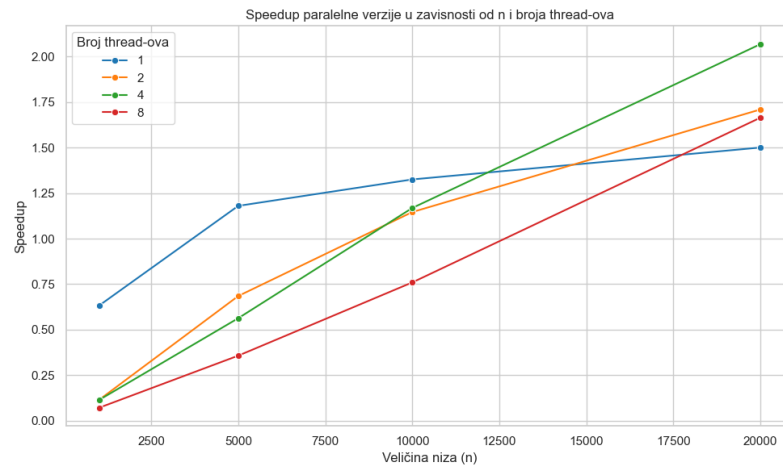
Ukratko, rezultati pokazuju da je Bubble Sort algoritam, kao inherentno sekvencijalna metoda, ograničen u skalabilnosti, i da paralelizacija ima smisla tek kod dovoljno velikih ulaza i optimalnog broja niti. Efikasnost se kreće od idealnih 1 (kod 1 niti i $n=5000$ i $n=10000$) do veoma niskih vrijednosti, što potvrđuje da paralelno izvršavanje nije uvijek korisno, posebno kod jednostavnih algoritama i manjih podataka.



8.5 Grafikon koji prikazuje vrijeme izvršavanja u zavisnosti od veličine niza

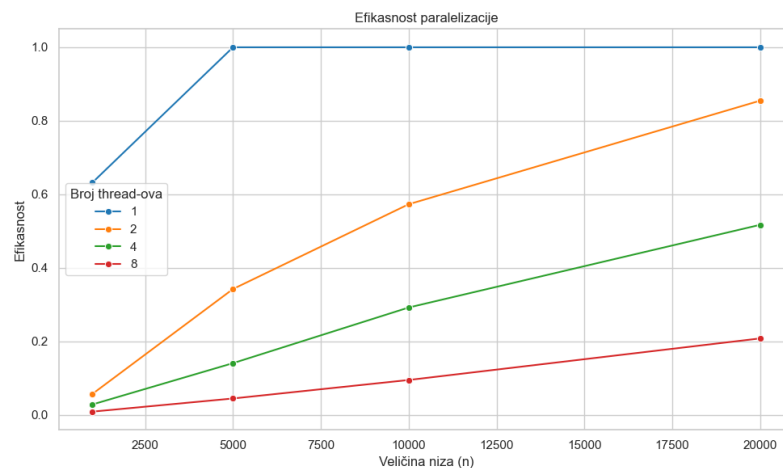
Na grafikonu 8.5 se jasno vidi da vrijeme linearno raste sa povećanjem veličine niza za sve verzije algoritma. Sekvencijalna verzija ima najmanje vrijeme izvršavanja za male nizove (npr. do 5000 elemenata), dok se za veće ulaze (npr. 20000 elemenata) paralelna verzija sa 4 niti pokazuje najefikasnijom. Zanimljivo je da verzija sa 8 niti konstantno ima nešto duže vrijeme, što ukazuje da se kod velikog broja niti javlja dodatna režija zbog sinhronizacije i koordinacije⁵.

⁵ Režija paralelizacije označava dodatno vrijeme i resurse koje program mora da potroši da bi više niti (ili procesa) moglo pravilno i istovremeno da radi. To uključuje sinhronizaciju (čekanje da sve niti završe određeni dio posla prije nastavka), raspodjelu posla i komunikaciju među nitima.



8.6 Grafikon koji prikazuje speedup paralelne verzije u zavisnosti od veličine i broja thread-ova

Na grafikonu 8.6 uočava se da speedup raste sa veličinom niza i brojem niti, ali ne proporcionalno. Najveći speedup (~ 2.07) postignut je sa 4 niti pri veličini niza od 20000 elemenata, što znači da je paralelna verzija bila više nego dvostruko brža od sekvencijalne. Kod 8 niti je ubrzanje manje, što potvrđuje da povećanje broja niti ne garantuje bolje performanse zbog režije koju ono nosi.



8.7 Grafikon koji prikazuje efikasnost paralelizacije u zavisnosti od veličine niza i broja thread-ova

Na grafikonu 8.7 jasno se vidi da efikasnost opada kako se broj niti povećava, a poboljšava se sa većim veličinama niza. Verzija sa 1 niti ima idealnu efikasnost (1.0), dok se kod 8 niti postiže niska efikasnost, posebno za manje nizove (npr. ispod 0.1 za $n = 5000$). Ovo ukazuje da za jednostavan algoritam kao što je Bubble Sort, koji je prirodno sekvencijalan, nema mnogo koristi od ekstremne paralelizacije osim ako se radi sa veoma velikim skupovima podataka.

Vizuelna analiza potvrđuje da paralelizacija Bubble Sort algoritma ima smisla samo kod većih nizova i umjerenog broja niti, dok kod manjih ulaza i prevelikog broja niti dolazi do smanjenja performansi zbog režije upravljanja paralelnim izvršavanjem.

3. SELECTION SORT ALGORITAM

Selection sort ili sortiranje odabirom je algoritam sortiranja koji radi tako što u svakoj iteraciji pretražuje nesortirani dio niza kako bi pronašao najmanji (u slučaju uzlaznog sortiranja tj. sortiranja u rastućem poretku) ili najveći (u slučaju silaznog sortiranja tj. sortiranja u opadajućem poretku) element. Kada pronađe taj element, algoritam ga zamjenjuje sa prvim elementom nesortiranog dijela niza. Time, nakon svake završene iteracije, niz je podijeljen na dva dijela – lijevi dio koji je sortiran, i desni dio koji je nesortiran. Sa svakom novom iteracijom, sortiran dio raste za jedan element, a nesortiran dio se smanjuje.

Ovaj postupak se ponavlja sve dok niz u potpunosti ne bude sortiran. Glavna ideja algoritma ogleda se upravo u „odabiru“ najmanjeg ili najvećeg elementa preostalog dijela niza i njegovom smještanju na početak tog dijela. Selection sort je poznat po jednostavnoj implementaciji, ali mu je vremenska složenost $O(n^2)$, što ga čini neefikasnim za sortiranje većih nizova. Zbog toga se uglavnom koristi u obrazovne svrhe ili kod manjih skupova podataka gdje efikasnost nije od presudnog značaja.

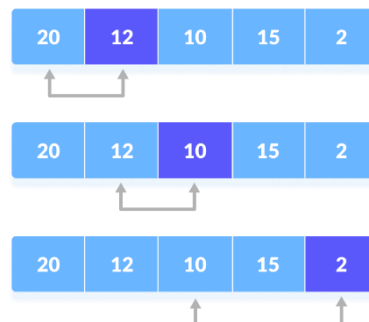
3.1 Implementacija algoritma

Na početku svake iteracije, algoritam uzima prvi element nesortiranog dijela niza i postavlja ga kao trenutni minimum. U prvoj iteraciji, to je prvi element cijelog niza, dok u narednim iteracijama indeks ovog početnog elementa napreduje ka desno, kako se dio sortiranog niza širi.



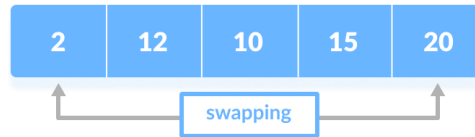
9. Postavljanje početnog elementa niza kao trenutni minimum

Nakon odabira početnog minimuma, algoritam poredi ovaj element sa svim ostalim elementima u nesortiranom dijelu niza. Ako se prilikom poređenja naiđe na element koji je manji od trenutnog minimuma, onda se pronađeni element označava kao novi minimum. Algoritam zatim nastavlja poređenja, prelazeći redom preko svakog narednog elementa sve do posljednjeg elementa u nesortiranom dijelu niza. Na kraju ovog koraka, pronađen je pravi minimalni element nesortiranog dijela niza.



10. Traženje stvarnog minimuma

Kad se završava poređenje svih elemenata u nesortiranom dijelu, algoritam je pronašao element sa najmanjom vrijednošću. Taj minimalni element zatim mijenja mjesta sa prvim elementom nesortiranog dijela niza (ako već nije na tom mjestu). Ovom zamjenom minimalni element zauzima svoju tačnu poziciju na početku nesortiranog dijela niza. Nakon ovog koraka, niz je podijeljen na dva dijela: sortirani dio (lijevo) i nesortirani dio (desno).

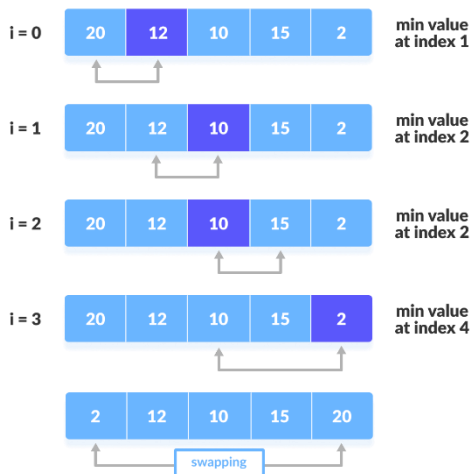


11. Minimum se postavlja na početak nesortiranog dijela niza

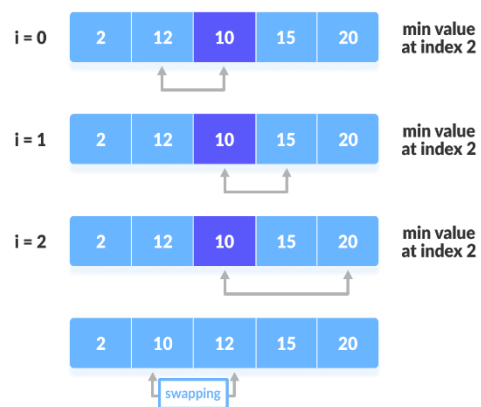
U sljedećoj iteraciji algoritam pomjera početnu poziciju (indeks) za jedan korak udesno, počevši od prvog elementa preostalog nesortiranog dijela niza, te ponovo prolazi kroz prethodna tri koraka: ponovo odabira prvi element nesortiranog dijela kao privremeni minimum, traži stvarni minimum u ostatku niza i postavlja pronađeni minimalni element na početak nesortiranog dijela.

Iteracije se ponavljaju sve dok algoritam ne prođe kroz cijeli niz. Na kraju rada algoritma, svi elementi se nalaze na svojim odgovarajućim pozicijama, a niz je u potpunosti sortiran u rastućem redoslijedu. Ovakvim pristupom Selection sort algoritam postepeno „gradi“ sortiran niz od najmanjih ka najvećim elementima, uvijek pronalazeći i izdvajajući najmanji preostali element u svakom koraku.

step = 0



step = 1



12. Prva i druga iteracija



13. Treća i četvrta iteracija

Na slici 14 prikazan je pseudo-kod Selection sort algoritma koji prima ulazni niz (*array*) i njegovu veličinu (*size*). Algoritam prolazi kroz niz u petlji (*for i*) od indeksa 0 do *size - 1*. Na početku svake iteracije spoljašnje petlje, indeks prvog nesortiranog elementa (*trenutno i*) postavlja se kao početni minimum. Zatim, pomoću unutrašnje petlje (*for j*), algoritam kreće od sljedećeg elementa (*indeks i + 1*) i poredi ga sa trenutnim minimumom (*array[current minimum]*). Ako tokom prolaska kroz niz pronađe element koji je manji od trenutnog minimuma, taj novi element označava kao novi minimum (*ažurira indeks minimuma na j*). Kada se unutrašnja petlja završi, provjerava se da li se pronađeni minimalni element nalazi na indeksu različitom od početne pozicije (*i*). Ako je tako, algoritam zamjenjuje mjesta trenutnog minimuma sa elementom na početku nesortiranog dijela (*array[i]*). Ukoliko je minimum već na ispravnom mjestu, nema potrebe za zamjenom. Ovaj proces se ponavlja sve dok algoritam ne prođe kroz sve elemente niza, čime se osigurava da svaki element dođe na svoje ispravno mjesto, a niz na kraju ostaje sortiran.

```
selectionSort(array, size)
  for i from 0 to size - 1 do
    set i as the index of the current minimum
    for j from i + 1 to size - 1 do
      if array[j] < array[current minimum]
        set j as the new current minimum index
    if current minimum is not i
      swap array[i] with array[current minimum]
  end selectionSort
```

14. Pseudo-kod za Selection sort algoritam

Ako želimo da Selection sort algoritam sortira niz u opadajućem poretku (od najvećeg ka najmanjem), onda se radi veoma sličan postupak kao za rastući poredak. Jedina razlika je to da se u svakoj iteraciji traži najveći element u nesortiranom dijelu niza umjesto najmanji kao što je slučaj kod sortiranja u rastućem poredku.

3.2 Složenost algoritma

3.2.1 Vremenska složenost (Time complexity)

Vremenska složenost Selection sort algoritma u svim slučajevima (najboljem, najgorem i prosječnom) jeste $O(n^2)$. U svakom slučaju, Selection sort mora da prođe kroz cijeli niz, tražeći minimum u svakom prolazu kroz nesortirani dio. To znači da

se uvijek mora obaviti isti broj poređenja, bez obzira na stanje ulaznog niza (bilo da je niz već sortiran, potpuno nesortiran ili djelimično sortiran).

Broj poređenja za svaku iteraciju izgleda ovako: U prvom prolazu algoritam izvrši $(n - 1)$ poređenja, u drugom prolazu $(n - 2)$ poređenja, u trećem prolazu $(n - 3)$ poređenja i tako dalje, sve dok posljednji prolaz ne završi s jednim poređenjem.

Kad zbrojimo sva poređenja dobijemo sljedeći izraz:

$$(n - 1) + (n - 2) + (n - 3) + \dots + 1 = n(n - 1) / 2$$

Ovaj izraz je približno jednak $n^2/2$, što pokazuje kvadratnu složenost. U notaciji velikog O, konstante zanemarujemo, te ostaje $O(n^2)$.

3.2.2 Prostorna složenost (Space complexity)

Selection sort ima prostornu složenost $O(1)$, što znači da algoritam koristi minimalnu količinu dodatne memorije za svoj rad. Konkretno, koristi se samo jedna dodatna varijabla koja služi za pamćenje indeksa trenutno najmanjeg elementa u nesortiranom dijelu niza. Bez obzira na veličinu ulaznog niza, količina dodatne memorije koju koristi Selection sort ostaje konstantna (uvijek ista).

Algoritam Selection sort nije stabilan. To znači da ukoliko postoje elementi jednake vrijednosti, njihov redosljed u konačnom sortiranom nizu ne mora ostati isti kao u originalnom nizu. Drugim riječima, Selection sort može promijeniti relativni redosljed istovrijednih elemenata.

Selection sort algoritam se najčešće koristi u situacijama gdje:

- Treba sortirati male skupove podataka:

Zbog jednostavnosti implementacije, Selection sort je pogodan za male nizove ili popise, gdje njegova kvadratna složenost $O(n^2)$ ne predstavlja problem u smislu efikasnosti.

- Trošak zamjene elemenata nije bitan:

Selection sort vrši relativno mali broj zamjena elemenata (tačno jednu zamjenu po prolazu kroz niz). Ako je zamjena elemenata skupa operacija, Selection sort može biti bolji izbor od algoritama poput Bubble sorta, koji imaju više zamjena elemenata.

- Provjera svih elemenata je obavezna:

Ukoliko je potrebno pregledati svaki element niza, Selection sort uvijek prolazi kroz sve elemente, što osigurava detaljnu provjeru.

- Troškovi pisanja u memoriju su značajni (kao u flash memoriji):

Selection sort algoritam radi mali broj pisanja u memoriju (tačno $O(n)$), jer u svakoj iteraciji izvrši maksimalno jednu zamjenu. To ga čini pogodnim za korišćenje u uređajima koji imaju ograničen broj ciklusa pisanja u memoriju

(poput flash memorije), u poređenju sa Bubble sort algoritmom koji pravi veći broj zamjena i ima $O(n^2)$ pisanja u memoriju.⁶

3.3 Paralelizacija algoritma

Iako Selection sort ima jednostavnu strukturu, njegova paralelizacija je ograničena zbog sekvencijalne zavisnosti među iteracijama. Naime, svaka nova iteracija počinje tek kada se prethodna završi, jer se minimum uvijek traži u preostalom, nesortiranom dijelu niza.

Jedini dio algoritma koji se može djelimično paralelizovati jeste traženje minimuma. Ova operacija se može ubrzati korišćenjem paralelne redukcije, gdje svaki proces traži lokalni minimum u svom segmentu, a zatim se lokalni minimumi upoređuju da bi se pronašao globalni minimum. Ipak, ostatak algoritma ostaje sekvencijalan, jer se sortiranje nastavlja tek kada se prethodni minimum postavi na svoje mjesto.

Zbog ovih zavisnosti i potrebe za čestom sinhronizacijom, paralelna verzija Selection sorta nije efikasna i nije troškovno optimalna. S obzirom da se koristi n procesa, ukupni trošak algoritma (vrijeme $\times$ broj procesa) iznosi $O(n^2)$ — što je identično kao i kod sekvencijalne verzije, pa se troškovna optimalnost ne postiže.

Zbog navedenih ograničenja, Selection sort se rijetko koristi u paralelnim sistemima, a značajno efikasniji izbor predstavljaju algoritmi poput parallel merge sorta i odd-even transposition sorta, koji omogućavaju veći stepen istovremene obrade i bolju skalabilnost.

3.4 Implementacija u okviru projekta

Projekat obuhvata implementaciju sekvencijalne i, u mjeri u kojoj je to algoritamski izvodljivo, paralelizovane verzije Selection sort algoritma. Cilj ove postavke je da se kroz konkretan kod i eksperimentalne rezultate prikaže kako paralelizacija ovog algoritma ne donosi značajne performanse, upravo zbog njegove inherentno sekvencijalne prirode i zavisnosti među iteracijama.

Na slici 14.1 prikazan je kod za sekvencijalnu verziju Selection sort algoritma. Funkcija *selectionSort* prima pokazivač na niz i njegovu dužinu. Spoljašnja for petlja prolazi kroz svaki element niza osim posljednjeg, a za svaki indeks i pretpostavlja da je *arr[i]* najmanji. Unutrašnja for petlja zatim prolazi kroz preostali dio niza i traži indeks *min_idx* elementa sa najmanjom vrijednošću. Kada se najmanji element pronađe, koristi se *std::swap* (iz biblioteke *algorithm*) da se zamijeni sa elementom na poziciji i .

⁶ „Selection Sort Algorithm“, Programiz, pristupljeno 29. mart 2025, dostupno na: <https://www.programiz.com/dsa/selection-sort>.

```
#include <algorithm>

void selectionSort(int* arr, int n) {
    for (int i = 0; i < n - 1; ++i) {
        int min_idx = i;
        for (int j = i + 1; j < n; ++j)
            if (arr[j] < arr[min_idx])
                min_idx = j;
        std::swap(arr[i], arr[min_idx]);
    }
}
```

14.1 Implementacija sekvencijalnog Selection sort algoritma u C++

Na slici 14.2 implementirana je ista logika uz pokušaj paralelizacije putem OpenMP direktiva. Glavna razlika je u korišćenju `#pragma omp parallel` bloka unutar svake iteracije spoljne for petlje. Svaki thread koristi privatnu promjenljivu `local_min_idx` i u okviru `#pragma omp for nowait` pretražuje dio niza u potrazi za lokalnim minimumom. Zatim se, unutar `#pragma omp critical`, upoređuje `arr[local_min_idx]` sa `arr[min_idx]` i po potrebi ažurira globalni `min_idx`. Na kraju svake iteracije spoljne petlje, vrši se zamjena elemenata pomoću `std::swap`. Ova struktura zahtijeva sinhronizaciju između thread-ova, zbog čega koristi critical sekciju da bi se izbjegle trke podataka prilikom ažuriranja globalnog minimuma.

```
#include <omp.h>
#include <algorithm>

void selectionSortParallel(int* arr, int n) {
    for (int i = 0; i < n - 1; ++i) {
        int min_idx = i;

        #pragma omp parallel
        {
            int local_min_idx = min_idx;

            #pragma omp for nowait
            for (int j = i + 1; j < n; ++j) {
                if (arr[j] < arr[local_min_idx]) {
                    local_min_idx = j;
                }
            }

            #pragma omp critical
            {
                if (arr[local_min_idx] < arr[min_idx]) {
                    min_idx = local_min_idx;
                }
            }
        }

        std::swap(arr[i], arr[min_idx]);
    }
}
```

14.2 Implementacija paralelizovanog Selection sort algoritma u C++

U programskom kodu na slici 14.3 implementirano je testno okruženje za evaluaciju performansi sekvencijalne i paralelne verzije Selection Sort algoritma. Kod uključuje dvije ključne funkcije: `averageSequentialTime` i `averageParallelTime`, koje vrše višestruko izvršavanje sortiranja radi izračunavanja prosječnog vremena. Korišćenje `omp_set_num_threads(threads)` omogućava kontrolu nad brojem thread-ova tokom testiranja paralelne verzije. U `main` funkciji, kroz petlju se prolazi kroz unaprijed definisane veličine nizova i brojeve thread-ova. Za svaki par `n` i `threads` vrši se generisanje slučajnog niza, mjerenje vremena izvršavanja i računanje metrika: `speedup`, `efficiency` i `cost`. Svi rezultati se bilježe u CSV fajl što omogućava dalju analizu i grafički prikaz performansi(fajl `SelectionSortGraphs.py`).

```

void generateRandomArray(int* arr, int n, int maxVal = 10000) {
    for (int i = 0; i < n; ++i)
        arr[i] = rand() % maxVal;
}

void copyArray(int* src, int* dest, int n) {
    for (int i = 0; i < n; ++i)
        dest[i] = src[i];
}

double averageSequentialTime(int* original, int* buffer, int n, int runs = 5) {
    double total = 0.0;
    for (int i = 0; i < runs; ++i) {
        copyArray(original, buffer, n);
        auto start = std::chrono::high_resolution_clock::now();
        selectionSort(buffer, n);
        auto end = std::chrono::high_resolution_clock::now();
        total += std::chrono::duration(end - start).count();
    }
    return total / runs;
}

double averageParallelTime(int* original, int* buffer, int n, int threads, int runs = 5) {
    double total = 0.0;
    omp_set_num_threads(threads);
    for (int i = 0; i < runs; ++i) {
        copyArray(original, buffer, n);
        double start = omp_get_wtime();
        selectionSortParallel(buffer, n);
        double end = omp_get_wtime();
        total += (end - start);
    }
    return total / runs;
}

int main() {
    std::srand(time(nullptr));

    std::vector<int> n_values = {1000, 5000, 10000, 20000}; // Po želji dodaj
    std::vector<int> thread_values = {1, 2, 4, 8};

    std::ofstream out("SelectionSortResults.csv");
    out << std::fixed << std::setprecision(6);
    out << "n;threads;seq_time;par_time;speedup;efficiency;cost\n";

    for (int n : n_values) {
        int* original = new int[n];
        int* seq_arr = new int[n];
        int* par_arr = new int[n];

        generateRandomArray(original, n);

        double seq_time = averageSequentialTime(original, seq_arr, n);

        for (int threads : thread_values) {
            double par_time = averageParallelTime(original, par_arr, n, threads);

            double speedup = seq_time / par_time;
            double efficiency = speedup / threads;
            double cost = par_time * threads;

            if (efficiency > 1.0) efficiency = 1.0;

            out << n << ";" << threads << ";" << seq_time << ";" << par_time <<
                << speedup << ";" << efficiency << ";" << cost << "\n";

            std::cout << "n=" << n << ", threads=" << threads
                << " -> speedup=" << speedup
                << ", efficiency=" << efficiency << ", cost=" << cost <<
                << "\n";

            delete[] original;
            delete[] seq_arr;
            delete[] par_arr;
        }

        out.close();
        std::cout << "\nRezultati su sacuvani u 'SelectionSortResults.csv'\n";
        return 0;
    }
}

```

14.3 Kod programa za analizu performansi paralelne i sekvencijalne verzije Selection sort algoritma u C++

3.4.1 Analiza rezultata

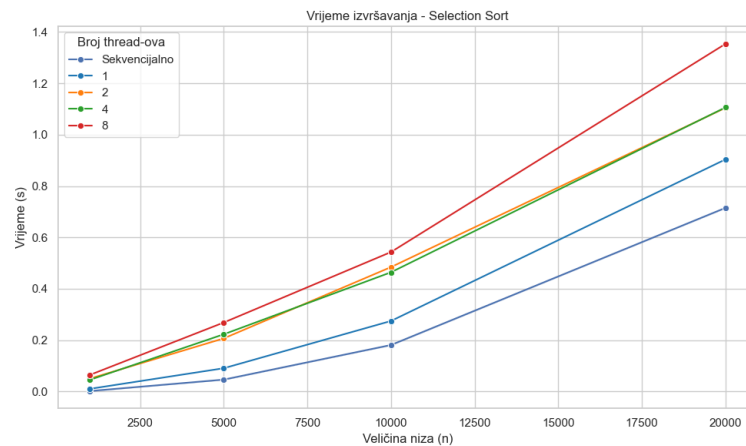
n	threads	seq_time	par_time	speedup	efficiency	cost
1000	1	0.001864	0.0104	0.179269	0.179269	0.0104
1000	2	0.001864	0.0504	0.036992	0.018496	0.1008
1000	4	0.001864	0.0456	0.040886	0.010221	0.1824
1000	8	0.001864	0.0648	0.028772	0.003596	0.5184
5000	1	0.046315	0.0906	0.511199	0.511199	0.0906
5000	2	0.046315	0.2068	0.223958	0.111979	0.4136
5000	4	0.046315	0.2228	0.207875	0.051969	0.8912
5000	8	0.046315	0.2686	0.17243	0.021554	2.1488
10000	1	0.181046	0.2746	0.65931	0.65931	0.2746
10000	2	0.181046	0.4844	0.373754	0.186877	0.9688
10000	4	0.181046	0.4642	0.390018	0.097505	1.8568
10000	8	0.181046	0.543	0.333419	0.041677	4.344
20000	1	0.714962	0.9036	0.791238	0.791238	0.9036
20000	2	0.714962	1.1044	0.647376	0.323688	2.2088
20000	4	0.714962	1.106	0.64644	0.16161	4.424
20000	8	0.714962	1.3534	0.528271	0.066034	10.8272

14.4 Tabelarni prikaz dobijenih rezultata

Analiza eksperimentalnih rezultata pokazuje da paralelizacija Selection sort algoritma ne donosi značajne performanse, već naprotiv – u većini slučajeva usporava izvršavanje. Najveći ostvareni speedup iznosi svega 0.791238 pri $n = 20000$ i 2 thread-a, što je daleko ispod teorijskog maksimuma, dok je najniži zabilježen 0.028772 kod $n = 1000$ i 8 thread-ova, što jasno pokazuje degradaciju performansi pri pokušaju prekomjerne paralelizacije. Efikasnost takođe opada sa porastom broja thread-ova, krećući se od relativno dobrih 0.323688 (pri $n = 20000$, threads = 2) do ekstremno loših vrijednosti ispod 0.01 (npr. 0.003596 za $n = 1000$, threads = 8).

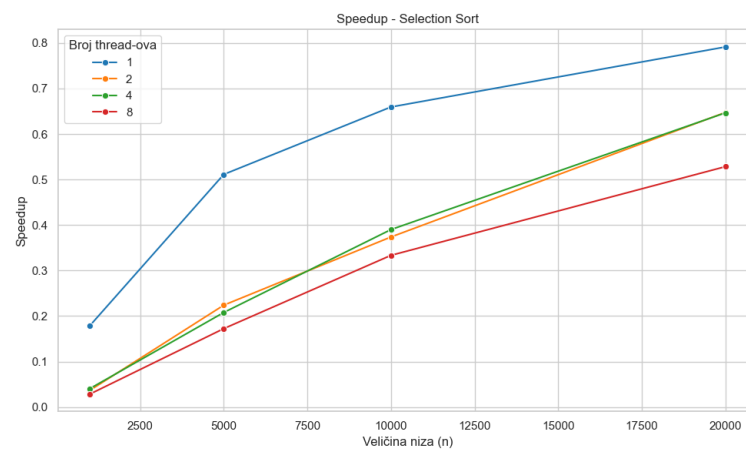
Pored toga, mjereni cost, koji predstavlja ukupno potrošeno vrijeme svih thread-ova, dodatno potvrđuje neefikasnost pristupa: najniža vrijednost 0.1008 zabilježena je kod manjeg ulaza sa samo 2 thread-a, dok je najviša vrijednost 10.8272 postignuta

za $n = 20000$ i 8 thread-ova – gdje je izvršavanje ne samo neefikasno, već i višestruko skuplje u resursima. Ovi rezultati jasno potvrđuju teorijsku tvrdnju da Selection Sort, zbog svojih sekvencijalnih zavisnosti u svakom koraku, nije podoban za efektivnu paralelizaciju pomoću OpenMP pristupa.



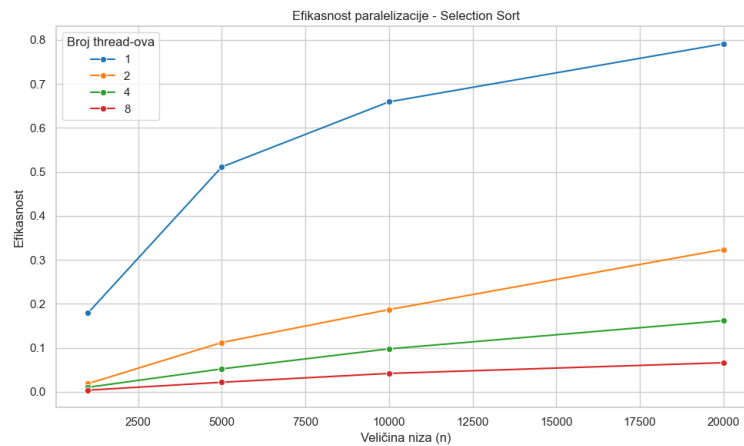
14.5 Grafikon koji prikazuje vrijeme izvršavanja u zavisnosti od veličine niza

Na grafikonu 14.5 jasno se vidi da paralelne verzije sa većim brojem thread-ova (posebno 4 i 8) imaju duže vrijeme izvršavanja u odnosu na sekvencijalnu verziju. Ova razlika postaje izraženija sa porastom veličine ulaza, što ukazuje na to da trošak upravljanja paralelnim sekcijama nadmašuje eventualnu dobit od raspodjele posla.



14.6 Grafikon koji prikazuje speedup u odnosu na veličinu niza

Na grafikonu 14.6 uočava se da vrijednosti ostaju znatno ispod idealne skale, čak i za najveći ulaz ($n = 20000$), speedup ne prelazi 0.8. Najveći speedup je zabilježen za upotrebu 2 thread-a, dok se sa 4 i 8 niti dobija sve slabiji rezultat, što je u skladu sa teorijom o ograničenju paralelizacije kod algoritama sa zavisnostima između iteracija.



14.7 Grafikon koji prikazuje efikasnost paralelizaciju u odnosu na veličinu niza

Graf 14.7 oslikava efikasnost paralelizacije koja drastično opada kako raste broj niti, te se kod 8 thread-ova kreće ispod 0.1 za sve veličine nizova. Ovi grafovi potvrđuju da pokušaj paralelizacije Selection Sort algoritma rezultira degradacijom performansi, te da njegova priroda jednostavno nije pogodna za efikasno izvršavanje u paralelnom okruženju.

4. INSERTION SORT ALGORITAM

Insertion sort, ili sortiranje umetanjem, jednostavan je algoritam za sortiranje niza elemenata koji radi tako što jedan po jedan uzima elemente iz nesortiranog dijela niza i postavlja ih na njihovo pravo mjesto unutar već sortiranog dijela niza.

U početku, algoritam pretpostavlja da je prvi element niza već sortiran. Nakon toga, uzima se sljedeći element iz nesortiranog dijela niza i poredi se redom s elementima u sortiranom dijelu, s desna na lijevo. Ako je ovaj odabrani element manji (u slučaju uzlaznog sortiranja) od elementa u sortiranom dijelu, elementi se pomjeraju za jedno mjesto udesno kako bi napravili mjesta za ubacivanje novog elementa na njegovo odgovarajuće mjesto. Proces se nastavlja sve dok se ne pronađe prava pozicija za novi element. Ovaj postupak se ponavlja za sve preostale elemente u nesortiranom dijelu niza, sve dok čitav niz ne postane sortiran.

Princip rada ovog algoritma je veoma sličan načinu na koji ljudi ručno sortiraju karte tokom kartaške igre. Prva uzeta karta predstavlja osnov za slaganje ostalih karata. Ako je sledeća uzeta karta veća od karte u ruci, stavlja se s desne strane, inače s lijeve strane. Na isti način, ostale nerazvrstane karte se uzimaju i stavljaju na svoje mjesto.

Dakle, sortiranje umetanjem korak po korak gradi sortirani niz, uvijek osiguravajući da je dio niza koji je obrađen uredno sortiran. Prednosti ovog algoritma su jednostavna implementacija i efikasnost za manje ili djelimično sortirane nizove. Međutim, njegova vremenska složenost u najgorem slučaju je $O(n^2)$, što ga čini manje efikasnim za velike nizove.

4.1 Implementacija algoritma

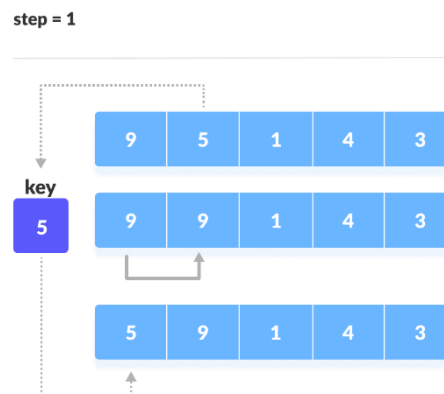
Pretpostavimo da trebamo sortirati sljedeći niz.



15. Početni niz koji treba sortirati

Prvi korak

Pretpostavljamo da je prvi element (9) već sortiran. Sada prelazimo na drugi element (5) i privremeno ga izdvajamo kao ključ (key). Zatim uspoređujemo ovaj ključ (5) s prvim (sortiranim) elementom (9). Budući da je ključ (5) manji od prvog elementa (9), pomjeramo 9 jedno mjesto udesno i stavljamo ključ (5) na prvu poziciju u nizu. Nakon završetka ovog koraka niz je [5, 9, 1, 4, 3]. Dakle, ako je prvi element veći od ključa, ključ se postavlja ispred prvog elementa.

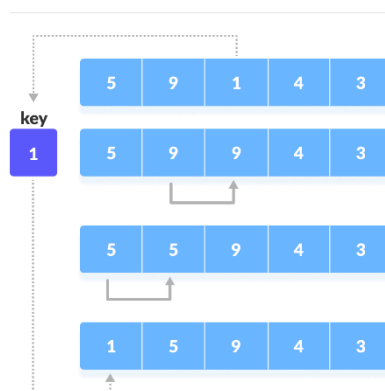


16. Ilustracija prvog koraka

Drugi korak

Sad imamo dva elementa ([5, 9]) koji su već sortirani. Sljedeći nesortirani element (treći po redu) je broj 1 koji opet izdvajamo kao ključ (key). Poredeći ključ (1) s elementima u sortiranom dijelu niza (9, zatim 5), uočavamo da je broj 1 manji od oba elementa. Pomjeramo oba ova elementa (9 i 5) za jedno mjesto udesno kako bismo napravili mjesta za naš ključ (1). Postavljamo ključ (1) na početak niza jer je najmanji. Nakon završetka ovog koraka niz je [1, 5, 9, 4, 3].

step = 2

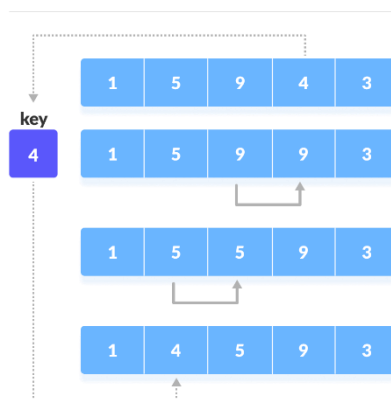


17. Ilustracija drugog koraka

Treći korak

U ovom trenutku prva tri elementa ([1, 5, 9]) su već sortirana. Sljedeći nesortirani element (četvrti po redu) je broj 4 koji uzimamo kao ključ. Ključ (4) uspoređujemo s prethodno sortiranim elementima (9, 5, 1), krećući od posljednjeg sortiranog elementa prema početku. Element 9 je veći od ključa 4 pa ga pomjeramo jedno mjesto udesno. Zatim uspoređujemo ključ 4 s elementom 5 koji je također veći od ključa, pa i njega pomjeramo jedno mjesto udesno. Element 1 je manji od ključa 4, pa ne pravimo dalja pomjeranja. Ključ (4) sada stavljamo na poziciju odmah iza elementa 1. Nakon završetka ovog koraka niz je [1, 4, 5, 9, 3].

step = 3



18. Ilustracija trećeg koraka

Na isti način nastavljamo iteracije za svaki naredni nesortirani element (npr. broj 3), poredeći ga redom s elementima iz sortiranog dijela i umećući ga na pravo mjesto, dok cijeli niz ne bude potpuno sortiran.

Ovim postupkom, Insertion sort postepeno proširuje sortirani dio niza sve dok cijeli niz ne bude pravilno uređen.

step = 4



19. Ilustracija četvrtog koraka

Na slici 20 je prikazan pseudo-kod algoritma Insertion sort. Prvo se označava prvi element u nizu (*array*) kao već sortiran. Nakon toga, za svaki sljedeći nesortirani element (označen kao *X*) iz niza izvršava se proces „umetanja“. Element *X* se uzima i poredi s elementima koji su već sortirani. Ovaj proces poređenja i pomjeranja se vrši pomoću petlje koja kreće od indeksa posljednjeg sortiranog elementa (*lastSortedIndex*) prema indeksu 0. Ako je trenutni element na poziciji *j* veći od elementa *X*, trenutni element (*array[j]*) se pomjera za jedno mjesto udesno. Kada algoritam naiđe na element koji nije veći od *X*, prekida petlju, te se element *X* postavlja (umeće) na trenutno slobodnu poziciju. Postupak se ponavlja za sve preostale nesortirane elemente sve dok cijeli niz ne postane sortiran.

```
insertionSort(array)
  mark first element as sorted
  for each unsorted element X
    'extract' the element X
    for j <- lastSortedIndex down to 0
      if current element j > X
        move sorted element to the right by 1
      break loop and insert X here
  end insertionSort
```

20. Pseudo-kod za Insertion sort algoritam

4.2 Složenost algoritma

4.2.1 Vremenska složenost (Time complexity)

Insertion sort ima različite vremenske složenosti u zavisnosti od redoslijeda elemenata u nizu:

- **Najgori slučaj: $O(n^2)$**

Najgori slučaj se događa kada elementi niza imaju obrnut redoslijed u odnosu na željeni (na primjer, niz je silazno sortiran, a mi ga želimo uzlazno sortirati). U ovom slučaju, svaki element iz nesortiranog dijela mora se porediti sa svim elementima već sortiranog dijela. U prvoj iteraciji, pravi se maksimalno 1

poređenje, u drugoj iteraciji maksimalno 2 poređenja, u trećoj maksimalno 3 poređenja ..., u posljednjoj iteraciji maksimalno $(n-1)$ poređenja. Kada saberemo broj poređenja kroz sve iteracije, dobijamo izraz:

$$1+2+3+\dots+(n-1)=n(n-1)/2$$

Ovaj izraz je približno jednak $n^2/2$, što znači da je vremenska složenost u najgorem slučaju $O(n^2)$.

- **Najbolji slučaj: $O(n)$**

Najbolji slučaj se događa kada je niz već sortirani. U ovom slučaju, svaki sljedeći element koji algoritam provjerava je već na svom odgovarajućem mjestu. Zbog toga se, u najboljem slučaju, unutrašnja petlja (koja poredi elemente) nikada ne izvršava, već algoritam prolazi kroz niz samo jednom. Samim tim, složenost je linearna, odnosno $O(n)$.

- **Prosječni slučaj: $O(n^2)$**

Prosječni slučaj nastaje kad su elementi nasumično raspoređeni. U praksi ovo je najčešći scenarij. U ovom slučaju, broj usporedbi je u prosjeku sličan kao i u najgorem slučaju, što rezultira prosječnom složenošću od $O(n^2)$.

4.2.2 Prostorna složenost (*Space complexity*)

Insertion sort ima prostornu složenost $O(1)$, što znači da koristi konstantnu količinu dodatnog prostora. Konkretno, ovaj algoritam koristi samo jednu dodatnu varijablu (*key*) za privremeno čuvanje trenutnog elementa kojeg treba umetnuti na odgovarajuće mjesto.

Insertion sort je stabilan algoritam, što znači da elementi s istim vrijednostima nakon sortiranja ostaju u istom redoslijedu kao što su bili u originalnom nizu. Stabilnost je važna kod sortiranja složenijih struktura podataka, gdje je redoslijed jednako vrijednih elemenata bitan.

Insertion sort je jednostavan algoritam, efikasan za male ili već djelimično sortirane nizove. Međutim, zbog vremenske složenosti $O(n^2)$ u prosječnom i najgorem slučaju, nije pogodan za sortiranje velikih nizova.⁷

4.3 Paralelizacija algoritma

Iako je Insertion sort jednostavan i efikasan za male ili djelimično sortirane nizove, njegova paralelizacija predstavlja značajan izazov zbog sekvencijalne zavisnosti između koraka algoritma. U osnovi, Insertion sort funkcionira tako što za svaki novi element iz nesortiranog dijela niza traži odgovarajuću poziciju u već sortiranom dijelu, pomjerajući elemente udesno dok ne pronađe pravo mjesto. Ovaj proces zahtijeva da sortirani dio niza bude već ispravno formiran prije nego što se obradi sljedeći element. Zbog toga, svaka iteracija algoritma zavisi od rezultata prethodne, što ga čini sekvencijalnim po svojoj prirodi. Na primjer, treći element se ne može pravilno smjestiti dok prethodna dva nisu već sortirana. Ova zavisnost

⁷ „Insertion Sort Algorithm“, Programiz, pristupljeno 29. mart 2025, dostupno na: <https://www.programiz.com/dsa/insertion-sort>.

onemogućava klasičnu paralelizaciju u kojoj bi više elemenata moglo simultano da „traže“ svoje mjesto, jer bi to moglo dovesti do narušavanja reda i neispravnog sortiranja.

Ipak, određeni pokušaji ograničene paralelizacije postoje, i najčešće su teorijskog ili obrazovnog karaktera. Jedna od varijanti jeste pipelining pristup, gdje se različite faze umetanja izvode u kaskadi kroz više procesa. U tom modelu, svaki proces „prima“ element, obrađuje ga djelimično i prosleđuje dalje. Međutim, ovakav pristup ne ostvaruje stvarni paralelizam u smislu vremenskog ubrzanja, već više simulira sekvencijalni tok kroz niz procesa. Drugi pristup pokušava da podijeli niz na više manjih podnizova, sortira ih paralelno pomoću insertion sorta, i zatim izvrši spajanje (merge). Ipak, ovakav pristup praktično uvodi potpuno novi algoritam, koji više nije klasičan Insertion sort, već hibrid sa metodama nalik merge sortu.

Zbog svega navedenog, može se zaključiti da je Insertion sort izuzetno teško paralelizovati na efikasan način, jer su njegove osnovne operacije međuzavisne. Čak i kada bi se dio algoritma paralelizovao, dobici u performansama bi bili minimalni, dok bi složenost implementacije i komunikacije među procesima porasla.

U modelu u kojem se koristi jedan proces po procesoru, svaki proces bi imao ograničenu i nesamostalnu ulogu, jer ne bi mogao nezavisno da izvrši umetanje svog elementa bez znanja o stanju prethodnih elemenata u nizu.

Dakle, zbog svoje sekvencijalne strukture i zavisnosti među koracima, Insertion sort se ne smatra pogodnim za paralelnu obradu. Njegova primjena u paralelnim algoritmima je rijetka i više eksperimentalna nego praktična

4.4 Implementacija u okviru projekta

Projekat, slično pristupu kod Selection sorta, obuhvata implementaciju i testiranje sekvencijalne i, u granicama koje algoritam dozvoljava, paralelne verzije Insertion sort algoritma. Osnovni cilj je da se putem praktičnog koda i merenja performansi prikaže da ni ovaj algoritam, zbog svoje prirodne zavisnosti između iteracija, nije pogodan za efikasnu paralelizaciju.

Funkcija `insertionSort` prikazana na slici 20.1 predstavlja sekvencijalnu implementaciju Insertion sort algoritma, koja sortira vektor cijelih brojeva u rastućem redoslijedu. Algoritam započinje od drugog elementa niza i svaki element tretira kao ključ koji treba ubaciti na odgovarajuću poziciju unutar prethodnog, već sortiranog dijela niza. U tu svrhu koristi se promjenljiva j koja se kreće unazad kroz sortirani dio, poredeći elemente s ključem i pomjerajući one koji su veći jedno mjesto udesno. Kada se pronađe ispravna pozicija, ključ se smješta na to mjesto. Ova implementacija vjerno odražava osnovnu ideju insertion sorta – postepeno "ubacivanje" svakog elementa na njegovo mjesto.

```

#include <iostream>
#include <vector>

void insertionSort(std::vector<int>& arr) {
    int n = arr.size();
    for (int i = 1; i < n; ++i) {
        int key = arr[i];
        int j = i - 1;

        // Pomjeranje elemenata većih od ključa
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

```

20.1 Sekvencijalna implementacija Insertion sort algoritma u C++

Funkcija `parallelInsertionSort` prikazana na slici 20.1 predstavlja pokušaj paralelizacije Insertion sort algoritma uz korišćenje OpenMP-a. U ovoj implementaciji koristi se direktiva `#pragma omp parallel for` kako bi se više iteracija spoljašnje petlje izvršavale istovremeno – svaka nit paralelno pokušava da sortira jedan element tako što ga ubacuje na odgovarajuće mjesto u prethodni dio niza. Međutim, insertion sort je algoritamski takav da svaka nova iteracija zavisi od rezultata prethodnih, pa paralelno izvršavanje može dovesti do konflikta pri pristupu podacima. Da bi se izbjegli ti konflikti, pristup elementima niza obezbijeđen je `#pragma omp critical` sekcijama, koje omogućavaju da samo jedna nit u datom trenutku mijenja ili čita elemente niza. Zbog toga ova verzija funkcioniše ispravno, ali praktično eliminiše svaku prednost paralelizacije jer se većina operacija izvršava serijski, uz dodatni overhead sinhronizacije. Upravo zbog toga, ova verzija je namjerno uključena u projekat kako bi se pokazalo da pokušaj paralelizacije algoritma koji je inherentno sekvencijalan ne daje bolje performanse – naprotiv, često ih pogoršava.

```

void parallelInsertionSort(std::vector<int>& arr) {
    int n = arr.size();

    #pragma omp parallel for default(none) shared(arr, n)
    for (int i = 1; i < n; ++i) {
        int key, j;

        // Prvo uzmi ključ (mora izvan kritične sekcije)
        #pragma omp critical
        {
            key = arr[i];
            j = i - 1;
        }

        // Premještanje elemenata koji su veći od ključa
        while (j >= 0) {
            int temp;
            #pragma omp critical
            {
                temp = arr[j];
            }

            if (temp > key) {
                #pragma omp critical
                {
                    arr[j + 1] = temp;
                }
                j--;
            } else {
                break;
            }
        }

        #pragma omp critical
        {
            arr[j + 1] = key;
        }
    }
}

```

20.2 Paralelna implementacija Insertion sort algoritma u C++

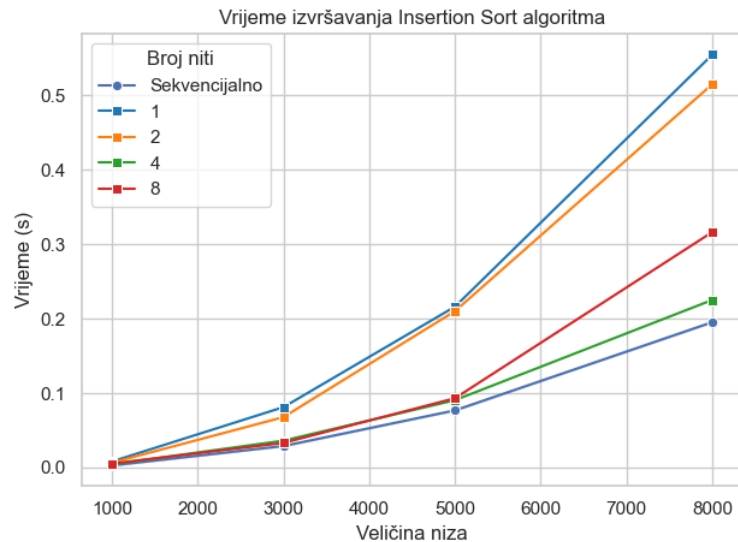
Kao i za prethodno opisane algoritme, projekat uključuje i za Insertion sort algoritam program *InsertionSortMain.cpp* s istom logikom. Program automatski generiše nizove slučajnih brojeva različitih veličina i izvršava sortiranje pomoću obje verzije algoritma, pri čemu mjeri vrijeme izvršavanja više puta radi dobijanja prosječnih vrijednosti. Sekvencijalno vrijeme se mjeri korišćenjem *std::chrono*, dok se paralelno vrijeme bilježi pomoću *omp_get_wtime()*, a broj niti se postavlja funkcijom *omp_set_num_threads*. Na osnovu izmjerenih vremena izračunavaju se metrike kao što su speedup, efikasnost i trošak (cost), i svi rezultati se snimaju u CSV fajl koji se koristi za analize i vizualizacije pomoću fajla *InsertionSortGraphs.py*.

4.4.1 Analiza rezultata

n	threads	seq_time	par_time	speedup	efficiency	cost
1000	1	0.002958	0.0082	0.360731	0.360731	0.0082
1000	2	0.002958	0.0066	0.448182	0.224091	0.0132
1000	4	0.002958	0.0044	0.67227	0.168068	0.0176
1000	8	0.002958	0.0054	0.547779	0.068472	0.0432
3000	1	0.029001	0.0812	0.35716	0.35716	0.0812
3000	2	0.029001	0.0678	0.427749	0.213875	0.1356
3000	4	0.029001	0.0362	0.801144	0.200286	0.1448
3000	8	0.029001	0.0332	0.873537	0.109192	0.2656
5000	1	0.076918	0.2162	0.355773	0.355773	0.2162
5000	2	0.076918	0.2102	0.365929	0.182964	0.4204
5000	4	0.076918	0.0906	0.848987	0.212247	0.3624
5000	8	0.076918	0.0932	0.825302	0.103163	0.7456
8000	1	0.195392	0.5546	0.352312	0.352312	0.5546
8000	2	0.195392	0.515	0.379402	0.189701	1.03
8000	4	0.195392	0.2252	0.867638	0.216909	0.9008
8000	8	0.195392	0.3164	0.617547	0.077193	2.5312

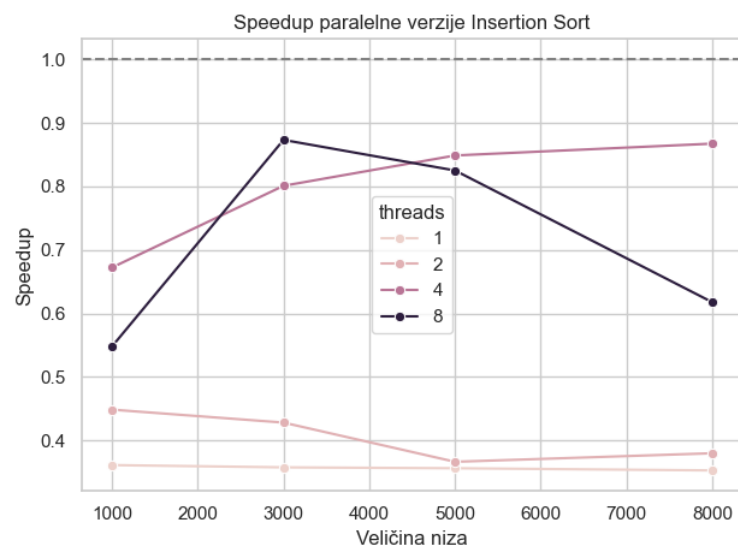
20.3 Tabelarni prikaz dobijenih rezultata

Dobijeni rezultati, prikazani u tabeli 20.3, jasno potvrđuju da Insertion sort algoritam ne pokazuje značajno poboljšanje performansi kada se paralelizuje, čak i pri većim veličinama ulaza. U svim testiranim slučajevima, sekvencijalno vrijeme izvršavanja ostaje konstantno unutar grupe istih ulaznih podataka, dok paralelno vrijeme, i pored povećanja broja niti, ne pokazuje stabilan trend ubrzanja. Najveći zabilježeni speedup iznosi 0.8735 za ulaz veličine 3000 sa 8 niti, što je i dalje manje od idealne vrijednosti 1, dok je efikasnost istovremeno pala na svega 0.1091. Najniža efikasnost zabilježena je za mali ulaz ($n=1000$) sa 8 niti, gdje iznosi svega 0.0684. Istovremeno, metrika cost (koja označava ukupnu utrošenu procesorsku snagu) značajno raste sa brojem niti, što dodatno potvrđuje da paralelizacija donosi više štete nego koristi. U slučaju ulaza $n=8000$ i 8 niti, cost dostiže najvišu vrijednost od 2.5312, što je više od trostrukog vremena sekvencijalne verzije. Ovi podaci jasno ilustruju da pokušaj paralelizacije Insertion sort algoritma dovodi do povećanog vremenskog i računarskog troška, što je posljedica njegove sekvencijalne prirode i potrebe za čestom sinhronizacijom među nitima.



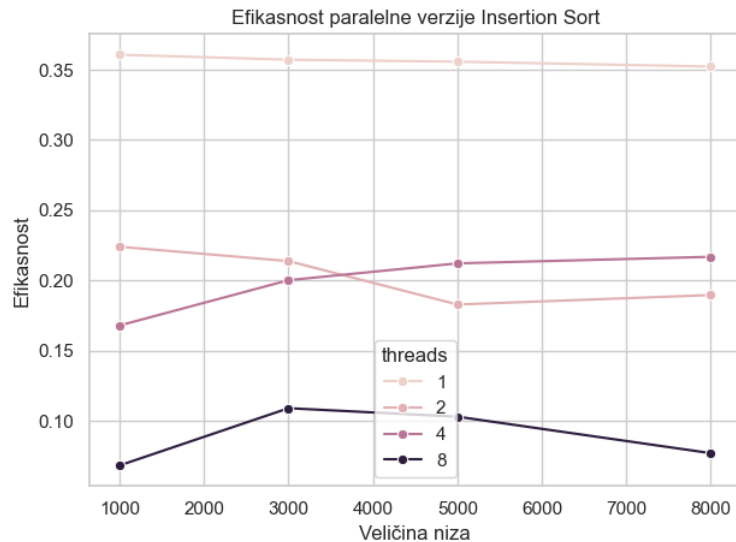
20.4 Grafikon koji prikazuje vrijeme izvršavanja u zavisnosti od veličine niza

Na grafikonu 20.4 vidi se da sekvencijalna verzija algoritma gotovo u svim slučajevima ima najkraće vrijeme izvršavanja, naročito kod manjih ulaza. Paralelna verzija sa 1 i 2 niti je konstantno sporija, dok verzije sa 4 i 8 niti donekle poboljšavaju vrijeme, ali ni u jednom slučaju ne nadmašuju sekvencijalnu implementaciju. Povećanje broja niti ne donosi proporcionalno smanjenje vremena – što ukazuje na neefikasnu upotrebu paralelizacije zbog algoritamske zavisnosti među iteracijama.



20.5 Grafikon koji prikazuje speedup u zavisnosti od veličine niza

Na grafikonu 20.5 jasno se uočava da vrijednost speedup-a nigdje ne prelazi 1, što znači da paralelna verzija nikada nije bila brža od sekvencijalne. Najveći speedup (~ 0.87) ostvaren je kod veličine niza 3000 sa 8 niti, ali i tada rezultat ne opravdava upotrebu paralelizacije. Verzije sa manjim brojem niti (1 i 2) konstantno pokazuju niži speedup, što dodatno potvrđuje da overhead od paralelnih niti nadmašuje svaku potencijalnu dobit.



20.6 Grafikon koji prikazuje efikasnost paralelne verzije u odnosu na veličinu niza

Uočljivo je na grafikonu 20.6 da sa povećanjem broja niti efikasnost opada, a posebno je niska za 8 niti (ispod 0.1 u nekim slučajevima). Ovo znači da se sve više procesorske snage troši, ali se ne ostvaruje proporcionalna ubrzanja. Najvišu efikasnost pokazuje verzija sa jednom niti (što praktično odgovara sekvencijalnoj obradi), dok višestruke niti pokazuju da paralelna implementacija nije skalabilna.

Svi prikazani grafikoni potvrđuju eksperimentalnu tezu tj. da Insertion sort algoritam nije pogodan za paralelizaciju. Pokušaj paralelne obrade ne donosi ni brzinu ni efikasnost, već u većini slučajeva rezultira sporijim izvršavanjem i većim troškovima. Ovi rezultati su posljedica sekvencijalne prirode algoritma, gdje svaka iteracija zavisi od prethodnih, što onemogućava istinsku paralelnu distribuciju posla.

5. MERGE SORT ALGORITAM

Merge sort je efikasan algoritam za sortiranje koji se zasniva na strategiji *Podijeli i osvoji* (Divide and Conquer). Njegova osnovna ideja je da se veliki problem sortiranja podijeli na više manjih i jednostavnijih podproblema, koji se zatim rješavaju nezavisno, nakon čega se njihova rješenja kombinuju u jedno konačno i potpuno rješenje. Algoritam se sastoji iz tri glavna koraka: podjela (divide), rješavanje (conquer) i spajanje (combine).

U fazi podjele, niz koji se sortira rekurzivno se dijeli na dvije približno jednake polovine. Ukoliko niz sadrži više od jednog elementa, izračunava se srednji indeks q između lijevog i desnog kraja niza, i niz se razdvaja na dva dijela – lijevu i desnu polovinu. Ovaj proces se ponavlja sve dok svi dobijeni podnizovi ne sadrže samo po jedan element. Takvi nizovi se smatraju trivijalno sortiranim, jer jedan element ne može biti u pogrešnom redosledu.

Kada su svi nizovi svedeni na pojedinačne elemente, započinje faza rješavanja, tj. spajanja podnizova. U ovoj fazi, dva po dva podniza koja su već sortirana kombinuju

se u veće sortirane cjeline. Tokom spajanja se porede početni elementi oba podniza, i manji od njih se ubacuje u novi niz. Proces se nastavlja sve dok se svi elementi ne iskoriste, pri čemu se preostali elementi jednog od podnizova jednostavno dodaju na kraj. Spajanje se ponavlja sve dok se svi nivoi rekurzije ne završe i ne dobije se konačan, sortirani niz.

Merge Sort je stabilan algoritam, što znači da čuva originalni redosljed elemenata sa jednakom vrijednošću. Međutim, pošto koristi dodatne pomoćne nizove tokom procesa spajanja, nije in-place algoritam. Njegova vremenska složenost je $O(n \log n)$ u svim slučajevima (najbolji, prosječan i najgori), što ga čini pogodnim za obradu velikih količina podataka.

5.1 Implementacija algoritma

Pretpostavimo da trebamo da sortiramo sljedeći niz:

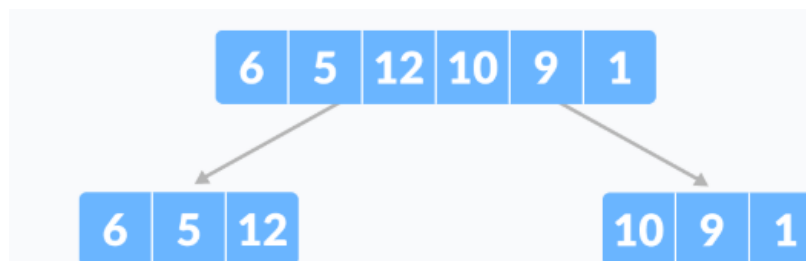


21. Početni niz koji treba sortirati

Prvi korak – Divide (podijeli)

Prva operacija algoritma je podjela niza na dvije jednake polovine:

Lijevi podniz: (6, 5, 12) i desni podniz: (10, 9, 1).

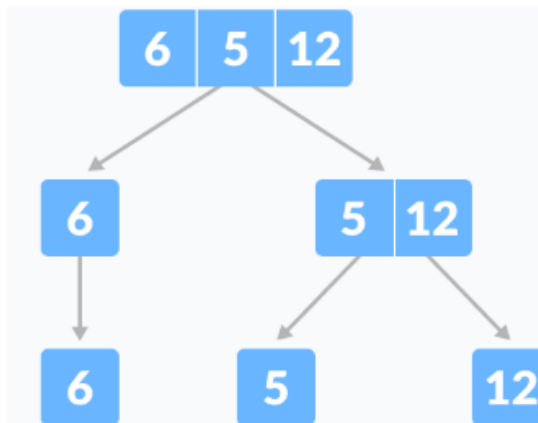


22. Ilustrovani prikaz niza nakon prvog koraka

Drugi korak

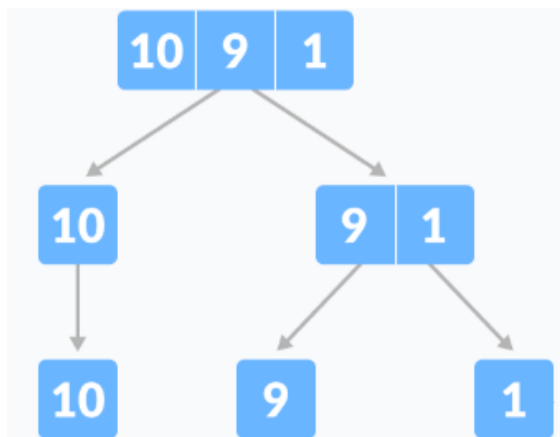
U ovom koraku se lijevi podniz rekurzivno dijeli sve dok se ne dobije sortirani niz dužine jedan. Postupak je sljedeći. Niz (6, 5, 12) se dijeli na 2 podniza: (6) i (5, 12). Zatim (5, 12) ide u još jednu podjelu: (5) i (12).

Sada imamo nizove: (6), (5) i (12). Svaki sadrži jedan element, što znači da su to bazni slučajevi algoritma (sortirani po definiciji).



23. Ilustrovani prikaz lijevog podniza nakon koraka broj 2

Isti postupak se ponavlja i za desni podniz (10, 9, 1). Prvo imamo podnizove (10) i (9, 1). Zatim se rekurzivno postupak ponavlja za niz (9, 1), te na kraju ovoga koraka imamo 3 niza sa po jednim elementom: (10), (9), i (1).



24. Ilustrovani prikaz desnog podniza nakon koraka broj 2

Treći korak – spajanje

Sada počinje faza kombinovanja, gdje se sortirani nizovi sa po jednim elementom spajaju u veće sortirane podnizove.

- Lijeva grana spajanja:
 1. Prvo se spajaju (5) i (12):
 - a. 5 je manji od 12 → rezultat: (5, 12)
 2. Zatim se (6) i (5, 12) spajaju:
 - a. 6 i 5 → 5 ide prvi
 - b. 6 i 12 → 6 ide drugi
 - c. ostaje 12 → ide treći
 → rezultat: (5, 6, 12)

Niz (5, 6, 12) predstavlja konačno sortirani rezultat lijeve polovine početnog niza.

- Desna grana spajanja:

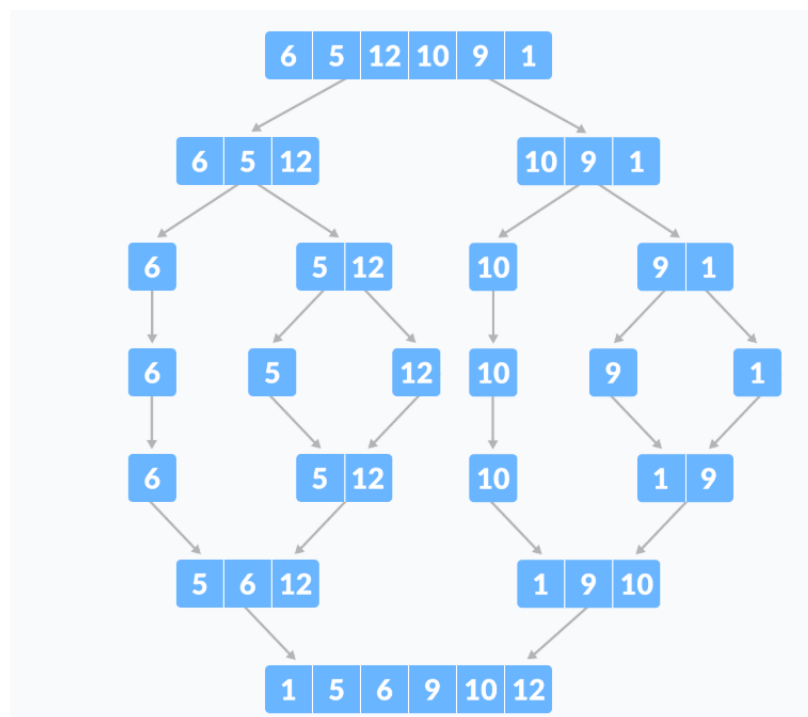
1. Prvo se spajaju (9) i (1):
 - a. 1 je manji od 9 → rezultat: (1, 9)
2. Zatim se (10) i (1, 9) spajaju:
 - a. 10 i 1 → 1 ide prvi
 - b. 10 i 9 → 9 ide drugi
 - c. ostaje 10 → ide treći
 → rezultat: (1, 9, 10)

Ovo je konačno sortirani rezultat desne polovine početnog niza.

Na kraju se ove dvije sortirane polovine spajaju u konačan, sortiran niz. Spajanje ide ovako:

1. 5 i 1 → 1 ide prvi
2. 5 i 9 → 5 ide drugi
3. 6 i 9 → 6 ide treći
4. 12 i 9 → 9 ide četvrti
5. 12 i 10 → 10 ide peti
6. ostaje 12 → ide šesti

Konačni rezultat sortiranja je niz: (1, 5, 6, 9, 10, 12).



25. Ilustracija kompletnog merge sort algoritma

Prikazana funkcija MergeSort(A, p, r) na slici 26 predstavlja rekurzivnu implementaciju Merge sort algoritma. Ona prima tri parametra: niz A koji treba sortirati, te indekse p i r koji označavaju početak i kraj dijela niza koji se sortira. Prvi korak u funkciji je provjera da li je lijeva granica veća od desne (if p > r). Ovo

predstavlja bazni slučaj rekurzije – kada je podniz prazan ili nepravilno definisan, funkcija se ne izvršava dalje i odmah se vraća. U praksi, bazni slučaj se često postavlja kao $p \geq r$, što obuhvata i situaciju kada podniz ima tačno jedan element, jer se takav niz smatra već sortiranim.

Ako bazni slučaj nije zadovoljen, funkcija računa sredinu podniza kao $q = (p + r) / 2$. Ova vrijednost predstavlja indeks koji dijeli trenutni podniz na dvije polovine: lijevu od p do q i desnu od $q + 1$ do r . Sljedeći koraci su dva rekurzivna poziva: `mergeSort(A, p, q)` za sortiranje lijeve polovine i `mergeSort(A, q + 1, r)` za sortiranje desne polovine. Svaki od ovih poziva ponaša se na isti način – dijeli podniz na još manje dijelove dok se ne dođe do nizova sa jednim elementom.

Kada su oba podniza sortirana, funkcija `merge(A, p, q, r)` se poziva kako bi se ta dva dijela spojila u jedan veći sortirani podniz. Funkcija `merge` ima ključnu ulogu u algoritmu – uzima dva sortirana podniza (`A[p..q]` i `A[q+1..r]`) i kombinuje ih u jedan novi niz `A[p..r]`, zadržavajući uzlazni redosljed elemenata. Na taj način se nizovi postepeno "grade" nazad, od najmanjih elemenata ka cijelini, sve dok se ne dobije potpuno sortirani niz. Cjelokupni proces kombinuje efikasnost podjele sa preciznim i kontrolisanim spajanjem, što omogućava Merge sort algoritmu da postigne stabilnu i optimalnu vremensku složenost od $O(n \log n)$ bez obzira na početni raspored elemenata.

```
MergeSort(A, p, r):  
    if p > r  
        return  
    q = (p+r)/2  
    mergeSort(A, p, q)  
    mergeSort(A, q+1, r)  
    merge(A, p, q, r)
```

26. Implementacija Merge sort algoritma

5.2 Složenost algoritma

5.2.1 Vremenska složenost (Time Complexity)

Merge sort algoritam ima konzistentnu vremensku složenost $O(n \log n)$ u najboljem, najgorem i prosječnom slučaju. Ova osobina čini Merge sort izuzetno predvidivim i efikasnim algoritmom za sortiranje, posebno kod velikih skupova podataka.

- **Najbolji slučaj – $O(n \log n)$**

U najboljem slučaju, niz je već djelimično ili potpuno sortirani. Ipak, Merge sort neće to iskoristiti kao što to rade neki drugi algoritmi (npr. Quick Sort), jer uvijek vrši rekurzivnu podjelu niza i spajanje, bez obzira na raspored elemenata.

Dakle, čak i u najboljem slučaju, vrijeme izvršavanja ostaje $O(n \log n)$, jer se niz dijeli $\log_2 n$ puta i svaki nivo podjele zahvata ukupno n elemenata tokom spajanja.

- **Prosječan slučaj – $O(n \log n)$**

U prosječnom slučaju, raspored elemenata u nizu je nasumičan. Merge sort i dalje rekurzivno dijeli niz i vrši spajanje na isti način, što rezultira istim brojem operacija kao i u najboljem slučaju. Zato i prosječna složenost ostaje $O(n \log n)$.

- **Najgori slučaj – $O(n \log n)$**

U najgorem slučaju, niz je obrnutog redosleda (potpuno nesortiran). Ipak, kao i u ostalim slučajevima, Merge sort i dalje funkcionira identično: vrši jednaku količinu rekurzivnih podjela i spajanja. Zbog toga čak i u najgorem mogućem rasporedu, broj operacija se ne povećava značajno i složenost ostaje $O(n \log n)$.

5.2.2 Prostorna složenost (Space Complexity)

Prostorna složenost Merge sort algoritma iznosi $O(n)$. To znači da Merge sort zahtijeva dodatni memorijski prostor proporcionalan veličini ulaznog niza.

Tokom faze spajanja (merge), Merge sort koristi dodatne pomoćne nizove da bi smjestio sortirane vrijednosti iz lijevog i desnog podniza. Zatim se ti elementi prepisuju nazad u glavni niz. Pošto u najgorem slučaju može biti potrebno čuvanje $n/2$ elemenata iz lijevog i $n/2$ iz desnog podniza, ukupna dodatna memorija koja se koristi je $O(n)$.

Zbog toga Merge sort nije in-place algoritam (ne sortira sve unutar samog niza, već koristi dodatni prostor za rad).

Dakle, Merge sort je stabilan, predvidiv i efikasan algoritam, ali za razliku od nekih drugih, žrtvuje memoriju da bi obezbijedio dosljedne performanse.⁸

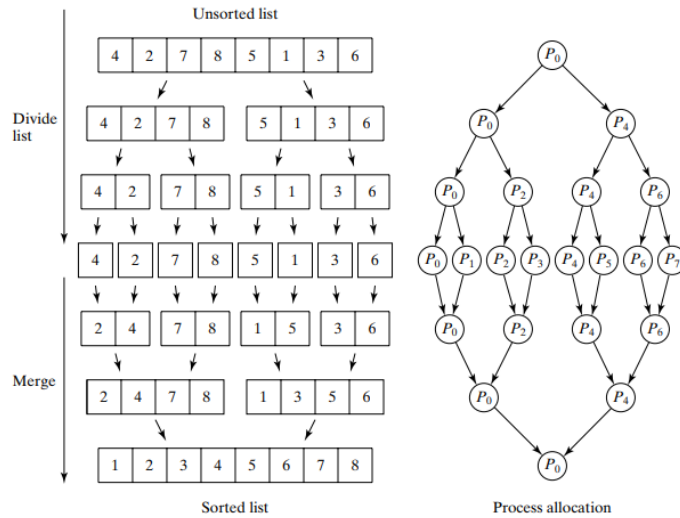
5.3 Paralelizacija algoritma

Paralelna implementacija algoritma Merge sort prirodno proizilazi iz njegove specifične „podijeli-pa-vladaj” (divide-and-conquer) strukture. Prilikom paralelizacije, prvobitna lista se sukcesivno dijeli na manje dijelove, koji se potom distribuiraju na više dostupnih procesora – slika 27.1. Proces dijeljenja počinje tako što glavni procesor (na primjer, procesor P0) šalje polovinu podataka drugom procesoru (P4). U sljedećem koraku, procesor P0 šalje četvrtinu početne liste procesoru P2, a procesor P4 šalje svoju četvrtinu liste procesoru P6. Ovaj postupak dijeljenja nastavlja se sve dok svi dostupni procesori ne dobiju svoje manje dijelove liste koje treba sortirati.

Kada se podaci rasporede na procesore, svaki od njih nezavisno vrši sortiranje primljenih podlista. Nakon što svaki procesor zasebno završi lokalno sortiranje, slijedi faza spajanja (engl. merge). U ovoj fazi obrađene podliste se sukcesivno kombinuju u veće sortirane cjeline. Proces spajanja odvija se od nižih nivoa stabla

⁸ “Merge Sort,” Programiz, pristupljeno 31. mart 2025, dostupno na: <https://www.programiz.com/dsa/merge-sort>

ka višim nivoima, pri čemu procesori na nižim nivoima šalju svoje već sortirane podliste procesorima na višim nivoima. Postupak se nastavlja sve dok se sve podliste ponovo ne sjedine u jednu potpuno sortiranu listu, koju na kraju posjeduje glavni procesor (P0).



27.1 - primjer alg. pomocu alociranja stabla procesora

Važno je istaći da se komunikacija među procesorima tokom faza dijeljenja i spajanja odvija simetrično, ali u suprotnim smjerovima. U fazi dijeljenja, komunikacija se odvija od glavnog procesora prema nižim nivoima stabla, dok u fazi spajanja komunikacija ide obrnuto, od nižih nivoa ka glavnom procesoru. Ukupno komunikaciono vrijeme ovakvog paralelnog pristupa može se približno izraziti formulom:

$$t_{\text{comm}} \approx 2(\log p)t_{\text{startup}} + 2nt_{\text{data}}$$

27.2 - formula za računanje vremena izvršavanja alg.

gdje je p broj procesora, $startup$ vrijeme potrebno za inicijalizaciju komunikacije, a $data$ vrijeme potrebno za prenos pojedinačnog podatka između procesora.

Što se tiče računanja, ono se odvija isključivo tokom faze spajanja podlista. U toj fazi svaki korak zahtijeva određeni broj operacija poređenja i prebacivanja elemenata. U najgorem slučaju, za paralelno spajanje svih podlista veličine s u svakom pojedinačnom koraku spajanja, potrebno je $2s-1$ operacija. Imajući u vidu da ukupno postoji $\log(n)$ koraka spajanja, ukupno računanje može se izraziti sljedećim zbirnim izrazom:

$$\sum_{i=1}^{\log(n)} (2^i - 1)$$

27.3 - formula za ukupno računanje vremena izvršavanja

Ovaj izraz, kada se sumira, rezultira ukupnim brojem koraka proporcionalnim veličini liste n , što daje konačnu vremensku složenost $O(n)$. Drugim riječima, kada zanemarimo komunikaciono vrijeme, računarska složenost paralelne implementacije Merge sort-a linearno zavisi od broja elemenata koje je potrebno sortirati.

Iako paralelizacija omogućava efikasnije iskorišćavanje procesorskih resursa i znatno ubrzava proces sortiranja, jedan od njenih nedostataka je neujednačeno opterećenje procesora tokom izvođenja algoritma. Na početku i pri kraju algoritma, pojedini procesori mogu biti manje angažovani, što dovodi do izvjesnog gubitka efikasnosti i neravnomjerne iskorišćenosti računarskih resursa.

Na slici 27.4 prikazane su prednosti i mane sekvencijalnog i paralelnog pristupa Merge sort algoritmu, respektivno.⁹

Sr. No	Sequential Merge	Parallel Merge
1	Merge sort is divide and conquer based approach	Merge sort is compare-exchange based approach
2	Single processor is used	Multiple processor are used
3	Slower execution	Faster execution
4	Less efficient	More efficient
5	Can solve problem in sequential manner	Can solve problem in parallel manner
6	$O(n \log n)$ in all three cases	$O(\log n)$

27.4 - sekvencijalni i paralelni Merge sort

5.4 Implementacija u okviru projekta

Sekvencijalna verzija Merge sort algoritma, prikazana na slici 27.4.1, implementirana je kroz dvije funkcije: mergeSort i merge. Funkcija mergeSort je rekurzivna i koristi princip “podijeli i osvoji” tako što dijeli ulazni niz na sve manje podnizove dok ne dođe do podniza sa jednim elementom, koji se smatra sortiranim. Niz se dijeli određivanjem srednjeg indeksa preko izraza $\text{mid} = \text{left} + (\text{right} - \text{left}) / 2$, čime se izbjegava potencijalno prekoračenje prilikom sabiranja velikih indeksa. Kada se dosegne osnovni slučaj rekurzije, funkcija prestaje da se poziva i počinje proces spajanja.

⁹ Barry Wilkinson i Michael Allen, Parallel Programming: Techniques and Applications Using Networked Workstations and Parallel Computers, Second Edition, Prentice Hall, 2005, str. 332–324.

```

void mergeSort(std::vector<int>& arr, int left, int right) {
    if(left >= right) return;

    int mid = left + (right - left) / 2;

    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);

    merge(arr, left, mid, right);
}

```

Slika 27.4.1 - sekvencijalna implementacija funkcije merge sort u C++

Tu nastupa funkcija merge – slika 27.4.2, koja uzima dva susjedna sortirana podniza (lijevi i desni) i kombinuje ih u jedan veći sortirani niz. To se postiže poređenjem elemenata iz obje polovine i njihovim ubacivanjem u privremeni vektor temp tako da se očuva rastući redoslijed. Nakon što su svi elementi iz obje polovine ubačeni u temp, taj vektor se prepisuje nazad u originalni niz na odgovarajućim pozicijama. Ovaj pristup garantuje stabilno sortiranje sa vremenskom složenošću $O(n \log n)$, gdje je n broj elemenata niza. Korišćenje `std::vector` omogućava fleksibilan rad sa nizom, a sam algoritam je deterministički i efikasan čak i za veće količine podataka.

```

void merge(std::vector<int>& arr, int left, int mid, int right) {
    std::vector<int> temp(right - left + 1);
    int i = left, j = mid + 1, k = 0;

    while(i <= mid && j <= right) {
        if(arr[i] <= arr[j])
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }

    while(i <= mid) temp[k++] = arr[i++];
    while(j <= right) temp[k++] = arr[j++];

    for(int t = 0; t < temp.size(); ++t)
        arr[left + t] = temp[t];
}

```

Slika 27.4.2 - sekvencijalna implementacija merge funkcije u C++

Kod prikazan na slici 27.4.3 implementira paralelnu verziju Merge sort algoritma koristeći C++ i OpenMP biblioteku. Sastoji se od dvije glavne funkcije: merge i parallelMergeSort. Funkcija merge je pomoćna funkcija zadužena za spajanje dva već sortirana podniza u jedan. Prima reference na vektor arr, te indekse left, mid i right, koji određuju granice podnizova koje treba spojiti. U ovoj funkciji kreira se privremeni vektor temp veličine jednakoju broju elemenata između left i right. Korišćenjem indeksa i i j, koji označavaju početke lijevog i desnog podniza, porede se elementi i manji od njih se smješta u temp. Kada jedan od podnizova bude iscrpljen, preostali elementi iz drugog podniza se jednostavno kopiraju u temp. Na kraju, sadržaj privremenog vektora se vraća u originalni niz arr, počevši od pozicije left, čime se ta sekcija niza efektivno sortira.

Glavna funkcija, parallelMergeSort, implementira rekurzivni Merge sort sa mogućnošću paralelizacije. Ona prima referencu na niz arr, indekse left i right, kao i parametar depth koji označava trenutni nivo rekurzije i podrazumijevano je postavljen na nulu. Na početku funkcije provjerava se da li je lijeva granica veća ili

jednaka desnoj ($\text{if}(\text{left} \geq \text{right})$), u kom slučaju funkcija prekida jer je podniz trivijalan (ima jedan ili nijedan element). Zatim se računa sredina niza, i niz se dijeli na dvije polovine. U zavisnosti od vrijednosti parametra `depth`, koristi se različit pristup: ako je dubina rekurzije manja ili jednaka četiri, tada se pomoću OpenMP direktive `#pragma omp parallel sections` omogućava da se lijeva i desna polovina niza sortiraju paralelno u različitim tredovima. Svaka grana poziva `parallelMergeSort` sa povećanom dubinom ($\text{depth} + 1$). Ukoliko je dubina veća od četiri, funkcija se poziva rekurzivno ali sekvencijalno, čime se izbjegava dalje grananje tredova. Na taj način se uvodi kontrolisana paralelizacija — algoritam koristi višetrednu obradu dokle god to ima smisla u odnosu na broj dostupnih jezgara i režijske troškove upravljanja tredovima. Na kraju, nakon što su lijeva i desna polovina niza sortirane, poziva se funkcija `merge` kojom se rezultati spajaju u jednu sortiranu cjelinu.

Važno je istaći da je upotreba parametra `depth` ključna za stabilnost i efikasnost algoritma. Uvođenje ograničenja na dubinu paralelizacije (u ovom slučaju do dubine 4) omogućava stvaranje najviše $2^4 = 16$ tredova, što odgovara realnim mogućnostima većine savremenih procesora. Time se izbjegava stvaranje prekomjernog broja tredova koje bi dovelo do zagušenja sistema i pada performansi. OpenMP ovdje funkcionira kao visok nivo apstrakcije za paralelno programiranje, omogućavajući korisniku da relativno jednostavno paralelizuje postojeći sekvencijalni algoritam bez potrebe za eksplicitnim upravljanjem tredovima. Na kraju, ova implementacija predstavlja izbalansiran kompromis između paralelnog ubrzanja i kontrole nad upotrebom resursa, što je čini pogodnom za upotrebu u akademskim i praktičnim okruženjima gdje su performanse od značaja.

```
void parallelMergeSort(std::vector<int>& arr, int left, int right, int depth = 0) {
    if(left >= right) return;

    int mid = left + (right - left) / 2;

    if(depth <= 4) { // ograničimo dubinu paralelizacije da se izbjegne previše niti
        #pragma omp parallel sections
        {
            #pragma omp section
            parallelMergeSort(arr, left, mid, depth + 1);

            #pragma omp section
            parallelMergeSort(arr, mid + 1, right, depth + 1);
        }
    } else {
        parallelMergeSort(arr, left, mid, depth + 1);
        parallelMergeSort(arr, mid + 1, right, depth + 1);
    }

    merge(arr, left, mid, right);
}
```

Slika 27.4.3 - paralelna implementacija merge sort alg. u C++

5.4.1 Analiza rezultata

Nakon implementacije sekvencijalne i paralelne verzije Merge sort algoritma, izvršena je analiza performansi korišćenjem automatski generisanih ulaza različitih veličina i broja tredova. Eksperiment je sproveden nad nizovima veličine 1.000, 5.000, 10.000 i 20.000 elemenata, dok su testirani brojevi tredova uključivali 1, 2, 4 i 8, u skladu sa realnim mogućnostima savremenih procesora. Za svaki slučaj,

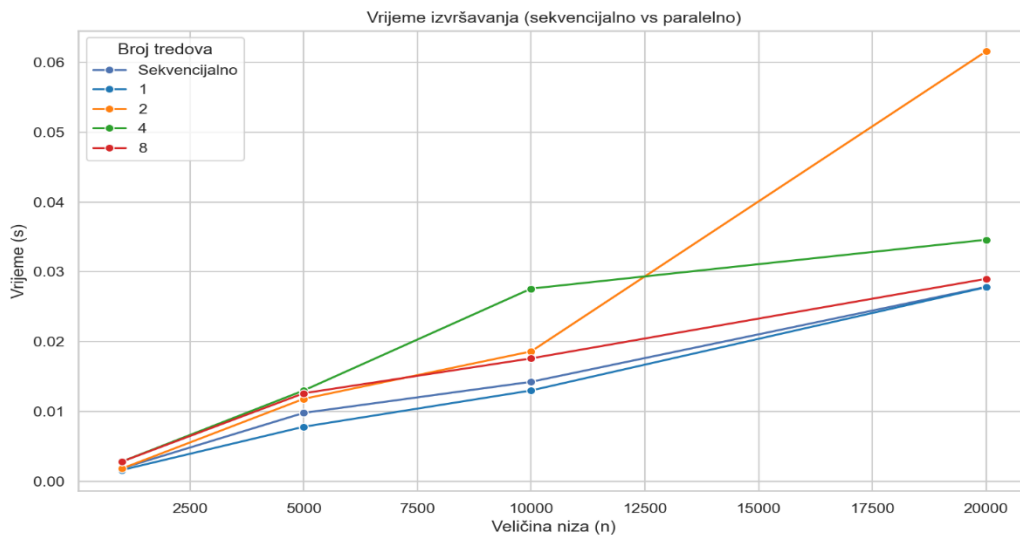
generisan je novi nasumični niz koji je identično korišćen za testiranje i sekvencijalne i paralelne verzije algoritma, čime je obezbijeđena konzistentnost ulaza. Vrijeme izvršavanja mjerilo se više puta (po defaultu 5) kako bi se dobila pouzdana prosječna vrijednost. Sekvencijalna verzija koristila je `std::chrono::high_resolution_clock` za precizno mjerenje vremena, dok je paralelna verzija koristila OpenMP-ovu funkciju `omp_get_wtime()`, pogodnu za višedretvena okruženja. Vizuelna reprezentacija rezultata prikazana je u Tabeli 27.4.4.

n	threads	seq_time	par_time	speedup	efficiency	cost
1000.0	1.0	0.001849	0.0016	1.155893	1.0	0.0016
1000.0	2.0	0.001849	0.0018	1.027471	0.513735	0.0036
1000.0	4.0	0.001849	0.0028	0.660524	0.165131	0.0112
1000.0	8.0	0.001849	0.0028	0.660524	0.082566	0.0224
5000.0	1.0	0.009789	0.0078	1.254935	1.0	0.0078
5000.0	2.0	0.009789	0.0118	0.829534	0.414767	0.0236
5000.0	4.0	0.009789	0.013	0.752961	0.18824	0.052
5000.0	8.0	0.009789	0.0126	0.776863	0.097108	0.1008
10000.0	1.0	0.014246	0.013	1.095881	1.0	0.013
10000.0	2.0	0.014246	0.0186	0.765937	0.382969	0.0372
10000.0	4.0	0.014246	0.0276	0.516176	0.129044	0.1104
10000.0	8.0	0.014246	0.0176	0.809457	0.101182	0.1408
20000.0	1.0	0.027848	0.0278	1.00174	1.0	0.0278
20000.0	2.0	0.027848	0.0616	0.452085	0.226042	0.1232
20000.0	4.0	0.027848	0.0346	0.804867	0.201217	0.1384
20000.0	8.0	0.027848	0.029	0.960289	0.120036	0.232

Tabela 27.4.4 - tabelarni prikaz rezultata performansi

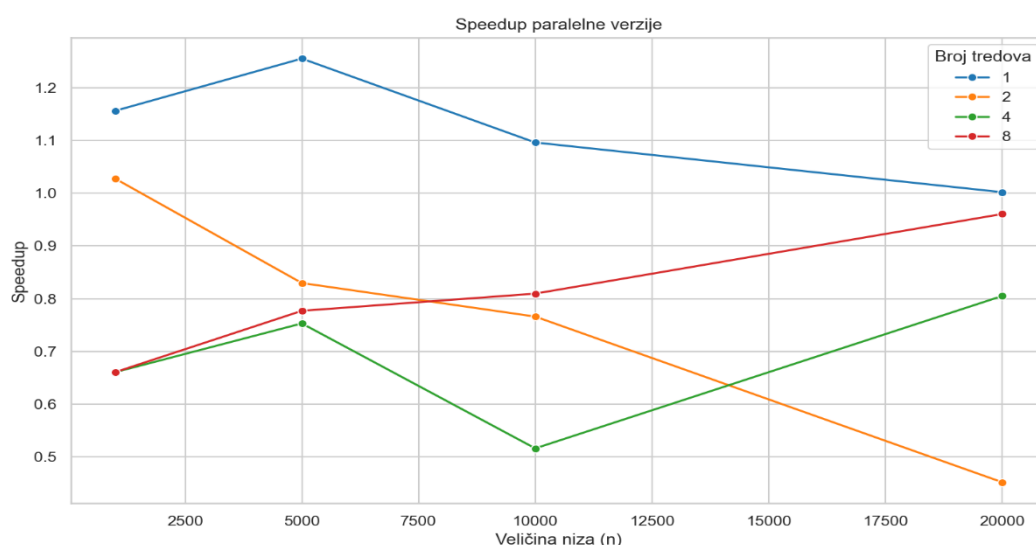
Za svaki eksperimentalni slučaj izračunate su tri ključne metrike: **speedup**, koji označava ubrzanje paralelne verzije u odnosu na sekvencijalnu; **efficiency**, kao omjer ubrzanja i broja tredova, čime se mjeri efikasnost iskorišćenosti paralelizacije; i **cost**, koji predstavlja ukupni vremenski resurs upotrijebljen kroz proizvod paralelnog vremena i broja tredova. Da bi se izbjegle numeričke nepravilnosti, efikasnost je ograničena na maksimalnu vrijednost 1.0. Svi rezultati su upisani u CSV fajl, što je omogućilo jednostavnu vizuelnu obradu podataka putem Python skripte.

Korišćenjem biblioteka `pandas`, `matplotlib` i `seaborn`, kreirana su tri grafikona: prikaz vremena izvršavanja, prikaz ubrzanja (speedup) i prikaz efikasnosti, čime su jasno uočeni trendovi i granice performansi algoritma u zavisnosti od ulazne veličine i broja tredova. Na osnovu grafikona, može se zaključiti da paralelna verzija Merge Sort algoritma ostvaruje značajno ubrzanje u odnosu na sekvencijalnu kod većih ulaznih nizova i da je broj tredova potrebno pažljivo optimizovati u odnosu na veličinu problema kako bi se izbjegao pad efikasnosti usljed prevelikog overhead-a



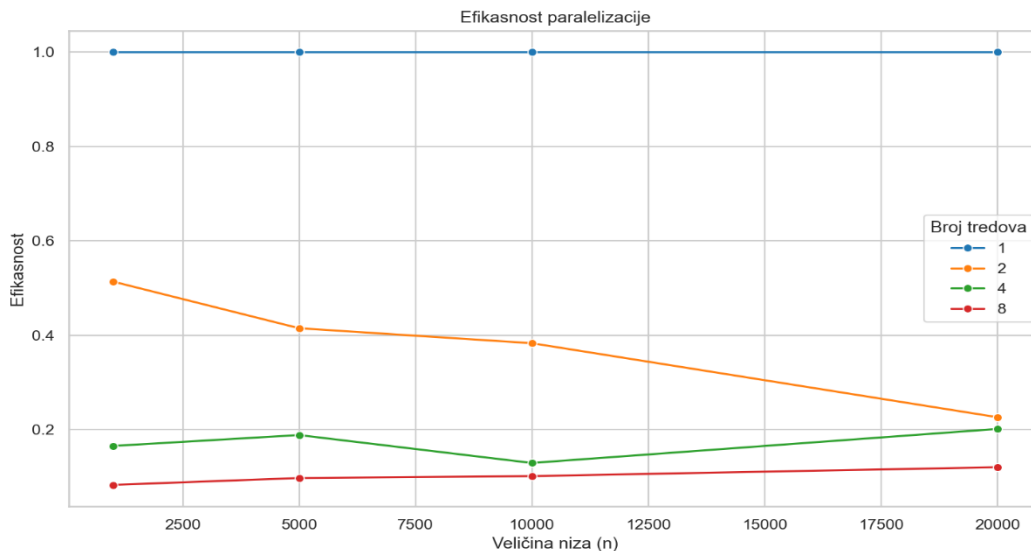
27.4.5 - Grafikon koji prikazuje vrijeme izvršavanja u zavisnosti od veličine niza

Na grafikonu 27.4.5 je prikazano vrijeme izvršavanja Merge Sort algoritma za različite veličine ulaznog niza, upoređujući sekvencijalnu i paralelne verzije sa 1, 2, 4 i 8 thread-ova. Uočava se da sekvencijalna verzija ima stabilan i predvidiv rast vremena sa porastom veličine niza, dok paralelne verzije ne pokazuju očekivano ubrzanje za sve konfiguracije. Konkretno, verzija sa 2 thread-ova ima najlošije performanse, naročito kod većih nizova, što ukazuje na neefikasnu podjelu posla i dodatni overhead. Samo pri većim vrijednostima n i većem broju thread-ova (4 i 8), paralelizacija počinje da pokazuje umjereno bolja vremena u poređenju sa sekvencijalnim pristupom.



27.4.6 - Grafikon koji prikazuje speedup paralelne verzije u zavisnosti od veličine i broja thread-ova

Na grafikonu 27.4.6 je prikazan odnos između veličine ulaznog niza i ostvarenog ubrzanja (speedup-a) paralelne verzije Merge Sort algoritma za različit broj tredova (1, 2, 4 i 8). Može se uočiti da kod manjih nizova ($n = 1000$ i 5000) povećanje broja tredova ne dovodi nužno do boljih performansi – zbog dodatnog overhead-a paralelizacije, speedup se čak i smanjuje. Tek kod većih veličina nizova ($n = 10000$ i 20000), upotreba većeg broja tredova počinje da daje stabilnija ubrzanja, iako su ona i dalje ispod teorijskog maksimuma, što ukazuje na ograničenu efikasnost paralelizacije za ovaj algoritam i način implementacije.



27.4.7 - Grafikon koji prikazuje efikasnost paralelizacije u zavisnosti od veličine niza i broja thread-ova

Na ovom grafikonu 27.4.7 prikazana je efikasnost paralelizacije Merge Sort algoritma za različite veličine ulaznog niza i broj tredova. Efikasnost za jedan tred je konstantna i maksimalna, što je očekivano jer nema paralelizacije. Kod višestrukih tredova, efikasnost opada kako se broj tredova povećava, a naročito je niska za 4 i 8 tredova, što ukazuje na lošije iskorišćavanje paralelnih resursa. Efikasnost se blago popravlja kod većih nizova, ali i dalje ostaje ispod očekivane, što ukazuje na prisutne režijske troškove i neoptimalnu skalabilnost algoritma u datoj implementaciji.

6. QUICK SORT ALGORITAM

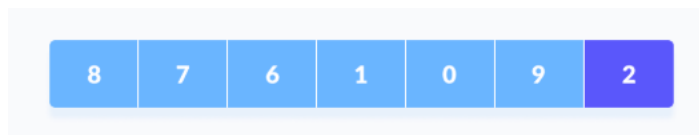
Quick sort je jedan od najbržih i najefikasnijih algoritama za sortiranje, koji se temelji na paradigmi "*Podijeli i osvoji*" (Divide and Conquer). Suština algoritma je u tome da se niz dijeli na manje podnizove pomoću specijalno odabranog elementa koji se naziva pivot. Pivot je ključni element jer određuje granicu između elemenata koji su manji od njega i onih koji su veći. Tokom procesa podjele, pivot se pozicionira tako da su svi elementi manji od pivota smješteni lijevo, a svi elementi veći od pivota desno u odnosu na njegovu poziciju. Nakon toga se isti postupak rekurzivno primjenjuje na lijevu i desnu polovinu niza – svaki od tih podnizova se ponovo dijeli oko svog pivota, sve dok svi podnizovi ne budu svedeni na nizove sa jednim elementom. U tom trenutku se svi elementi nalaze na tačnim pozicijama, i niz se

smatra sortiranom strukturom. Iako se elementi tehnički ne „spajaju“ kao kod Merge Sorta, krajnji rezultat predstavlja potpuno sortirani niz. Quick Sort je poznat po svojoj izuzetnoj brzini u praksi, jer sortira *in-place* (bez dodatnog prostora) i ima prosječnu vremensku složenost $O(n \log n)$, što ga čini idealnim za velike podatke i primjene gdje je memorijska efikasnost važna.

6.1 Implementacija algoritma

Prvi korak

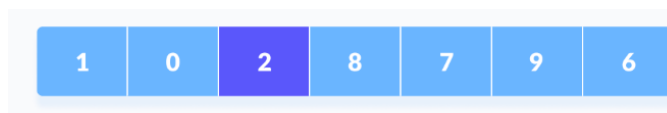
Prvi korak u radu Quick Sort algoritma je odabir pivot elementa, koji predstavlja centralnu tačku oko koje će se niz reorganizovati. U konkretnoj implementaciji prikazanoj na slici 28, kao pivot se bira posljednji (desni) element niza, što je česta i jednostavna praksa. Za dati niz [8, 7, 6, 1, 0, 9, 2], pivot je broj 2. Uloga pivota je ključna jer se svi elementi u nizu porede s njim i niz se reorganizuje tako da svi elementi manji od pivota budu sa lijeve strane, a svi veći ili jednaki sa desne strane. Iako se u ovom koraku pivot još uvijek ne pomjera na konačnu poziciju, on služi kao orijentir za dalji raspored elemenata. Kroz naredne korake algoritma, pivot će biti pravilno postavljen između podnizova manjih i većih elemenata, čime će se podjela niza izvršiti na pravilan način i omogućiti dalji rekurzivni rad algoritma.



28. Odabir pivot elementa

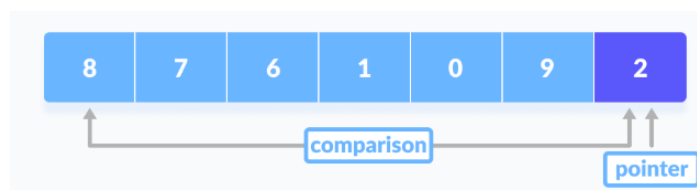
Drugi korak

Nakon što se u Quick sort algoritmu odabere pivot element – što je u ovom slučaju posljednji element niza, broj 2 – prelazi se na ključnu fazu algoritma: reorganizaciju elemenata niza u odnosu na pivot (slika 24).



29. Reorganizacija elemenata u odnosu na pivot

Cilj ove faze je da svi elementi manji od pivota budu premješteni na lijevu stranu, a svi veći ili jednaki na desnu, čime se pivot pravilno pozicionira za dalji rekurzivni rad. Proces započinje tako što se jedan pokazivač (pointer) fiksira na pivot, dok se drugi koristi za poređenje elemenata od početka niza (slika 30).



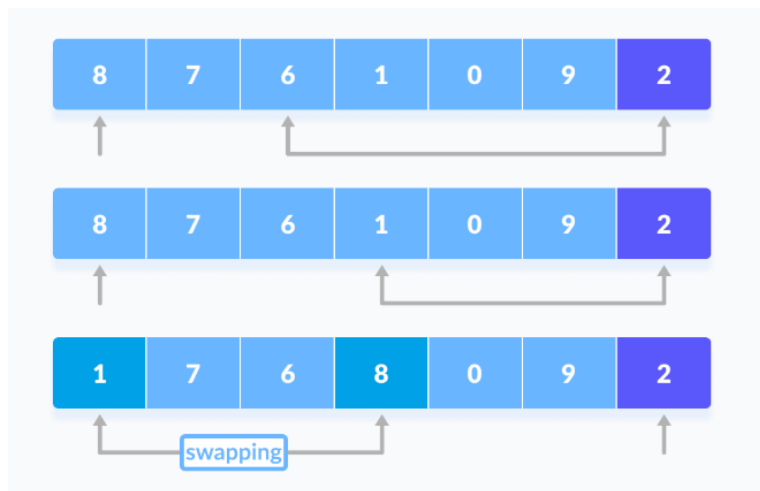
30. Poređenje elemenata sa pivotom

U trenutku kada algoritam naiđe na element veći od pivot, zabilježi se pozicija tog elementa kao potencijalna za zamjenu (slika 31).



31. Označavanje pozicije elementa

Kada se kasnije naiđe na element koji je manji od pivot elementa, vrši se zamjena s prethodno označenim većim, što se može vidjeti na primjeru zamjene brojeva 1 i 8, čime 1 prelazi lijevo, a 8 se pomjera desno (slika 32).



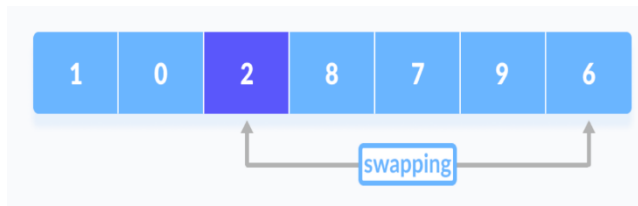
32. Proces poređenja i zamjene elemenata

Ovaj postupak se ponavlja – svaki novi manji element se zamjenjuje sa narednim većim elementom pronađenim desno, kao što je slučaj sa brojem 0, koji zamjenjuje broj 7 (slika 33).



33. Primjer zamjene dva elementa

Proces reorganizacije traje sve dok se ne obradi pretposljednji element niza. Kada je poređenje završeno, pivot element (2) se zamjenjuje sa prvim većim elementom iz desnog segmenta, čime zauzima svoju konačnu poziciju u sredini, između manjih i većih elemenata (slika 34). Na taj način, niz je podijeljen na dva dijela – lijevi dio koji sadrži elemente manje od pivotu i desni dio koji sadrži elemente veće ili jednake – čime je postavljena osnova za naredne rekurzivne pozive koji se izvode na ta dva podniza.



34. Zamjena mjesta pivot elementu

Treći korak – podjela podnizova

Nakon što je pivot pravilno pozicioniran i niz podijeljen na dva dijela, Quick sort algoritam nastavlja rad tako što rekurzivno primjenjuje isti proces na podnizove s lijeve i desne strane pivotu. Ova faza je prikazana na slici 35, gdje se vidi kako se pivot element ponovo bira — prvo za lijevi, a zatim za desni podniz. Svaki od tih podnizova se dalje dijeli sve dok se ne svede na podnizove koji sadrže samo jedan element. Naime, kada podniz sadrži jedan jedini element, on se smatra sortiranim po definiciji, jer više nema elemenata s kojima bi mogao biti u pogrešnom redosledu.



35. Biranje pivot elementa za desni i lijevi podniz

Na slici 36 prikazan je proces rekurzivnog sortiranja lijevog podniza [8, 7, 6, 1, 0, 9, 2]. Nakon prvobitnog particionisanja, niz postaje [1, 0, 2, 8, 7, 9, 6], gdje se zatim lijevi segment [1, 0] dalje razdvaja i sortira sve dok se ne svede na [0, 1]. To su već sortirani elementi koji zauzimaju svoje tačne pozicije. Time je lijevi segment u potpunosti sortiran.



36. Rekurzivno sortiranje lijevog podniza

Na *slici 37* ilustruje se paralelni proces za desni podniz [8, 7, 9, 6], koji nakon više rekurzivnih poziva i pivot rotacija postupno prelazi u [6, 7, 8, 9]. U svakom koraku se bira novi pivot, a preostali elementi se reorganizuju na osnovu njega, sve dok svaki manji podniz ne postane trivijalno sortiran.



37. Rekurzivno sortiranje desnog poniza

Na kraju, kada su i lijevi i desni segment potpuno sortirani kroz niz rekurzivnih podjela i reorganizacija, svi elementi se već nalaze na svojim tačnim pozicijama u nizu. Bitno je naglasiti da Quick sort ne vrši eksplicitno spajanje kao Merge sort, već se rezultat dobija kroz pozicioniranje pivota i rekurzivnu reorganizaciju. Na osnovu prikazanih ilustracija, može se jasno pratiti kako niz [8 7 6 1 0 9 2] postepeno postaje sortirani niz [0 1 2 6 7 8 9], čime je proces sortiranju završen.

Prikazani kod na slici 38 predstavlja implementaciju Quick sort algoritma pomoću dvije funkcije – quickSort i partition. Funkcija quickSort koristi rekurziju kako bi sortirala niz između indeksa leftmostIndex i rightmostIndex. Ako je lijeva granica manja od desne, algoritam poziva funkciju partition koja pronalazi tačnu poziciju pivot elementa, odnosno poziciju na koju on pripada u sortiranom nizu. Nakon toga, Quick Sort se rekurzivno poziva za lijevi i desni dio niza. Sama funkcija partition bira posljednji element kao pivot, i koristi pomoćnu varijablu storeIndex kako bi organizovala sve elemente manje od pivota na lijevu stranu. Tokom prolaska kroz niz, svaki element manji od pivota zamjenjuje se s elementom na trenutnoj storeIndex poziciji, a zatim se pivot postavlja odmah iza posljednjeg manjeg elementa. Funkcija na kraju vraća novu poziciju pivota, što omogućava dalju rekurzivnu podjelu niza. Na ovaj način, niz se efikasno sortira bez potrebe za dodatnim memorijskim prostorom, što čini Quick sort jednim od najbržih i najčešće korišćenih algoritama za sortiranje.

```
quickSort(array, leftmostIndex, rightmostIndex)
  if (leftmostIndex < rightmostIndex)
    pivotIndex <- partition(array, leftmostIndex, rightmostIndex)
    quickSort(array, leftmostIndex, pivotIndex - 1)
    quickSort(array, pivotIndex, rightmostIndex)

partition(array, leftmostIndex, rightmostIndex)
  set rightmostIndex as pivotIndex
  storeIndex <- leftmostIndex - 1
  for i <- leftmostIndex + 1 to rightmostIndex
    if element[i] < pivotElement
      swap element[i] and element[storeIndex]
      storeIndex++
  swap pivotElement and element[storeIndex+1]
  return storeIndex + 1
```

38. Implementacija Quick sort algoritma

6.2 Složenost algoritma

6.2.1 Vremenska složenost (Time Complexity)

Vremenska složenost Quick Sort algoritma zavisi od toga kako se bira pivot element prilikom svakog rekurzivnog poziva. Ona se izražava u tri osnovna slučaja: najbolji, prosječni i najgori.

- **Najbolji slučaj – $O(n \log n)$**

Quick Sort ostvaruje najbolju efikasnost kada se pivot bira tako da ravnomjerno dijeli niz na dva jednako velika podniza. To se dešava kada je pivot tačno srednji element ili blizu sredine niza. U tom slučaju, niz se dijeli na polovinu sa svakim rekurzivnim pozivom, a ukupno se izvrši $\log n$ nivoa rekurzije. Na svakom nivou, potrebno je $O(n)$ operacija za reorganizaciju elemenata, pa je ukupna složenost $O(n \log n)$.

- **Prosječan slučaj – $O(n \log n)$**

Kada se pivot bira nasumično ili kada ne dijeli niz ravnomjerno, ali ni ekstremno loše, govorimo o prosječnom slučaju. Statistički gledano, ako se pivot rasporedi “razumno ravnomjerno” kroz više poziva, Quick sort i dalje pravi logaritamski broj nivoa i vrši linearan broj operacija po nivou. Zato i u prosjeku Quick Sort ima složenost $O(n \log n)$, što ga čini vrlo efikasnim za većinu ulaza.

- **Najgori slučaj – $O(n^2)$**

Najgori slučaj nastaje kada se pivot stalno bira kao najmanji ili najveći element, što dovodi do toga da se jedan podniz sastoji od $n - 1$ elemenata, a drugi bude prazan. U tom slučaju se rekurzija izvršava n puta, a svaki nivo vrši n operacija, što rezultira kvadratnom složenošću $O(n^2)$. Ovo se može dogoditi, na primjer, ako je ulazni niz već sortiran, a pivot se bira uvijek kao krajnji element.

6.2.2 Prostorna složenost

Quick sort je poznat po tome što je in-place algoritam, što znači da ne koristi dodatni prostor za nizove kao što to čini Merge sort. Međutim, koristi se memorija na pozivnom steku (call stack) zbog rekurzivnih poziva.

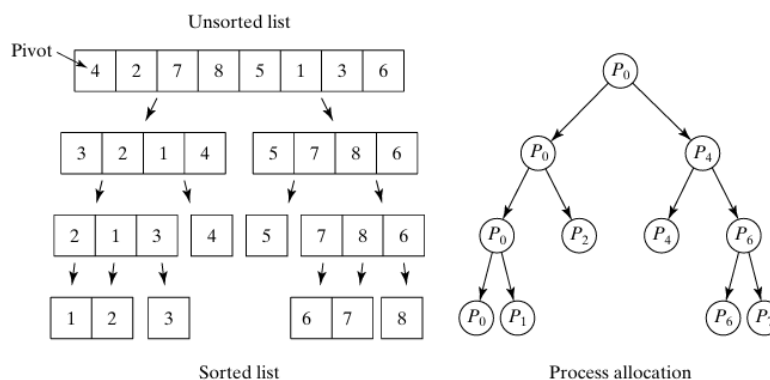
U najboljem i prosječnom slučaju, stek se koristi za oko $\log n$ nivoa rekurzije, pa je prostorna složenost $O(\log n)$.

U najgorem slučaju, ako se ne koristi optimizacija (npr. tail recursion ili randomizacija pivota), broj nivoa može biti i do n , što bi značilo složenost $O(n)$.

U praksi, ovaj algoritam je memorijski veoma efikasan jer koristi minimalan dodatni prostor osim privremenih zamjena i pokazivača.¹⁰

6.3 Paralelizacija algoritma

Quick sort je prirodno rekurzivan algoritam, što znači da se njegovo izvođenje lako može predstaviti kao binarno stablo – pri svakom rekurzivnom pozivu lista se dijeli u dvije podliste, lijevu i desnu, na osnovu izabranog pivota. Upravo ova struktura dijeljenja pruža potencijal za paralelizaciju algoritma.



Slika 381 - Quick sort - stablo alociranja procesa

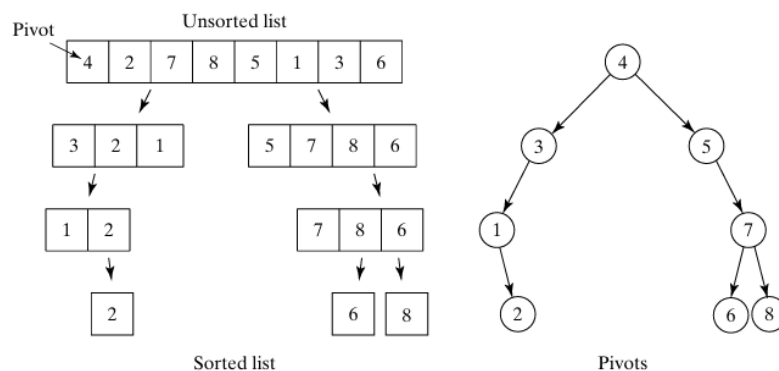
Jedan od očiglednih načina da se Quick sort algoritam paralelizuje jeste da jedan procesor započne sortiranje, a zatim jedan od rekurzivnih poziva – najčešće desnu polovinu – delegira drugom procesoru, dok prvi procesor nastavlja sa lijevom polovinom. Ovaj pristup dovodi do formiranja stabla poziva, vrlo sličnog strukturi koju nalazimo kod Merge sort algoritma. Na slici 38.1 jasno je prikazan tok sortiranja kroz grananje, gdje se pivot element nosi uz lijevu granu sve dok ne bude konačno smješten. Ključna osobina pivot elementa jeste da, jednom kada bude pozicioniran između dvije podliste, njegova pozicija postaje konačna – više se ne uzima u obzir u narednim fazama sortiranja. U tom smislu, pivot se može smatrati "završenim poslom".

Zbog toga se u alternativnom prikazu, kao što je prikazano na slici 38.2, stablo može redizajnirati tako da pivot izgleda kao "zadržan", i da se konačno sortiranje

¹⁰ "Quick sort," Programiz, pristupljeno 01. april 2025, dostupno na: <https://www.programiz.com/dsa/merge-sort>

posmatra kao in-order obilazak tog stabla: prvo lijeva podlista, zatim pivot, pa desna podlista.

Ipak, uprkos potencijalu za visoku paralelizaciju kroz rekurzivne grane, osnovni problem ostaje u tome što inicijalnu podjelu mora izvršiti samo jedan procesor. Bez obzira na broj dostupnih jezgara, pivot mora biti izabran i niz podijeljen prije nego što ostali procesori mogu preuzeti zadatke. Ovo stvara usko grlo (bottleneck) u performansama, jer početna sekvencijalnost ograničava ukupnu brzinu algoritma. Čak i ako je pivot idealno izabran i dijeli niz na dvije podjednake polovine, paralelizacija može početi tek nakon te prve, nezaobilazno sekvencijalne operacije. Zbog toga se Quick sort, iako efikasan, suočava s inherentnim ograničenjima kada je riječ o potpunom iskorišćavanju paralelizacije.



Slika 38.2 - Quick sort – alociranje pivot elementa

Kada je riječ o vremenskoj složenosti, paralelizovani Quick sort zadržava istu teorijsku vremensku složenost kao i sekvencijalna verzija – u prosjeku $O(n \log n)$, dok u najgorem slučaju (kada je pivot loše odabran i uvijek dijeli niz na jako nejednake dijelove) može dostići $O(n^2)$. Međutim, uz dobar izbor pivota (npr. median-of-three pristup), taj najgori slučaj se značajno rijetko dešava.

U paralelnoj varijanti, koristi se paralelizacija rekurzivnih poziva, pa se uz p procesora ukupno vrijeme izvršavanja smanjuje, te se može postići teoretski speedup blizu $O(\log n)$, uz ukupnu količinu posla ostajući $O(n \log n)$. Ipak, zbog sekvencijalne prirode prve podjele (kao što je već objašnjeno), kao i zbog troškova upravljanja nitima (synchronization, memory contention), ovaj speedup nije linearan i ima svoj praktični maksimum.

Među prednostima paralelnog Quick sorta ističu se njegova efikasnost kod velikih datasetova, gdje višestruki procesori ili jezgra mogu istovremeno sortirati različite dijelove niza, što značajno ubrzava ukupan proces. Takođe, zbog toga što koristi lokalnu memoriju i cache-friendly pristup, često nadmašuje Merge sort u realnim aplikacijama, posebno kada se koristi sa modernim višedretnim okruženjima i hardverom. Osim toga, jednostavno se implementira korišćenjem rekurzivnog fork-join modela.

Sa druge strane, mane uključuju kompleksniju kontrolu nad paralelizacijom – potrebno je pažljivo upravljati brojem niti kako se ne bi stvorio preveliki overhead. Prevelik broj paralelnih poziva može dovesti do zagušenja sistema, dok premali broj neće iskoristiti puni potencijal paralelizacije. Takođe, izbor lošeg pivota vodi do nebalansiranih grana, što umanjuje efikasnost raspodjele posla među procesorima. Još jedan izazov je što dijeljenje podataka među nitima može uzrokovati probleme

sa cache-koherencijom i pristupom memoriji, posebno u sistemima sa Non-Uniform Memory Access (NUMA) arhitekturom.¹¹

6.4 Implementacija u okviru projekta

Sekvencijalna verzija Quick sort algoritma, prikazana na slici 38.3 zasniva se na rekurzivnom principu „podijeli i osvoji“. Algoritam se sastoji od dvije ključne funkcije: `partition` i `quickSort`.

Funkcija `partition` služi za izbor pivota i reorganizaciju elemenata oko njega. Pivot se uzima kao poslednji element niza (`arr[high]`). Pokazivač `i` se koristi za označavanje kraja segmenta sa elementima manjim od pivota. Petlja prolazi kroz sve elemente od `low` do `high - 1`, i svaki put kada nađe element manji od pivota, uvećava `i` i vrši zamjenu tog elementa sa elementom na poziciji `i`. Nakon završetka petlje, pivot se smješta na tačno mjesto (odnosno između manjih i većih elemenata), čime se niz dijeli na dva dijela.

Funkcija `quickSort` je rekurzivna. Nakon što `partition` vrati indeks pivota, `quickSort` se poziva za lijevu i desnu podlistu oko tog pivota. Ova rekurzija se nastavlja dok `low` ne bude veći ili jednak `high`, što predstavlja bazni slučaj (niz sa 0 ili 1 element). Sekvencijalna implementacija je jednostavna, koristi samo osnovne strukture jezika C++ i dobro ilustruje osnovne koncepte sortiranja bez dodatnih biblioteka.

```
int partition(int arr[], int low, int high) {
    int pivot = arr[high]; // pivot je poslednji element
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1); // lijeva polovina
        quickSort(arr, pi + 1, high); // desna polovina
    }
}
```

Slika 38.3- sekvencijalna implementacija Quick sort alg. U C++

Paralelna verzija Quick Sort algoritma – slika 38.4 koristi OpenMP direktive kako bi se omogućilo istovremeno izvršavanje rekurzivnih poziva na više jezgara. Struktura algoritma ostaje identična sekvencijalnoj verziji, ali se koristi `#pragma omp parallel sections` da se dva rekurzivna poziva – za lijevu i desnu podlistu – mogu izvršavati paralelno.

Funkcija `quickSortParallel` prima dodatni parametar `depth`, koji označava dubinu rekurzije. Ograničavanjem dubine (npr. maksimalno 4 nivoa) sprečava se kreiranje

¹¹Barry Wilkinson i Michael Allen, *Parallel Programming: Techniques and Applications Using Networked Workstations and Parallel Computers*, Second Edition, Prentice Hall, 2005, str. 334–336.

prevelikog broja niti, što može dovesti do degradacije performansi. Ako je trenutna dubina manja od zadate granice, koristi se OpenMP direktiva za pokretanje sekcija, a svaka sekcija poziva sortiranje jedne od podlista. Kada je granica dosegnuta, rekurzija se nastavlja sekvencijalno. Na ovaj način postiže se balans između paralelizacije i kontrole nad brojem pokrenutih zadataka.

Funkcija `partition` je ista kao u sekvencijalnoj verziji jer je operacija lokalna i ne zahvata dijeljenje memorije među nitima. Cijeli niz se sortira in-place, bez potrebe za dodatnom memorijom, što ovu verziju čini efikasnom i skalabilnom za rad na modernim višeprosesorskim sistemima. Kada se koristi adekvatan broj niti (npr. 4 ili 8), paralelna verzija može ostvariti značajno smanjenje vremena izvršavanja u poređenju sa sekvencijalnom.

```
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSortParallel(int arr[], int low, int high, int depth = 0) {
    if (low < high) {
        int pi = partition(arr, low, high);

        // Ograniči paralelizaciju da se ne stvori previše niti
        if (depth < 4) {
            #pragma omp parallel sections
            {
                #pragma omp section
                quickSortParallel(arr, low, pi - 1, depth + 1);

                #pragma omp section
                quickSortParallel(arr, pi + 1, high, depth + 1);
            }
        } else {
            quickSortParallel(arr, low, pi - 1, depth + 1);
            quickSortParallel(arr, pi + 1, high, depth + 1);
        }
    }
}
```

Slika 38.4 - paralelna implementacija Quick sort alg. U C++

6.4.1 Analiza rezultata

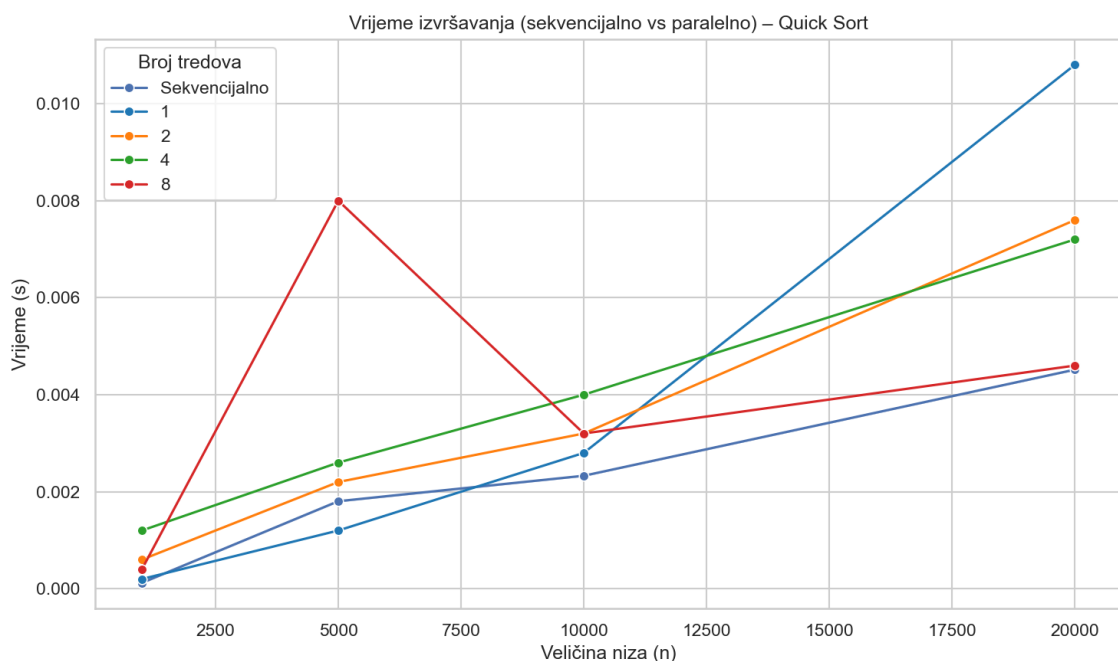
Za potrebe eksperimentalnog poređenja sekvencijalne i paralelne verzije Quick sort algoritma, izvršena su kontrolisana testiranja u okviru unaprijed definisanih uslova. Algoritmi su testirani nad nasumično generisanim nizovima različitih veličina kako bi se sagledalo njihovo ponašanje u zavisnosti od kompleksnosti ulaznih podataka. Konkretno, korišćene su veličine nizova od 1000, 5000, 10000 i 20000 elemenata. Za svaki niz, algoritmi su pokrenuti više puta, a rezultat predstavlja prosječno vrijeme izvršavanja. Paralelna verzija je testirana sa različitim brojevima tredova: 1, 2, 4 i 8, čime je omogućeno poređenje uticaja stepena paralelizacije na performanse. Mjerenje vremena je realizovano pomoću odgovarajućih mehanizama za precizno

profilisanje: `std::chrono` za sekvencijalnu verziju i `omp_get_wtime()` za paralelnu. Eksperimenti su dizajnirani tako da simuliraju realne uslove izvođenja i omoguće procjenu brzine, efikasnosti i skalabilnosti algoritma. Tablerani prikaz rezultata dat je slikom 38.5.

n	threads	seq_time	par_time	speedup	efficiency	cost
1000.0	1.0	0.000121	0.0002	0.603844	0.603844	0.0002
1000.0	2.0	0.000121	0.0006	0.201265	0.100633	0.0012
1000.0	4.0	0.000121	0.0012	0.100633	0.025158	0.0048
1000.0	8.0	0.000121	0.0004	0.301922	0.03774	0.0032
5000.0	1.0	0.001804	0.0012	1.503316	1.0	0.0012
5000.0	2.0	0.001804	0.0022	0.81997	0.409985	0.0044
5000.0	4.0	0.001804	0.0026	0.693815	0.173454	0.0104
5000.0	8.0	0.001804	0.008	0.225492	0.028186	0.064
10000.0	1.0	0.002329	0.0028	0.831625	0.831625	0.0028
10000.0	2.0	0.002329	0.0032	0.727667	0.363834	0.0064
10000.0	4.0	0.002329	0.004	0.582129	0.145532	0.016
10000.0	8.0	0.002329	0.0032	0.727667	0.090958	0.0256
20000.0	1.0	0.004516	0.0108	0.418154	0.418154	0.0108
20000.0	2.0	0.004516	0.0076	0.594219	0.29711	0.0152
20000.0	4.0	0.004516	0.0072	0.627233	0.156808	0.0288
20000.0	8.0	0.004516	0.0046	0.981767	0.122721	0.0368

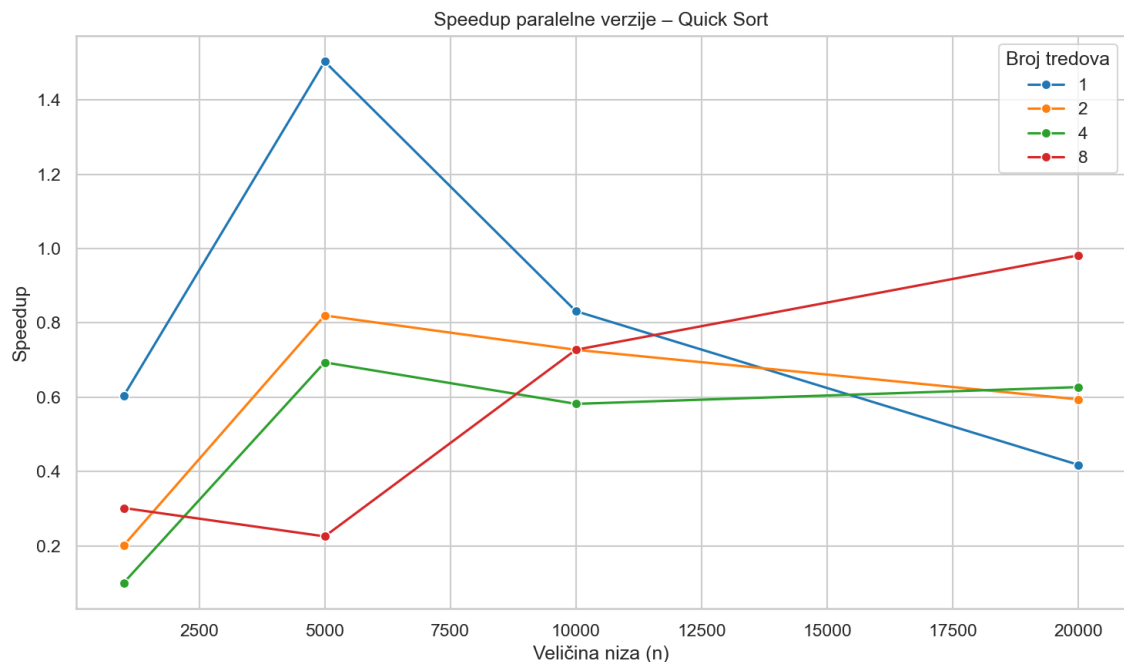
38.5 - Tabelarni prikaz rezultata performansi

Za vizuelnu analizu performansi koristi se Python skripta `QuickSortGraphs.py`, koja na osnovu CSV fajla (`QuickSortResults.csv`) generiše tri grafikona koristeći biblioteke `pandas`, `matplotlib` i `seaborn`. Prvi grafikon prikazuje vrijeme izvršavanja sekvencijalne i paralelne verzije algoritma u zavisnosti od veličine niza. Drugi prikazuje speedup, odnosno ubrzanje paralelne verzije u odnosu na sekvencijalnu. Treći grafikon prikazuje efikasnost paralelizacije, tj. koliko dobro se koriste dostupni tredovi. Ovi grafikoni omogućavaju jednostavno i jasno poređenje performansi različitih konfiguracija algoritma.



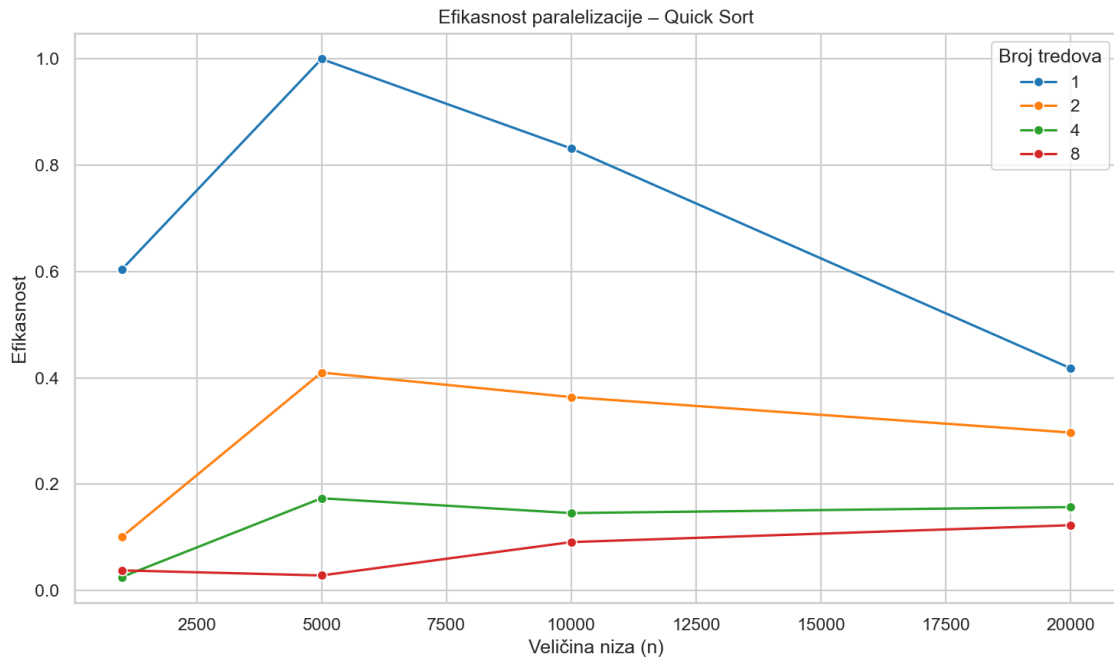
38.6 - Grafikon koji prikazuje vrijeme izvršavanja u zavisnosti od veličine niza

Na grafikonu 38.6 prikazano je vrijeme izvršavanja sekvencijalne i paralelne verzije Quick Sort algoritma za različite veličine ulaznog niza. Sekvencijalna verzija pokazuje stabilan rast vremena sa povećanjem veličine niza, dok paralelne verzije ostvaruju različite performanse u zavisnosti od broja tredova. Kod manjih nizova, paralelizacija često ne donosi poboljšanje, a može čak i pogoršati performanse zbog dodatnog overhead-a. Tek kod većih nizova (npr. 10000 i 20000) paralelne verzije sa više tredova počinju da ostvaruju bolje vrijeme u odnosu na sekvencijalnu.



38.7 - Grafikon koji prikazuje speedup paralelne verzije u zavisnosti od veličine i br. niti

Grafikon 38.7 prikazuje speedup – odnosno koliko puta je paralelna verzija brža od sekvencijalne – u zavisnosti od veličine ulaznog niza i broja tredova. Vidimo da speedup značajno varira i da često ostaje ispod očekivanih vrijednosti. Najveći speedup se postiže kod većih nizova i većeg broja tredova, ali ni tada nije linearan. Kod malih nizova, upotreba više tredova često daje slabiji speedup ili čak negativan efekat zbog neefikasne raspodjele zadataka i sinhronizacije.



38.8 - Grafikon koji prikazuje efikasnost paralelizacije u zavisnosti od veličine niza i br. niti

Na grafikonu 38.8 vizuelno je prikazana efikasnost, koja pokazuje koliko je stvarno iskorišćen potencijal paralelizacije u odnosu na broj tredova. Idealna efikasnost bi bila blizu 1 (100%), ali rezultati pokazuju da efikasnost opada sa povećanjem broja tredova. Posebno je niska za 4 i 8 tredova na manjim nizovima, što ukazuje na značajan overhead i nesrazmjernu raspodjelu posla. Efikasnost se blago popravlja kod većih nizova, ali ostaje daleko ispod optimalne, što ukazuje na izazove paralelizacije Quick Sort algoritma u realnim uslovima.

Rezultati pokazuju da paralelna verzija Quick Sort algoritma može donijeti ubrzanje u odnosu na sekvencijalnu, ali prvenstveno kod većih ulaznih nizova. Kod manjih nizova, paralelizacija često donosi zanemarljiv ili čak negativan efekat zbog režijskih troškova. Iako skalabilnost postoji, efikasnost opada sa većim brojem tredova, što ukazuje na potrebu za pažljivim podešavanjem dubine paralelizacije i optimizacijom algoritma.

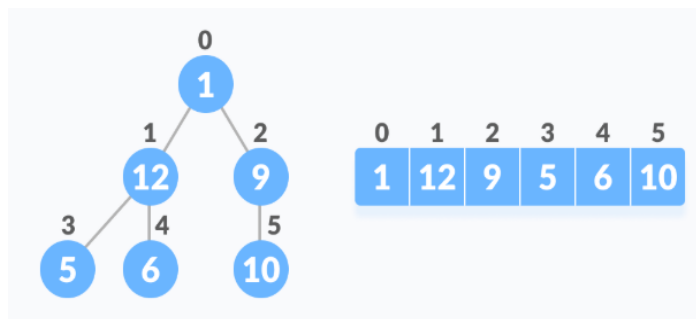
7. HEAP SORT ALGORITAM

Heap sort je efikasan i deterministički algoritam za sortiranje koji koristi strukturu podataka poznatu kao heap (hrpa), tačnije maks-heap – potpuno binarno stablo u kojem je svaki roditeljski čvor veći ili jednak od svojih potomaka. Ključna prednost ovog algoritma leži u tome što se heap može efikasno predstaviti pomoću običnog niza, bez potrebe za korišćenjem dodatnih pokazivača ili složenih struktura. Ova reprezentacija se oslanja na specifičan matematički odnos između indeksa u nizu i pozicija čvorova u binarnom stablu.

Naime, kompletno binarno stablo ima osobinu da za svaki element sa indeksom i u nizu: lijevo dijete se nalazi na indeksu $2i + 1$, desno dijete se nalazi na indeksu $2i + 2$, a roditelj bilo kog elementa nalazi se na indeksu $(i - 1) / 2$ (u oznaci cio dio pod).

Ovaj odnos omogućava izuzetno brzu i jednostavnu navigaciju kroz strukturu stabla korišćenjem samo aritmetike nad indeksima. Zahvaljujući tome, Heap sort se može implementirati unutar jednog jedinog niza (tzv. in-place), bez potrebe za dodatnim memorijskim strukturama.

U osnovi, Heap sort funkcioniše tako što prvo pretvara ulazni niz u maks-heap – strukturu u kojoj je najveći element uvijek na vrhu (korijenu) – a zatim postepeno uklanjanje najveći element, premješta ga na kraj niza i ponovo prilagođava preostali niz u validan heap. Ovaj proces se ponavlja sve dok svi elementi ne budu sortirani u uzlaznom redosledu. Zahvaljujući stabilnoj vremenskoj složenosti i malim prostornim zahtjevima, Heap sort je naročito pogodan za rad sa velikim količinama podataka u sistemima ograničenih resursa.



39. Ilustracija veze između niza i stabla

Na prikazanoj slici broj 39 jasno se vidi kako se elementi binarnog stabla mogu predstaviti pomoću običnog niza, prateći pravila indeksiranja karakteristična za heap strukturu. Svaki čvor u stablu ima određenu poziciju u nizu, a odnosi između roditelja i djece se računaju jednostavnom formulom: za čvor na indeksu i , njegovo lijevo dijete se nalazi na indeksu $2i + 1$, a desno dijete na indeksu $2i + 2$. Na primjeru, element 1 na indeksu 0 ima lijevo dijete 12 (indeks 1) i desno dijete 9 (indeks 2). Zatim, 12 na indeksu 1 ima lijevo dijete 5 (indeks 3) i desno dijete 6 (indeks 4). Ovakva organizacija omogućava jednostavno i efikasno upravljanje binarnim stablom unutar jednog niza, što je ključno za rad Heap sort algoritma.

7.1 Implementacija algoritma

Prvi korak – izgradnja maks heap-a

Prva faza u implementaciji Heap sort algoritma jeste izgradnja maks-heapa iz proizvoljnog niza. Maks-heap je binarno stablo u kojem je svaki roditeljski čvor veći od svojih potomaka, a kako bi se postigla ta struktura, koristi se funkcija heapify (slika 39.1), koja postepeno popravljala lokalna kršenja ovog pravila.

```

void heapify(int arr[], int n, int i) {
    // Find largest among root, left child and right child
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    // Swap and continue heapifying if root is not largest
    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}

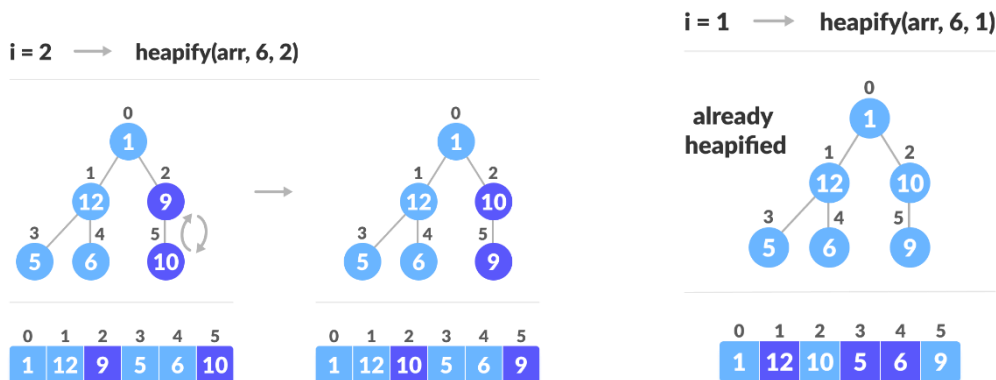
```

39.1. Implementacija heapify funkcije

Funkcija *heapify* predstavlja ključnu komponentu Heap sort algoritma i služi za održavanje maks-heap svojstva. Ona upoređuje vrijednosti roditelja i njegove djece, i ukoliko roditelj nije najveći, vrši zamjenu sa najvećim djetetom. Zatim se rekurzivno poziva za novo pomjerenu poziciju, sve dok se ne uspostavi validna heap struktura. Ova funkcija osigurava da lokalno podstablo bude pravilno organizovano kao maks-heap.

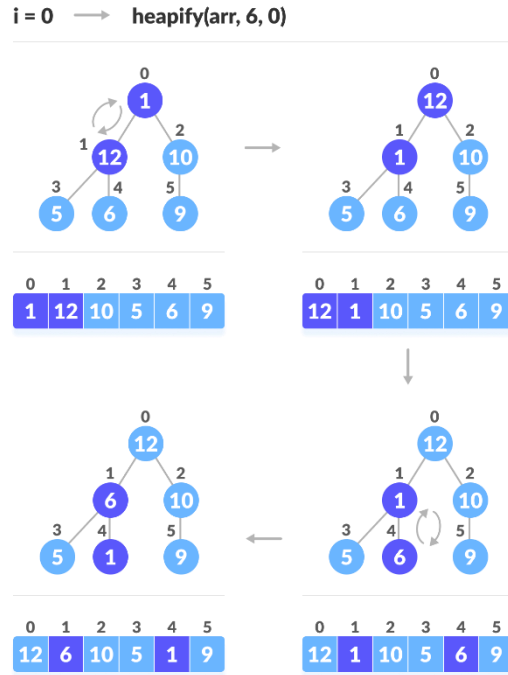
Proces izgradnje počinje od poslednjeg roditeljskog čvora, a ne od korijena. Za niz dužine n , prvi čvor koji nije list nalazi se na poziciji $n/2 - 1$. Svi čvorovi nakon toga su listovi i samim tim već zadovoljavaju uslov heap-a. Dakle, *heapify* se primjenjuje od indeksa $n/2 - 1$ do 0, pri čemu se koriguju lokalne greške od dna ka vrhu stabla – strategija poznata kao bottom-up pristup.

Na slici 40 (*heapify* na indeksu 2), čvor 9 ima desnog potomka 10, koji je veći, pa dolazi do zamjene i 10 zauzima roditeljsku poziciju. Rezultat je lokalno validan heap. Na slici broj 41 (*heapify* na indeksu 1), čvor 12 već zadovoljava uslov da je veći od svojih potomaka (5 i 6), pa se ništa ne mijenja.



Slike 40 i 41 – primjeri rada heapify funkcije

Na trećem koraku – slika 42 (*heapify* na indeksu 0), vršni čvor 1 krši pravilo jer je manji od svog lijevog djeteta 12, pa se vrši zamjena. Nakon toga se nastavlja *heapify* rekurzivno i dalje prema dolje, sve dok svi čvorovi ne budu u skladu s pravilom maks-heapa. U završnom stanju, niz [1, 12, 10, 5, 6, 9] postaje [12, 6, 10, 5, 1, 9], gdje je 12 sada na vrhu, a svi ostali elementi organizovani tako da zadovoljavaju maks-heap strukturu.



42. Primjer heapify funkcije i rekurzije

Ova faza je od presudnog značaja jer postavlja osnovu za ostatak Heap sort algoritma: tek nakon formiranja validnog maks-heapa može započeti proces uklanjanja najvećih elemenata i njihovog postavljanja na kraj niza, što vodi ka potpunom sortiranju.

Drugi korak – rad Heap sort algoritma

Nakon što se niz transformiše u maks-heap, svaki naredni korak u Heap sort algoritmu uključuje izvlačenje najvećeg elementa iz heap-a, njegovo premještanje na kraj niza, i obnavljanje strukture heap-a pomoću funkcije heapify. Pošto heap zadovoljava Max-heap svojstvo, najveći element se uvijek nalazi u korijenu stabla, tj. na indeksu 0.

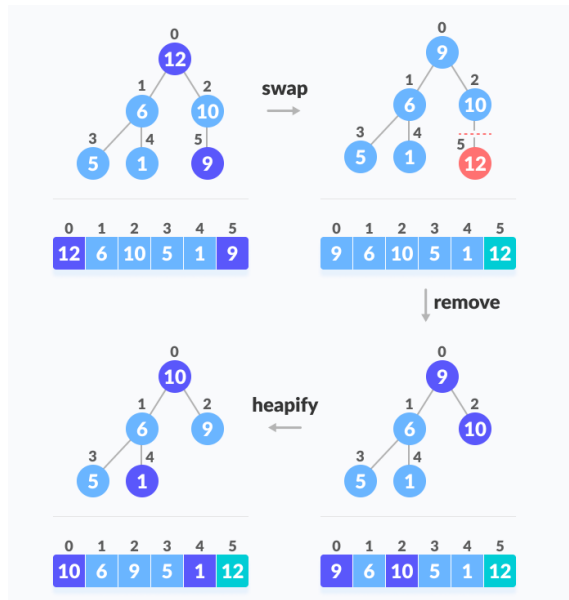
Proces se sastoji iz tri ključna koraka koji se neprestano ponavljaju:

1. Swap (zamjena): Najveći element (na vrhu) se zamjenjuje sa posljednjim elementom u nizu. Na taj način najveći element dolazi na svoje konačno mjesto u sortiranom nizu, na samom kraju.
2. Remove (uklanjanje): Nakon zamjene, veličina heap-a se smanjuje za 1, jer je najveći element već sortiran i više nije dio heap-a.
3. Heapify: Korijenski element (koji je zamijenjen manjim) možda više ne zadovoljava heap svojstvo, pa se poziva funkcija heapify da se obnovi Max-heap struktura u preostalom dijelu niza.

Ova tri koraka se ponavljaju sve dok svi elementi ne budu sortirani – tj. dok heap ne postane prazan.

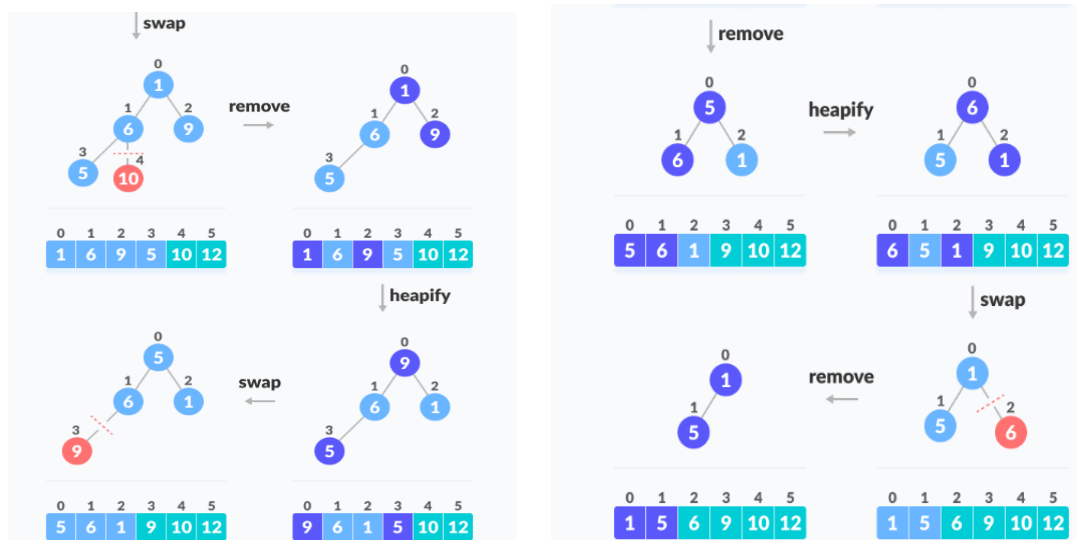
Slijedi detaljno objašnjenje ovih koraka, uz ilustracije.

Na prikazanoj slici 43 detaljno je predstavljen drugi dio rada Heap sort algoritma, tj. faza sortiranja nakon što je niz već transformisan u maks-heap. Na početku slike vidimo kompletan maks-heap, gdje je vrijednost 12 na vrhu, kao najveći element u stablu. U prvom koraku, korijenski element 12 se zamjenjuje sa posljednjim elementom 9 u nizu, nakon čega se 12 izdvaja kao sortirani i postavlja na kraj. Sada se heapify poziva za novi korijen (9), i budući da 10 kao desno dijete ima veću vrijednost, dolazi do zamjene: 10 dolazi na vrh, čime se ponovo uspostavlja maks-heap struktura.



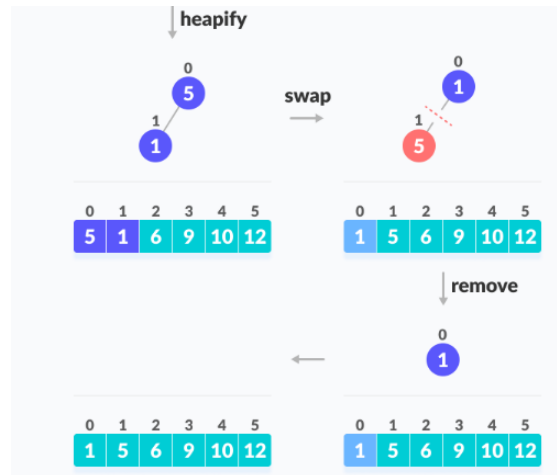
43. Primjer rada algoritma

U sljedećem koraku – slika 44, 10 se uklanja (tj. zamjenjuje sa 1), koji sada postaje novi korijen. Ponovo se poziva heapify: 1 je manji od oba potomka, pa ga prvo zamjenjuje 6, zatim ga 5 dodatno potiskuje niže u stablu, dok se struktura ponovo ne stabilizuje. Taj postupak se ponavlja rekurzivno – svaki put se korijen (najveći preostali element) premješta na kraj aktivnog dijela niza, zamjenjuje se sa posljednjim i heapify funkcija obnavlja lokalni poredak. (slika 45)



44. i 45.– nastavci primjera

Kako se heap smanjuje, tako elementi 9, 6, 5 i na kraju 1 dolaze na svoje odgovarajuće pozicije. Na svakom nivou se vizuelno prikazuje promjena u stablu i odgovarajuća izmjena u nizu ispod njega. U završnim koracima (slika 46) jasno se vidi kako se poslednji nesortirani elementi rotiraju i postavljaju u tačan redosljed – što na kraju rezultira potpuno sortiranim nizom [1, 5, 6, 9, 10, 12].



46. Kraj primjera

Date slike na jasan način prikazuju mehaniku sortiranja u Heap sort algoritmu: sve se vrte oko uklanjanja korijena i pozivanja heapify nad preostalim elementima. Istovremeno, slike vizualizuju kako se binarno stablo "ruši" odozgo dok se niz gradi od pozadi – što je suština rada ovog algoritma.

```
// Heap sort
for (int i = n - 1; i >= 0; i--) {
    swap(&arr[0], &arr[i]);

    // Heapify root element to get highest element at root again
    heapify(arr, i, 0);
}
```

47. Implementacija Heap sort algoritma

Prikazani kod na slici 47 prikazuje završnu fazu Heap sort algoritma – samu operaciju sortiranja. U svakoj iteraciji petlje, najveći element koji se nalazi na početku niza (indeks 0) zamjenjuje se sa posljednjim nesortiranim elementom (indeks i). Nakon zamjene, heapify funkcija se poziva za korijen kako bi se ponovo uspostavilo maks-heap svojstvo u preostalom dijelu niza. Petlja se izvršava dok svi elementi ne budu pravilno sortirani, od najmanjeg (na početku) do najvećeg (na kraju niza).

7.2 Složenost algoritma

7.2.1 Vremenska složenost (Time Complexity)

Heap sort algoritam ima vremensku složenost $O(n \log n)$ u svim slučajevima — najboljem, prosječnom i najgorem. Tokom faze izgradnje maks-heapa, algoritam primjenjuje heapify funkciju na sve unutrašnje čvorove (njih je oko $n/2$), a svaki od njih može zahtijevati najviše $\log n$ koraka da bi se uspostavilo heap svojstvo. To ovu fazu čini složenosti $O(n \log n)$. U fazi sortiranja, najveći element (iz korijena heap-a) se premješta na kraj niza, a ostatak se ponovo heapifikuje. Budući da se heapify poziva n puta, svaka s maksimalno $\log n$ operacija, složenost i ove faze je $O(n \log n)$. Ukupna vremenska složenost ostaje $O(n \log n)$, jer se faze izvode sekvencijalno, a ne ugniježđeno.

7.2.2 Prostorna složenost

Heap sort je algoritam koji se izvršava in-place, što znači da za svoje djelovanje ne zahtijeva dodatne strukture podataka, već sve promjene vrši unutar samog ulaznog niza. Zbog toga ima prostornu složenost $O(1)$ – koristi samo konstantnu količinu dodatne memorije bez obzira na veličinu ulaznih podataka. Ovo ga čini pogodnim za sisteme sa ograničenim memorijskim resursima. Međutim, Heap sort nije stabilan algoritam, što znači da može promijeniti redosljed elemenata koji imaju istu vrijednost tokom sortiranja.¹²

7.3 Paralelizacija algoritma

Paralelizacija Heap sort algoritma je tehnički moguća, ali je znatno manje pogodna i efikasna u odnosu na algoritme poput Merge sort-a ili Quick sort-a, zbog njegove specifične strukture i zavisnosti između operacija. Heap sort funkcioniše tako što se od ulaznog niza prvo izgradi maksimalna hrpa (max-heap), a zatim se elementi postepeno uklanjaju sa vrha hrpe i smještaju na kraj niza. Sam proces izgradnje hrpe ima složenost $O(n)$, dok operacije uklanjanja i održavanja svojstva gomile (heapify) imaju složenost $O(\log n)$ po elementu, što ukupno daje ukupnu složenost $O(n \log n)$.

Kada je riječ o paralelizaciji, najveća prepreka leži u zavisnostima među čvorovima u binarnom stablu hrpe – svaka heapify operacija zavisi od rezultata svojih potomaka, što znači da se ne mogu izvršavati potpuno nezavisno.

Ipak, određeni stepen paralelizacije moguć je u fazi izgradnje hrpe, i to posebno kod heapify poziva na nižim nivoima stabla (npr. listovi i njihovi neposredni roditelji), jer su ti pozivi međusobno nezavisni. Ovakav pristup poznat je kao bottom-up parallel heap construction, i omogućava da se izgradnja gomile u optimalnim uslovima smanji na vremensku složenost $O(\log n)$ uz korišćenje više procesorskih jezgara.

Međutim, druga faza Heap sort-a – vađenje maksimuma i reorganizacija hrpe – ostaje strogo sekvencijalna, jer svaka naredna operacija zavisi od prethodno ažurirane

¹² "Heap sort," *Programiz*, pristupljeno 01. april 2025, dostupno na: <https://www.programiz.com/dsa/merge-sort>

strukture hrpe. Zbog ovih faktora, ukupna vremenska složenost paralelizovanog Heap sort-a ostaje $O(n \log n)$, kao i kod sekvencijalne verzije, dok su stvarne performanse ograničene brojem paralelno izvršivih koraka.

Upravo zbog toga, Heap sort se u praksi rijetko koristi u paralelnim okruženjima, jer overhead sinhronizacije i zavisnosti često nadmašuju dobitke koje donosi paralelizacija. Iako je moguće paralelizovati određene dijelove Heap sort-a, njegova rekurzivna i zavisna struktura čini ga neefikasnim izborom za paralelnu obradu velikih skupova podataka, u poređenju sa algoritmima koji imaju bolju skalabilnost u paralelnim sistemima.¹³

7.4 Implementacija u okviru projekta

Na slici 47.1 je prikazana sekvencijalna implementacija Heap Sort algoritma koja se temelji na ideji binarnog max-heap stabla, koje omogućava efikasno selektovanje najvećeg elementa u svakom koraku. Ulazni niz se najprije transformiše u max-heap pomoću funkcije `heapify`, koja provjerava lokalnu heap osobinu (roditelj veći od djece) i po potrebi reorganizuje elemente rekurzivno. Ova faza, poznata kao "izgradnja heap-a" (`buildHeap`), pokreće se od sredine niza ka početku jer listovi već zadovoljavaju heap osobinu. Nakon toga slijedi faza sortiranja u kojoj se najveći element (koji se nalazi na vrhu heap-a) zamjenjuje sa posljednjim u nizu, a zatim se `heapify` ponovo poziva na smanjenom dijelu niza. Ovaj proces se ponavlja sve dok se svi elementi ne prebace na svoje odgovarajuće pozicije, a konačan rezultat je rastuće sortiran niz.

```
void heapify(int* arr, int n, int i) {
    int largest = i;
    int l = 2 * i + 1;
    int r = 2 * i + 2;

    if (l < n && arr[l] > arr[largest])
        largest = l;
    if (r < n && arr[r] > arr[largest])
        largest = r;

    if (largest != i) {
        std::swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(int* arr, int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (int i = n - 1; i >= 0; i--) {
        std::swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}
```

47.1 – sekvencijalna implementacija Heap sort alg. u C++

Paralelna implementacija – slika 47.2 pokušava da ubrza `buildHeap` fazu, jer su mnogi pozivi funkcije `heapify` u tom dijelu nezavisni — npr. čvorovi u donjoj polovini stabla se mogu istovremeno obrađivati. Ova paralelizacija se postiže

¹³ T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, str. 151–163; K. Batcher, "Design of a Parallel, Hardware Priority Queue," *IEEE Transactions on Computers*, vol. C-29, no. 10, 1980, str. 833–840.

korišćenjem OpenMP direktive `#pragma omp parallel for`, čime se različiti čvorovi povjeravaju odvojenim tredovima. Ipak, sama priroda algoritma nameće ograničenja: sortiranje (druga faza) ne može biti paralelizovano jer svaka iteracija zavisi od prethodne. Nakon što se najveći element premjesti na kraj niza, struktura heap-a se mora ažurirati prije nego što se pređe na sljedeći korak — dakle, postoji stroga sekvencijalna zavisnost među koracima. Osim toga, i u `buildHeap` fazi postoji rizik od “false sharing”-a i race condition-a ako se više tredova takmiči za pristup susjednim indeksima u nizu. Zbog svega navedenog, paralelizacija Heap Sort algoritma ima vrlo ograničen efekat i primjenjiva je samo djelimično, što znači da se značajna ubrzanja ne mogu očekivati kao kod algoritama poput Merge Sort-a.

```
void heapifyParallel(int* arr, int n, int i) {
    int largest = i;
    int l = 2 * i + 1;
    int r = 2 * i + 2;

    if (l < n && arr[l] > arr[largest])
        largest = l;
    if (r < n && arr[r] > arr[largest])
        largest = r;

    if (largest != i) {
        std::swap(arr[i], arr[largest]);
        heapifyParallel(arr, n, largest);
    }
}

void heapSortParallel(int* arr, int n) {
    #pragma omp parallel for
    for (int i = n / 2 - 1; i >= 0; i--)
        heapifyParallel(arr, n, i);

    for (int i = n - 1; i >= 0; i--) {
        std::swap(arr[0], arr[i]);
        heapifyParallel(arr, i, 0);
    }
}
```

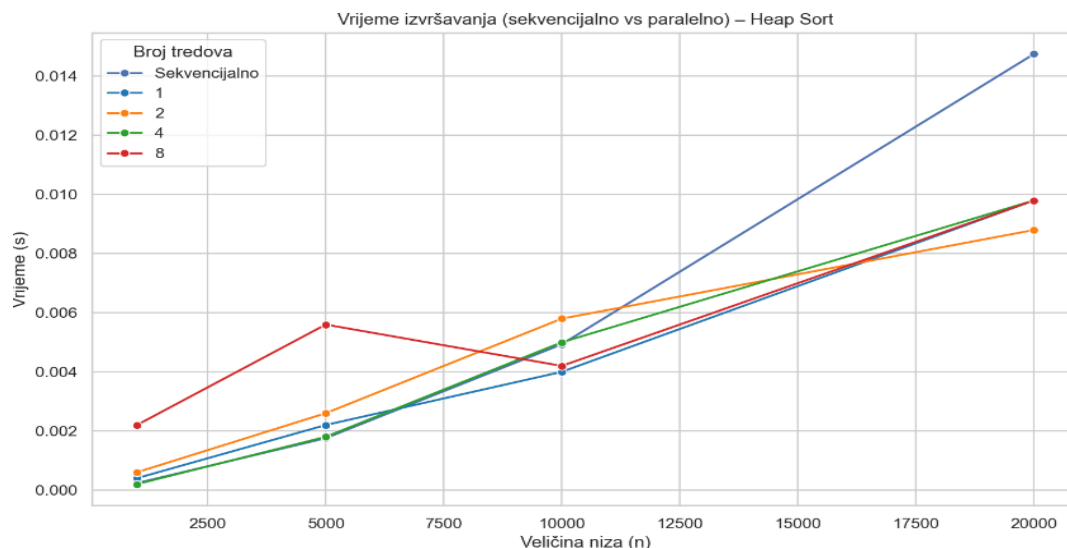
47.2 - paralelna implementacija Heap sort alg. u C++

7.4.1 Analiza rezultata

Dobijeni grafikoni predstavljaju vizuelnu potvrdu ograničenosti i neučinkovitosti paralelizacije Heap Sort algoritma. Takođe, na slici 47.3 imamo i tabelarni prikaz performansi paralelnog Heap sort algoritma.

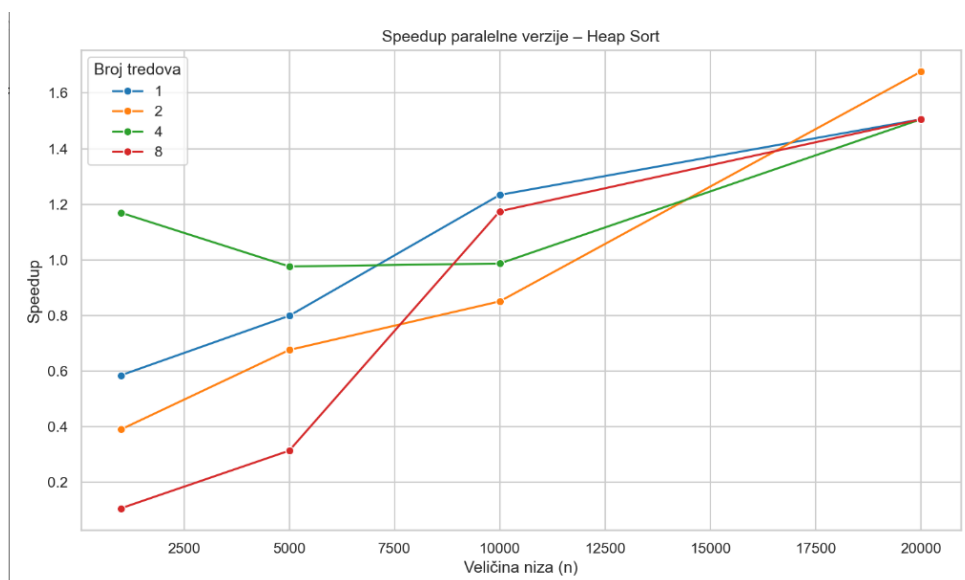
n	threads	seq_time	par_time	speedup	efficiency	cost
1000.0	1.0	0.000234	0.0004	0.584723	0.584723	0.0004
1000.0	2.0	0.000234	0.0006	0.389831	0.194915	0.0012
1000.0	4.0	0.000234	0.0002	1.169585	0.292396	0.0008
1000.0	8.0	0.000234	0.0022	0.106319	0.01329	0.0176
5000.0	1.0	0.001757	0.0022	0.79877	0.79877	0.0022
5000.0	2.0	0.001757	0.0026	0.675876	0.337938	0.0052
5000.0	4.0	0.001757	0.0018	0.97626	0.244065	0.0072
5000.0	8.0	0.001757	0.0056	0.313801	0.039225	0.0448
10000.0	1.0	0.004934	0.004	1.233536	1.0	0.004
10000.0	2.0	0.004934	0.0058	0.850713	0.425356	0.0116
10000.0	4.0	0.004934	0.005	0.986824	0.246706	0.02
10000.0	8.0	0.004934	0.0042	1.174787	0.146848	0.0336
20000.0	1.0	0.014755	0.0098	1.50558	1.0	0.0098
20000.0	2.0	0.014755	0.0088	1.676674	0.838337	0.0176
20000.0	4.0	0.014755	0.0098	1.505573	0.376393	0.0392
20000.0	8.0	0.014755	0.0098	1.505565	0.188196	0.0784

47.3 - Tabelarni prikaz rezultata performansi



47.4 - Grafikon koji prikazuje vrijeme izvršavanja u odnosu na veličinu ulaznog niza

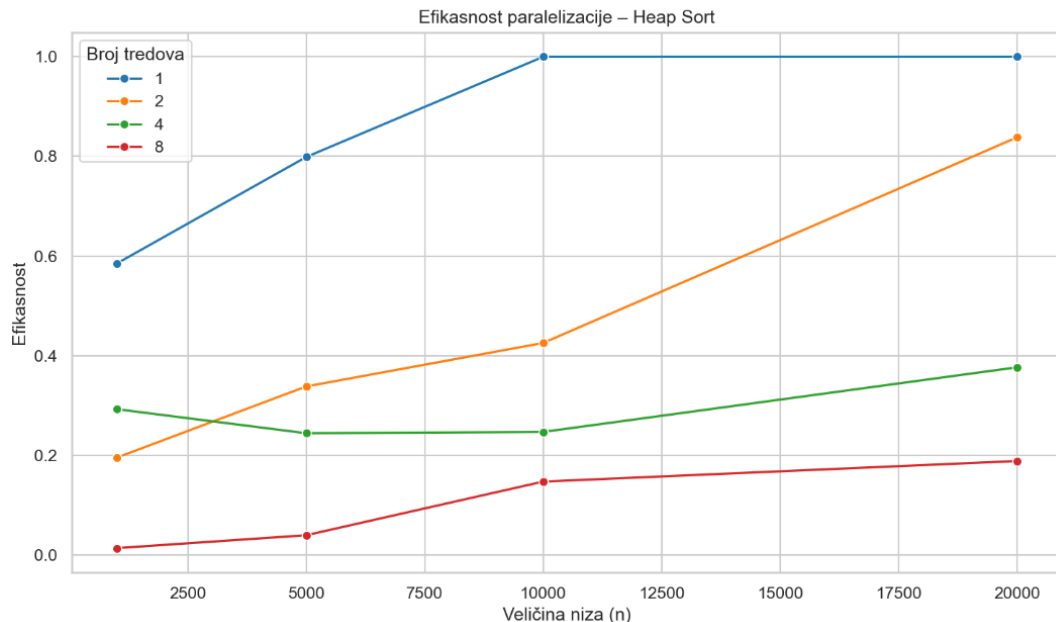
Na grafikonu 47.4, koji prikazuje vrijeme izvršavanja sekvencijalne i paralelne verzije u odnosu na veličinu ulaznog niza, uočava se da su razlike između dvije verzije minimalne. Paralelna verzija ne pokazuje značajno bolje performanse, a u nekim slučajevima je čak i sporija od sekvencijalne. Ovo se može objasniti činjenicom da je jedina faza u kojoj se može primijeniti paralelizacija — izgradnja heap-a (`buildHeap`) — vrlo kratka i ne dominira ukupnim vremenom izvršavanja. Faza sortiranja, koja je vremenski najintenzivnija i predstavlja glavni dio algoritma, ne može biti paralelizovana jer svaka iteracija direktno zavisi od prethodne: nakon svakog pomjeranja maksimalnog elementa sa vrha heap-a na kraj niza, neophodno je ažurirati heap prije nego što se nastavi dalje. Ova zavisnost stvara sekvencijalni lanac operacija koji onemogućava istovremeno izvršavanje.



47.5 - Grafikon koji prikazuje speedup paralelne verzije u zavisnosti od velicine i br. niti

Na drugom grafikonu 47.5, koji prikazuje speedup paralelne verzije, jasno je da vrijednosti ostaju vrlo niske i daleko od teoretskog maksimuma. U većini slučajeva, speedup jedva prelazi vrijednost 1, što znači da su paralelna i sekvencijalna verzija

gotovo jednako brze. U nekim testiranjima, naročito kod manjih nizova, paralelna verzija čak pokazuje negativan speedup, odnosno radi sporije, što se može objasniti dodatnim overhead-om koji nastaje prilikom kreiranja tredova i njihovom koordinacijom. Ovo dodatno potvrđuje da je pokušaj paralelizacije u ovom slučaju više opterećenje nego doprinos ubrzanju.



47.6 - Grafikon koji prikazuje efikasnost paralelizacije u zavisnosti od velicine niza i br. niti

Treći grafikon 47.6 prikazuje efikasnost paralelizacije, odnosno koliko se efektivno koristi dodatni broj tredova. Idealna efikasnost bi bila blizu 1 (100%), ali dobijeni rezultati pokazuju izrazito nisku efikasnost, često ispod 0.3, pa čak i ispod 0.1 pri većem broju tredova. To znači da dodavanje više tredova ne doprinosi proporcionalno ubrzanju, već da se resursi računarskog sistema slabo koriste. Uzrok tome je što većina posla i dalje ostaje u sekvencijalnom dijelu algoritma, dok se paralelni dio svede na kratak i statički ograničen segment koji ne može skalirati s brojem tredova.

Na osnovu sva tri grafikona može se zaključiti da Heap Sort algoritam nije pogodan za paralelizaciju, i da rezultati jasno potvrđuju teorijsko ograničenje koje proizlazi iz prirode algoritma. Grafički prikazi su ovdje korišćeni ne da pokažu prednosti paralelizacije, već upravo suprotno — da eksperimenti potkrijepe tezu da se paralelizacijom ovog algoritma ne postižu značajna poboljšanja, te da bi njegovo paralelno izvođenje u praksi najčešće bilo neefikasno ili čak kontraproduktivno.

8. ZAKLJUČAK

Sortiranje predstavlja ključnu operaciju u obradi podataka, kojom se elementi unutar skupa raspoređuju prema unaprijed definisanom redoslijedu – bilo da je riječ

o uzlaznom, silaznom, abecednom ili nekom drugom kriterijumu. U savremenim računski zahtjevnim okruženjima, sekvencijalni algoritmi za sortiranje često nijesu dovoljno efikasni kada se radi o velikim količinama podataka. Upravo zbog toga se poseže za paralelnim algoritmima, koji omogućavaju kraće vrijeme izvršavanja i bolju iskorišćenost dostupnih resursa.

U prethodnim poglavljima analizirali smo nekoliko najpoznatijih i najčešće korišćenih algoritama sortiranja. Pokazalo se da se određeni algoritmi, kao što su Bubble sort, Merge sort i Quick sort, mogu uspješno paralelizovati, čime se značajno povećava njihova efikasnost. S druge strane, algoritmi poput Selection sorta, Insertion sorta i Heap sorta, zbog svoje strukturne zavisnosti i sekvencijalne prirode, ne pružaju značajne benefite u paralelnom izvođenju.

U ovom radu detaljno smo objasnili način funkcionisanja pomenutih algoritama i razmotrili potencijale i ograničenja njihove paralelizacije. Praktična implementacija paralelnih rješenja i njihova evaluacija u okviru projektnog zadatka potvrdila je i demonstrirala teorijske činjenice.

LITERATURA

- [1] „Algoritmi sortiranja“, Škola koda, pristupljeno 29. mart 2025, dostupno na: <https://skolakoda.github.io/algoritmi-sortiranja>.
- [2] „Bubble Sort Algorithm“, Programiz, pristupljeno 29. mart 2025, dostupno na: <https://www.programiz.com/dsa/bubble-sort>.
- [3] Barry Wilkinson i Michael Allen, Parallel Programming: Techniques and Applications Using Networked Workstations and Parallel Computers, Second Edition, Prentice Hall, 2005, str. 303–311.
- [4] “Chapter 9: Odd-Even Transposition Sort,” Parallel and Distributed Algorithms, pristupljeno: 6. april 2025, dostupno na: <http://users.atw.hu/parallelcomp/ch09lev1sec3.html>
- [5] „Selection Sort Algorithm“, Programiz, pristupljeno 29. mart 2025, dostupno na: <https://www.programiz.com/dsa/selection-sort>.
- [6] „Insertion Sort Algorithm“, Programiz, pristupljeno 29. mart 2025, dostupno na: <https://www.programiz.com/dsa/insertion-sort>.
- [7] “Merge Sort,” Programiz, pristupljeno 31. mart 2025, dostupno na: <https://www.programiz.com/dsa/merge-sort>
- [8] Barry Wilkinson i Michael Allen, Parallel Programming: Techniques and Applications Using Networked Workstations and Parallel Computers, Second Edition, Prentice Hall, 2005, str. 332–324.
- [9] “Quick sort,” Programiz, pristupljeno 01. april 2025, dostupno na: <https://www.programiz.com/dsa/merge-sort>
- [10] Barry Wilkinson i Michael Allen, Parallel Programming: Techniques and Applications Using Networked Workstations and Parallel Computers, Second Edition, Prentice Hall, 2005, str. 334–336
- [11] “Heap sort,” Programiz, pristupljeno 01. april 2025, dostupno na: <https://www.programiz.com/dsa/merge-sort>

- [12] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 3rd ed., MIT Press, 2009, str. 151–163; K. Batcher, "Design of a Parallel, Hardware Priority Queue," IEEE Transactions on Computers, vol. C-29, no. 10, 1980, str. 833–840.